UNIVERSIDAD TECNOLÓGICA DEL PERÚ

FACULTAD DE INGENIERÍA

Carrera de Ingeniería de Sistemas e Informática

CURSO: DISEÑO DE PATRONES

PROYECTO FINAL – AVANCE N.° 2 (CAPÍTULOS I AL VI)

Sistema de Gestión Logística y Seguimiento de Envíos

“AppLogistica” para LogiRuta Perú S.A.C.

Aplicación de principios SOLID y patrones GoF creacionales, estructurales y de comportamiento

Integrantes:

[Apellidos y nombres del integrante 1] – [Código U]
[Apellidos y nombres del integrante 2] – [Código U]
[Apellidos y nombres del integrante 3] – [Código U]
[Apellidos y nombres del integrante 4] – [Código U]
[Apellidos y nombres del integrante 5] – [Código U]
[Apellidos y nombres del integrante 6] – [Código U]

Docente: [Nombre del docente]
Sección: [Código de sección]

Ciclo académico: 2026 – II
Fecha: Octubre de 2026

Lima – Perú
2026

ÍNDICE

Presentación del avance....3
CAPÍTULO I. INTRODUCCIÓN....4
1.1 Descripción de la empresa o negocio....4
1.2 Problemática actual....6
1.3 Objetivos....8
CAPÍTULO II. ANÁLISIS DEL NEGOCIO....9
2.1 Requerimientos Funcionales....10
2.2 Requerimientos No Funcionales....11
2.3 Casos de Uso....12
CAPÍTULO III. APLICACIÓN DE PRINCIPIOS SOLID....19
3.1 Principio de Responsabilidad Única (SRP)....21
3.2 Principio Abierto/Cerrado (OCP)....23
3.3 Principio de Sustitución de Liskov (LSP)....26
3.4 Principio de Segregación de Interfaces (ISP)....27
3.5 Principio de Inversión de Dependencias (DIP)....30
Beneficios obtenidos....32
CAPÍTULO IV. PATRONES CREACIONALES....33
4.1 Patrón Singleton....33
4.2 Patrón Factory....36
4.3 Patrón Abstract Factory....38
4.4 Patrón Builder....40
4.5 Patrón Prototype....43
Comparación de patrones creacionales....45
CAPÍTULO V. PATRONES ESTRUCTURALES....47
5.1 Patrón Adapter....47
5.2 Patrón Facade....51
5.3 Patrón Decorator....54
5.4 Patrón Composite....57
5.5 Patrón Proxy....61
5.6 Patrón Bridge....65
Comparación de patrones estructurales....69
CAPÍTULO VI. PATRONES DE COMPORTAMIENTO....71
6.1 Patrón Observer....73
6.2 Patrón State....80
6.3 Patrón Command....88
6.4 Patrón Memento....97
6.5 Aplicación complementaria: State y Observer en el empleado (ejercicio de la Sesión 12)....102

Comparación de patrones de comportamiento....108
CAPÍTULO VII. ANTIPATRONES IDENTIFICADOS....110
7.1 Concepto....110
7.2 Antipatrones encontrados....110
7.3 Solución propuesta....110
CAPÍTULO VIII. PATRONES GRASP....111
8.1 Experto en Información....111
8.2 Creador....111
8.3 Controlador....111
8.4 Bajo Acoplamiento....111
8.5 Alta Cohesión....111
8.6 Fabricación Pura....111
8.7 Polimorfismo....111
CAPÍTULO IX. IMPLEMENTACIÓN DEL SISTEMA....112
9.1 Arquitectura propuesta....112
9.2 Diagrama de clases....112
9.3 Diagrama de secuencia....113
9.4 Base de datos....113
CAPÍTULO X. PRUEBAS DEL SISTEMA....115
10.1 Casos de prueba....115
10.2 Evidencias....116
CAPÍTULO XI. CONCLUSIONES....119
CAPÍTULO XII. RECOMENDACIONES....120
REFERENCIAS BIBLIOGRÁFICAS....121
ANEXOS....122
Anexo A. Guía rápida de instalación y ejecución....122

Presentación del avance

El presente documento constituye el **segundo avance del Proyecto Final** del curso Diseño de Patrones. En él se analiza el caso empresarial *"Plataforma de logística y seguimiento de envíos"* y se diseña e implementa la solución **AppLogistica**, una aplicación de escritorio desarrollada en **Java con Apache NetBeans** que conserva la misma arquitectura y estructura de paquetes del proyecto de referencia del curso, *AppEmpresa* (uni.controller, uni.dao, uni.database, uni.entity, uni.formularios, uni.reportes, uni.service y uni.util), sobre una base de datos MySQL 8 construida con el mismo molde (tabla control de correlativos, procedimientos almacenados y transacciones JDBC).

El alcance de esta entrega cubre los contenidos desarrollados en clase hasta la **semana 13** (patrones Command y Memento):

- **Capítulo I y II:** descripción de la empresa, problemática, objetivos, requerimientos funcionales y no funcionales, y casos de uso.
- **Capítulo III:** aplicación de los cinco principios SOLID sobre el diseño de AppLogistica.
- **Capítulo IV:** patrones creacionales Singleton, Factory, Abstract Factory, Builder y Prototype.
- **Capítulo V:** patrones estructurales Adapter, Facade, Decorator, Composite, Proxy y Bridge.
- **Capítulo VI (nuevo):** patrones de comportamiento Observer, State, Command y Memento, con diagramas UML, implementación y pruebas.

Los capítulos VII al XII (antipatrones, GRASP, implementación completa, pruebas, conclusiones y recomendaciones) se mantienen en la estructura del informe y se desarrollarán en las siguientes entregas (semanas 14 a 18), aunque los capítulos IX y X ya incluyen un avance. Siguiendo la regla del proyecto, **cada patrón responde a tres preguntas**: ¿qué problema de la empresa se tiene?, ¿por qué el patrón lo resuelve? y ¿cómo se implementó en el sistema?

Todo el código presentado pertenece al proyecto **AppLogistica**, que compila sin errores y se **ejecuta como aplicación de escritorio** conectada a MySQL (ventana de acceso, menú principal y formularios de clientes, empleados, registro de envíos, seguimiento, manifiestos, reportes y alertas). Las evidencias de funcionamiento son de dos tipos: la salida real de las clases de prueba `uni.util.PruebaPatrones` y `uni.util.PruebaComportamiento` (equivalentes a `uni.database.prueba` de AppEmpresa, que usan DAO en memoria) y capturas de la aplicación ejecutándose contra la base de datos **logistica**. Las novedades de este avance respecto del anterior son: el Capítulo VI completo, la capa `uni.formularios`, los DAO de MySQL de todos los módulos y la actualización del script de base de datos.

CAPÍTULO I. INTRODUCCIÓN

1.1 Descripción de la empresa o negocio

LogiRuta Perú S.A.C. es una empresa peruana de logística de última milla y transporte de carga ligera, fundada en el año 2012 en la ciudad de Lima. Se dedica al recojo, clasificación, distribución y entrega de paquetes y documentos a nivel nacional. En sus inicios atendía principalmente la mensajería documentaria de estudios jurídicos y entidades financieras; sin embargo, el crecimiento del comercio electrónico en el país —acelerado a partir de 2020— transformó su cartera: hoy más del 60 % de su volumen proviene de tiendas virtuales y clientes corporativos que despachan pedidos diariamente.

La empresa opera con una sede central en San Isidro, un almacén de clasificación en el Callao, tiendas de recojo en Lima Metropolitana y un hub regional en Huancayo, desde donde atiende el corredor de la Sierra Central. Cuenta con alrededor de 320 colaboradores, una flota de 85 unidades (motos, furgonetas y camiones) y procesa en promedio **9 500 envíos diarios**, con picos superiores a **16 000 envíos** en campañas como Cyber Wow, Día de la Madre y Navidad.

Tabla 1
Datos generales de la empresa

Campo	Descripción
Nombre de la empresa	LogiRuta Perú S.A.C. (nombre ficticio para fines académicos)
Rubro	Servicios logísticos: courier, paquetería, mensajería y transporte interprovincial de carga ligera
Sede principal	Av. Javier Prado Este 1450, San Isidro, Lima
Infraestructura	1 agencia central, 1 almacén de clasificación (Callao), 3 tiendas de recojo, 1 hub regional (Huancayo)
Colaboradores	320 aproximadamente (operaciones, comercial, administración y TI)
Flota	85 unidades: 48 motos, 30 furgonetas y 7 camiones
Volumen	9 500 envíos diarios en promedio; picos de 16 000 en campañas
Clientes	Personas naturales, tiendas virtuales (e-commerce) y clientes corporativos con despacho masivo

Servicios que ofrece. El caso empresarial describe cinco líneas de servicio, cada una con condiciones distintas de tiempo, costo, prioridad, cobertura y tratamiento de incidencias. La Tabla 2 resume las reglas de negocio que el sistema debe soportar; estas reglas son la base de la jerarquía de clases `IServicioEnvio` que se diseña en los capítulos III y IV.

Tabla 2
Condiciones de cada tipo de servicio

Servicio	Tiempo de entrega	Tarifa base	Prioridad	Cobertura	Tratamiento de incidencias
Entrega express	24 horas	S/ 18.00 + S/ 3.50 por kg (máx. 30 kg)	1 (máxima)	Lima y Callao	Alerta inmediata al supervisor y al cliente; flujo sin etapa de consolidación
Envío estándar	3 días	S/ 10.00 + S/ 1.80 por kg	3	Nacional urbana	Pasa por consolidación en almacén; reprogramación automática
Mensajería programada	Fecha pactada	S/ 12.00 + S/ 2.20 por kg	2	Lima y capitales de región	La fecha comprometida es obligatoria; se notifica 24 h antes
Recojo en tienda	2 días	S/ 6.00 + S/ 1.20 por kg	4	Red de tiendas y agencias	El destinatario retira en ventanilla; no hay etapa “en ruta”
Transporte interprovincial	5 días	S/ 25.00 + S/ 2.50 por kg (máx. 500 kg)	3	Corredores interprovinciales	Doble clasificación (origen y hub de destino); incidencias climáticas frecuentes

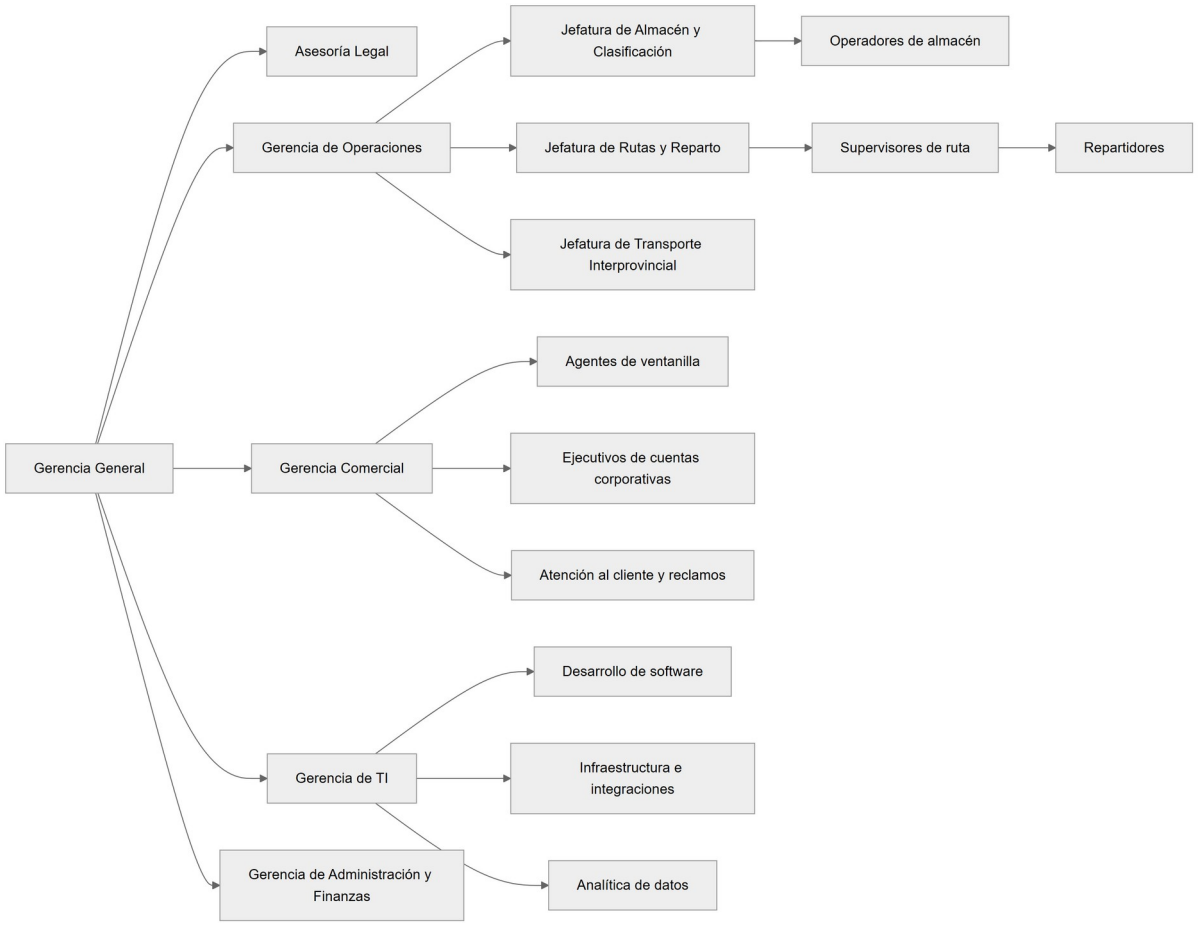
Misión. Conectar a personas y empresas de todo el Perú mediante un servicio de envíos rápido, seguro y trazable, que brinde información confiable en cada etapa del recorrido y contribuya al crecimiento del comercio electrónico nacional.

Visión. Ser, al año 2030, la empresa logística de referencia en el Perú por la confiabilidad de su información en tiempo real, la puntualidad de sus entregas y la capacidad de integrarse tecnológicamente con sus clientes y aliados.

Valores. Confiabilidad, puntualidad, transparencia de la información, orientación al cliente e innovación tecnológica.

Organigrama básico. La Figura 1 muestra la estructura organizacional. Se resaltan las áreas cuyos colaboradores son usuarios directos del sistema: agentes de ventanilla, operadores de almacén, supervisores de ruta y repartidores (Gerencia de Operaciones y Comercial), así como la Gerencia de TI, responsable del desarrollo y de las integraciones externas.

Figura 1
Organigrama básico de LogiRuta Perú S.A.C.



Nota. Elaboración propia del grupo de trabajo.

1.2 Problemática actual

LogiRuta Perú opera actualmente con un sistema heredado de escritorio desarrollado hace más de diez años, complementado con hojas de cálculo y mensajería informal entre el personal. Este sistema fue diseñado para un volumen de mensajería documentaria y no acompañó la expansión del negocio. A partir de entrevistas con los jefes de área y de la revisión del sistema actual, el grupo identificó los siguientes problemas:

a) Procesos manuales. Los repartidores registran los estados de entrega en una planilla impresa que se digita al final del turno. Como consecuencia, el estado de un envío se actualiza con **4 a 6 horas de retraso**, el cliente consulta información desactualizada y el centro de atención recibe en promedio 600 llamadas diarias del tipo “¿dónde está mi paquete?”. Los clientes corporativos, por su parte, envían sus pedidos en archivos Excel que un agente de ventanilla transcribe uno por uno.

b) Duplicidad de información. Los datos de un mismo envío se registran en el sistema de ventanilla, en la hoja de cálculo del almacén y en la planilla del repartidor. No existe una única

fuente de verdad: durante el último trimestre, el 7 % de los reclamos se originó por diferencias entre el estado informado al cliente y el estado real del paquete.

c) Baja escalabilidad. Cada tipo de servicio se programó como una copia del código del servicio estándar con pequeñas modificaciones (cálculo de tarifa, plazos y estados). Agregar un nuevo servicio —por ejemplo, la entrega “en el mismo día” que solicita el área comercial— exige modificar decenas de sentencias `if/switch` dispersas. En campañas de alto volumen el sistema abre una conexión a la base de datos por cada operación, lo que provoca saturación y tiempos de respuesta superiores a 8 segundos.

d) Dependencia excesiva entre módulos. Los formularios contienen directamente las consultas SQL, las reglas de tarifas y el envío de notificaciones. La integración con el proveedor de SMS está escrita “en duro” dentro del formulario de registro; cuando dicho proveedor sufrió una caída de dos días, la empresa no pudo notificar a sus clientes porque reemplazarlo implicaba modificar y recompilar varios formularios.

e) Trazabilidad insuficiente. Los cambios de estado pueden sobrescribirse sin dejar historial; no se sabe quién modificó un estado ni cuándo, lo que dificulta la atención de reclamos y la auditoría interna. Además, cualquier usuario autenticado puede cambiar el estado de cualquier envío.

f) Información no aprovechada para decisiones. La gerencia no cuenta con reportes de zonas con más incidencias, cumplimiento de tiempos prometidos ni productividad de reparto. Los reportes se arman manualmente en Excel una vez al mes.

Tabla 3
Relación entre problemas, impacto y enfoque de solución

Problema	Impacto en el negocio	Enfoque de diseño propuesto
Procesos manuales	Estados desactualizados, sobrecarga de atención al cliente	Actualización de estados desde el móvil del repartidor; registro masivo con plantillas (Prototype)
Duplicidad de información	Reclamos por información contradictoria	Única base de datos transaccional; fachada de registro (Facade)
Baja escalabilidad	Costo alto para agregar servicios; lentitud en campañas	OCP y Factory para servicios; Singleton para el acceso a datos
Dependencia entre módulos	Imposibilidad de cambiar de proveedor externo	DIP, Adapter y Abstract Factory para integraciones
Trazabilidad insuficiente	Pérdida de confianza del mercado, auditoría débil	Historial inmutable (ISP), Proxy de protección y auditoría
Información no aprovechada	Decisiones sin datos	Reportes estratégicos desacoplados del formato (Bridge)

1.3 Objetivos

Objetivo General

Diseñar una solución de software para la gestión logística y el seguimiento de envíos de LogiRuta Perú S.A.C., aplicando patrones de diseño y principios SOLID sobre una arquitectura en capas basada en el proyecto de referencia AppEmpresa, que permita controlar los envíos en tiempo real, integrarse con proveedores externos sin dependencia de uno solo y facilitar el crecimiento del negocio.

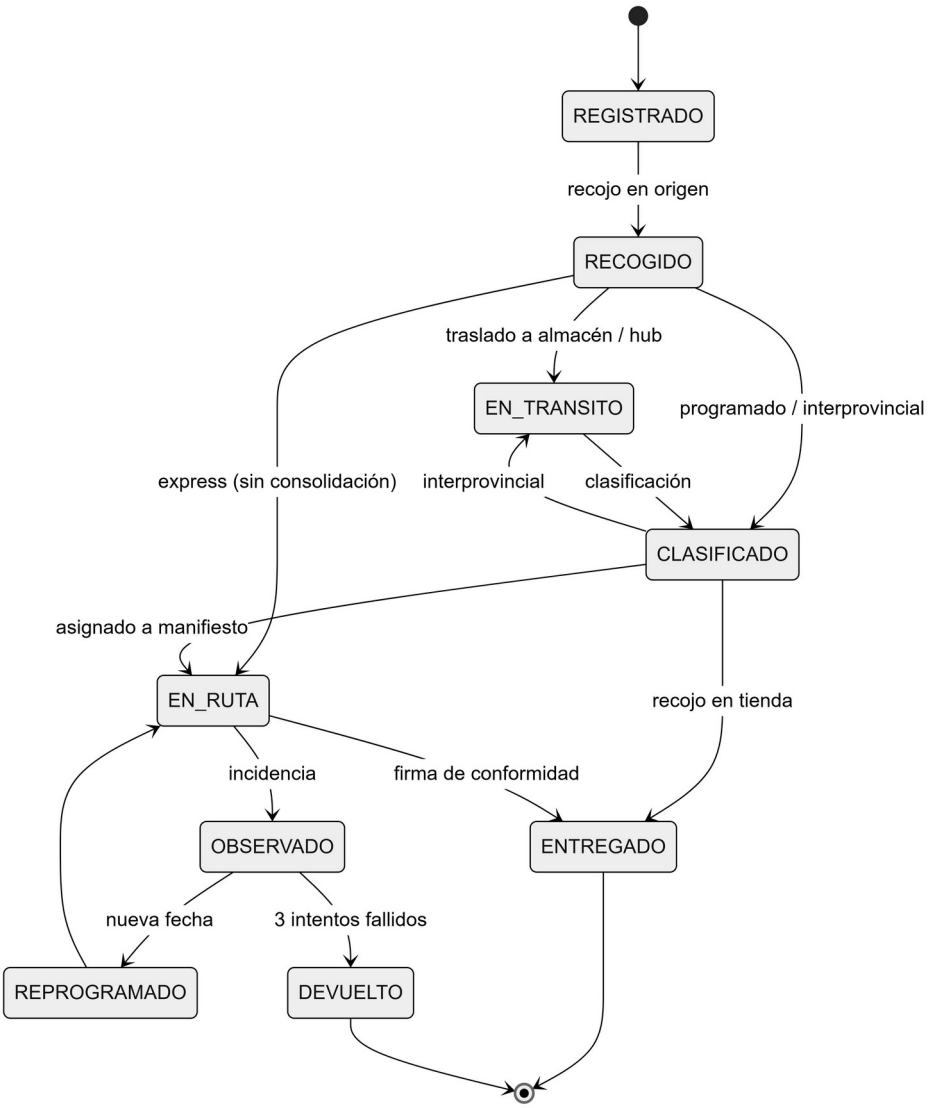
Objetivos Específicos

1. **Analizar los requerimientos** del caso empresarial, identificando actores, requerimientos funcionales y no funcionales, y modelándolos mediante casos de uso UML.
2. **Aplicar los principios SOLID** (SRP, OCP, LSP, ISP y DIP) para corregir las deficiencias de acoplamiento y cohesión del sistema actual y del molde de referencia.
3. **Implementar patrones GoF** creacionales (Singleton, Factory, Abstract Factory, Builder y Prototype) y estructurales (Adapter, Facade, Decorator, Composite, Proxy y Bridge), justificando cada uno con un problema concreto de la empresa; los patrones de comportamiento se abordarán en la siguiente entrega.
4. **Aplicar patrones GRASP** para asignar responsabilidades a las clases del sistema (a desarrollarse en el Capítulo VIII).
5. **Identificar y evitar antipatrones** como *God Object*, *Spaghetti Code* y *Copy-Paste Programming* presentes en el sistema heredado (a desarrollarse en el Capítulo VII).

CAPÍTULO II. ANÁLISIS DEL NEGOCIO

Antes de listar los requerimientos, es necesario comprender el proceso central del negocio: el **ciclo de vida de un envío**. Según el caso empresarial, un envío puede atravesar los estados *registrado, recogido, en tránsito, clasificado, en ruta, entregado, observado, reprogramado* o *devuelto*, y el recorrido depende del servicio contratado: un paquete express no pasa por consolidación, mientras que un envío interprovincial se clasifica dos veces (en origen y en el hub de destino). La Figura 2 modela estas transiciones; todas ellas deben quedar registradas en un historial que tanto el cliente como el personal interno puedan consultar.

Figura 2
Diagrama de estados del ciclo de vida de un envío



Nota. Elaboración propia del grupo de trabajo.

2.1 Requerimientos Funcionales

Los requerimientos funcionales se obtuvieron a partir del caso empresarial y de las necesidades de cada actor. Se priorizaron con la escala **Alta / Media / Baja** según su impacto en la operación diaria.

Tabla 4
Requerimientos funcionales del sistema AppLogistica

Código	Requerimiento	Descripción	Actor principal	Prioridad
RF01	Autenticar usuarios por rol	El sistema valida usuario y contraseña y habilita solo las opciones del rol (ventanilla, almacén, supervisor, repartidor, cliente corporativo, administrador).	Todos	Alta
RF02	Registrar clientes	Registrar, actualizar, buscar y listar clientes naturales (DNI) y corporativos (RUC), con correlativo automático C0001, C0002...	Agente de ventanilla	Alta
RF03	Registrar envíos	Registrar un envío con remitente, destinatario, direcciones, peso, tipo de servicio y opciones adicionales; genera el código LR######### y la guía.	Agente de ventanilla	Alta
RF04	Cotizar tarifa	Calcular el costo según el servicio (cargo fijo + costo por kg) y los adicionales elegidos (seguro, embalaje frágil, firma digital, rastreo GPS).	Agente / Cliente corporativo	Alta
RF05	Registrar envíos masivos	Permitir al cliente corporativo cargar cientos de pedidos a partir de plantillas de envío reutilizables o de su plataforma de comercio electrónico.	Cliente corporativo	Alta
RF06	Geolocalizar direcciones	Obtener latitud y longitud de la dirección de destino mediante un proveedor de mapas para asignar zona y ruta.	Sistema	Media
RF07	Clasificar y consolidar carga	Agrupar paquetes en sacos y paletas, y estos en el manifiesto de carga de un vehículo, calculando peso y cantidad de bultos.	Operador de almacén	Alta
RF08	Asignar rutas y repartidores	Asignar manifiestos a vehículos, rutas y repartidores por zona y fecha.	Supervisor de ruta	Alta
RF09	Actualizar estado del envío	Cambiar el estado de un envío (individual o de todo un manifiesto) con ubicación y observación, desde escritorio o dispositivo móvil.	Repartidor / Operador	Alta
RF10	Registrar historial de seguimiento	Guardar cada cambio de estado con fecha, hora, usuario y ubicación; el historial no puede modificarse ni eliminarse.	Sistema	Alta

Código	Requerimiento	Descripción	Actor principal	Prioridad
RF11	Consultar seguimiento	Consultar el historial y estado actual de un envío mediante su código.	Destinatario / Cliente	Alta
RF12	Notificar cambios de estado	Enviar notificaciones al destinatario por WhatsApp, SMS o correo al registrar el envío y en cada cambio relevante; el servicio express envía alertas inmediatas.	Sistema	Alta
RF13	Registrar incidencias	Registrar incidencias (dirección incorrecta, ausencia del destinatario, clima, incidente operativo); el envío pasa a OBSERVADO.	Repartidor / Supervisor	Alta
RF14	Reprogramar o devolver envíos	Reprogramar una nueva fecha de entrega o devolver el envío al remitente tras tres intentos fallidos.	Supervisor de ruta	Media
RF15	Registrar firma de conformidad	Capturar la firma digital del receptor y su DNI al entregar, generando un código de verificación.	Repartidor	Media
RF16	Gestionar reclamos	Registrar, atender y cerrar reclamos de clientes vinculados a un envío.	Agente / Destinatario	Media
RF17	Generar reporte de incidencias por zona	Mostrar las zonas con más incidencias en pantalla, CSV (Excel) o PDF.	Gerencia	Media
RF18	Generar reporte de cumplimiento de tiempos	Calcular el porcentaje de envíos entregados dentro del plazo prometido por servicio.	Gerencia	Media
RF19	Generar reporte de productividad de reparto	Mostrar entregas por repartidor, por ruta y por día.	Gerencia / Supervisor	Baja
RF20	Auditar operaciones sensibles	Registrar intentos de cambio de estado (permitidos y denegados) con usuario y rol.	Sistema	Media
RF21	Cambiar de proveedor de integración	Permitir operar con proveedores alternos de mapas, notificaciones y firma ante la caída del proveedor principal.	Administrador	Media

2.2 Requerimientos No Funcionales

Tabla 5
Requerimientos no funcionales

Código	Atributo de calidad	Requerimiento	Métrica de aceptación
RNF01	**Seguridad**	Contraseñas almacenadas con hash BCrypt; control de acceso por rol; el repartidor solo modifica los envíos asignados.	0 contraseñas en texto plano; 100 % de cambios de estado auditados.
RNF02	**Seguridad**	Las sentencias SQL usan `PreparedStatement` o procedimientos almacenados.	0 consultas construidas por concatenación.

Código	Atributo de calidad	Requerimiento	Métrica de aceptación
RNF03	**Escalabilidad**	Agregar un nuevo tipo de servicio o un nuevo proveedor externo sin modificar las clases existentes.	Nuevo servicio = 1 clase nueva + 1 línea en la fábrica.
RNF04	**Escalabilidad**	Soportar picos de campaña.	16 000 envíos/día y 50 usuarios concurrentes.
RNF05	**Mantenibilidad**	Arquitectura en capas con la estructura de paquetes de AppEmpresa; cada clase con una sola responsabilidad.	Ningún formulario contiene SQL; acoplamiento solo a interfaces entre capas.
RNF06	**Mantenibilidad**	Código documentado y probado con clases de prueba independientes de la base de datos.	Cada patrón con una prueba ejecutable.
RNF07	**Rendimiento**	Registro de un envío y consulta de seguimiento rápidos.	≤ 2 s para registrar; ≤ 1 s para consultar (con caché).
RNF08	**Disponibilidad**	Continuidad operativa ante caída de proveedores externos.	Conmutación a proveedores de contingencia sin recompilar.
RNF09	**Trazabilidad**	El historial de estados es inmutable.	Trigger que impide UPDATE/DELETE en `seguimientos`.
RNF10	**Usabilidad**	Interfaz Swing consistente; el repartidor actualiza un estado en máximo 3 pasos.	Prueba con usuarios: ≤ 3 clics por actualización.
RNF11	**Interoperabilidad**	Integración con mapas, firma digital, comercio electrónico y mensajería mediante interfaces propias.	Ninguna clase de negocio importa librerías de terceros directamente.
RNF12	**Portabilidad**	Ejecutable en Windows y Linux con JDK 11 o superior y MySQL 8.	Distribución como JAR desde NetBeans.

2.3 Casos de Uso

Actores del sistema. El caso empresarial señala que cada actor interactúa con el sistema desde una perspectiva distinta. La Tabla 6 los describe.

Tabla 6
Actores del sistema

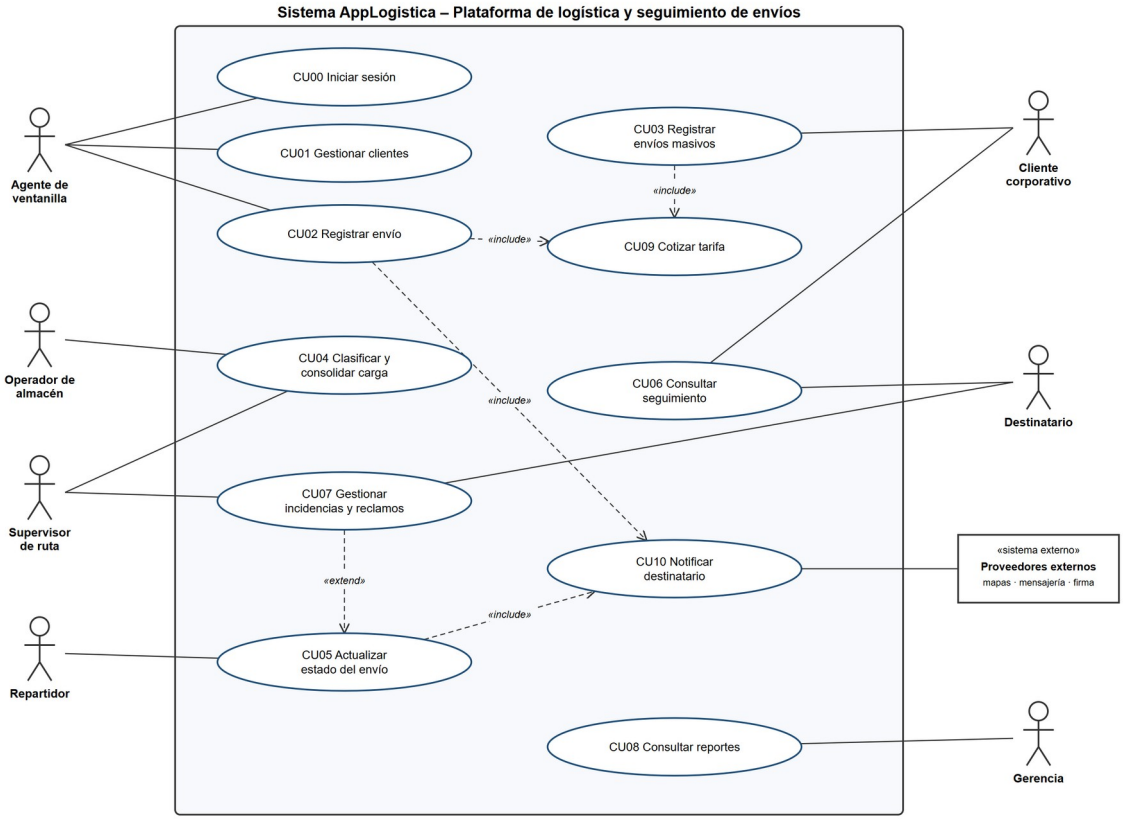
Actor	Tipo	Descripción y objetivo principal
Agente de ventanilla	Interno, primario	Atiende en agencia: registra clientes y envíos, cotiza tarifas y recibe reclamos.
Operador de almacén	Interno, primario	Clasifica paquetes y los consolida en sacos, paletas y manifiestos.
Supervisor de ruta	Interno, primario	Asigna rutas y repartidores, gestiona incidencias, reprogramaciones y devoluciones.
Repartidor	Interno, primario	Actualiza estados desde el móvil, registra incidencias y la firma de conformidad.
Cliente corporativo	Externo, primario	Registra grandes volúmenes de envíos y hace seguimiento de sus despachos.

Actor	Tipo	Descripción y objetivo principal
Destinatario	Externo, primario	Consulta el seguimiento, recibe notificaciones y presenta reclamos.
Gerencia	Interno, primario	Consulta reportes estratégicos para la toma de decisiones.
Proveedores externos	Sistema, secundario	Servicios de mapas, mensajería (WhatsApp, SMS, correo), firma digital y comercio electrónico.

Diagrama UML de casos de uso

La Figura 3 presenta el diagrama general de casos de uso. Los casos *Cotizar tarifa* (CU09) y *Notificar destinatario* (CU10) son reutilizados mediante relaciones **«include»** por el registro de envíos y la actualización de estados; *Gestionar incidencias* **«extiende»** la actualización de estado cuando la entrega no puede completarse. El caso CU00 *Iniciar sesión* es prerrequisito de todos los casos de los actores internos y del cliente corporativo; por legibilidad solo se dibuja su asociación con el agente de ventanilla.

Figura 3
Diagrama general de casos de uso del sistema AppLogistica



Nota. Elaboración propia del grupo de trabajo.

Descripción de Casos de Uso

Tabla 7
CU00 Iniciar sesión

Elemento	Descripción
Código / Nombre	CU00 – Iniciar sesión
Actores	Todos los actores internos y el cliente corporativo
Descripción	Permite al usuario autenticarse y cargar su perfil (rol y sucursal) en la sesión activa.
Precondición	El usuario está registrado y activo.
Flujo básico	1. El usuario ingresa usuario y contraseña en `LoginView`. 2. El sistema verifica las credenciales contra el hash almacenado. 3. El sistema guarda el empleado autenticado en la sesión (`SesionUsuario`). 4. El sistema muestra `PrincipalView` con el menú correspondiente al rol.
Flujos alternos	2a. Credenciales inválidas: se muestra un mensaje y se permite reintentar; al tercer intento fallido se bloquea la cuenta por 15 minutos.
Postcondición	Existe una sesión activa con el rol del usuario.
RF asociados	RF01

Tabla 8
CU01 Gestionar clientes

Elemento	Descripción
Código / Nombre	CU01 – Gestionar clientes
Actor	Agente de ventanilla
Descripción	Registrar, modificar, buscar y listar clientes naturales y corporativos.
Precondición	Sesión iniciada con rol AGENTE_VENTANILLA o ADMINISTRADOR.
Flujo básico	1. El agente selecciona “Clientes” en el menú. 2. Ingresa tipo de cliente, documento (DNI/RUC), razón social, correo y teléfono. 3. El sistema valida el formato del documento y que no exista duplicado. 4. El sistema genera el código correlativo (tabla `control`) y registra al cliente mediante `sp_Cliente_Adicionar`. 5. El sistema muestra el cliente en la lista.
Flujos alternos	3a. Documento duplicado: el sistema muestra los datos del cliente existente para su actualización. 3b. RUC con longitud distinta de 11 dígitos: se solicita corrección.
Postcondición	Cliente registrado o actualizado.
RF asociados	RF02

Tabla 9
CU02 Registrar envío

Elemento	Descripción
Código / Nombre	CU02 – Registrar envío
Actor	Agente de ventanilla (secundario: proveedores externos)

Elemento	Descripción
Descripción	Registra un envío individual, calcula su tarifa, geolocaliza el destino, guarda el primer estado y notifica al destinatario.
Precondición	Cliente remitente registrado; sesión activa.
Flujo básico	1. El agente ingresa remitente, destinatario, dirección de destino, peso y tipo de servicio. 2. El agente marca los adicionales: seguro, frágil, firma digital. 3. El sistema valida los datos y verifica que el servicio admita el envío (peso máximo, fecha programada, tienda de recojo). 4. El sistema geolocaliza la dirección de destino. 5. El sistema calcula la tarifa (*include* CU09). 6. El sistema genera el código LR########, registra el envío y el estado REGISTRADO en el historial. 7. El sistema notifica al destinatario (*include* CU10) e imprime la guía con la fecha estimada.
Flujos alternos	3a. Datos incompletos: se listan los errores y no se registra. 3b. Peso superior al permitido (p. ej. express > 30 kg): se sugiere otro servicio. 4a. Proveedor de mapas no disponible: se usa el proveedor de contingencia.
Postcondición	Envío en estado REGISTRADO con historial inicial.
RF asociados	RF03, RF04, RF06, RF10, RF12

Tabla 10
CU03 Registrar envíos masivos

Elemento	Descripción
Código / Nombre	CU03 – Registrar envíos masivos
Actor	Cliente corporativo
Descripción	Permite registrar cientos de pedidos a partir de una plantilla de envío del cliente o de su tienda virtual.
Precondición	Cliente corporativo con plantillas configuradas (origen, servicio, peso típico, seguro).
Flujo básico	1. El cliente selecciona una plantilla (p. ej. “ECOMMERCE_STD”). 2. Carga el archivo de pedidos o los importa desde su plataforma de comercio electrónico. 3. Por cada pedido, el sistema clona la plantilla y completa destinatario y dirección. 4. El sistema registra cada envío siguiendo el flujo de CU02 (pasos 3 a 7). 5. El sistema muestra el resumen: registrados, observados y costo total.
Flujos alternos	4a. Pedido con datos inválidos: se reporta en el resumen sin detener el lote.
Postcondición	Lote de envíos registrados.
RF asociados	RF05, RF04, RF12

Tabla 11
CU04 Clasificar y consolidar carga

Elemento	Descripción
Código / Nombre	CU04 – Clasificar y consolidar carga
Actores	Operador de almacén, supervisor de ruta

Elemento	Descripción
Descripción	Agrupa paquetes en sacos y paletas por zona, y estos en el manifiesto de un vehículo.
Precondición	Envíos en estado RECOGIDO o EN_TRANSITO en el almacén.
Flujo básico	1. El operador escanea los paquetes y los asigna a un saco de su zona. 2. Los sacos y paquetes voluminosos se ubican en paletas. 3. El supervisor crea el manifiesto del vehículo y agrega paletas, sacos y paquetes sueltos. 4. El sistema calcula el peso total y la cantidad de paquetes y valida la capacidad del vehículo. 5. Al confirmar, el sistema cambia a CLASIFICADO todos los paquetes del manifiesto.
Flujos alternos	4a. Peso mayor que la capacidad: el sistema impide cerrar el manifiesto.
Postcondición	Manifiesto listo para salir a ruta.
RF asociados	RF07, RF08, RF09

Tabla 12
CU05 Actualizar estado del envío

Elemento	Descripción
Código / Nombre	CU05 – Actualizar estado del envío
Actor	Repartidor (también operador y supervisor)
Descripción	Registra el avance del envío (EN_RUTA, ENTREGADO, etc.) con ubicación y observación.
Precondición	El envío está asignado al repartidor.
Flujo básico	1. El repartidor selecciona el envío o el manifiesto. 2. Elige el nuevo estado y el sistema captura la ubicación. 3. El sistema verifica el rol y la asignación del envío. 4. El sistema actualiza el estado y agrega un registro inmutable al historial. 5. Si el estado es ENTREGADO, captura la firma digital y el DNI del receptor. 6. El sistema notifica al destinatario (*include* CU10).
Flujos alternos	3a. Envío no asignado o rol sin permiso: se deniega y se registra en auditoría. 5a. No se pudo entregar: se ejecuta CU07 (*extend*).
Postcondición	Estado actualizado y trazado.
RF asociados	RF09, RF10, RF12, RF15, RF20

Tabla 13
CU06 Consultar seguimiento

Elemento	Descripción
Código / Nombre	CU06 – Consultar seguimiento
Actores	Destinatario, cliente corporativo
Descripción	Muestra el estado actual y el historial completo del envío.
Precondición	El usuario conoce el código del envío.

Elemento	Descripción
Flujo básico	1. El usuario ingresa el código LR########. 2. El sistema recupera el historial (desde caché si fue consultado recientemente). 3. El sistema muestra estados, fechas, ubicaciones y la fecha estimada de entrega.
Flujos alternos	2a. Código inexistente: se muestra un mensaje informativo.
Postcondición	Consulta registrada en auditoría.
RF asociados	RF11, RF20

Tabla 14
CU07 Gestionar incidencias y reclamos

Elemento	Descripción
Código / Nombre	CU07 – Gestionar incidencias y reclamos
Actores	Supervisor de ruta, destinatario (y agente de ventanilla para reclamos presenciales)
Descripción	Registra incidencias de entrega y reclamos; permite reprogramar o devolver el envío.
Precondición	Envío existente.
Flujo básico	1. Se registra el motivo de la incidencia (dirección incorrecta, ausencia, clima u operativo). 2. El sistema cambia el estado a OBSERVADO y registra el historial. 3. El supervisor decide: reprogramar (REPROGRAMADO) o devolver (DEVUELTO). 4. El sistema notifica al destinatario y al remitente.
Flujos alternos	1a. Reclamo del destinatario: se registra con estado ABIERTO y se asigna a atención al cliente.
Postcondición	Incidencia o reclamo registrado.
RF asociados	RF13, RF14, RF16

Tabla 15
CU08 Consultar reportes

Elemento	Descripción
Código / Nombre	CU08 – Consultar reportes
Actor	Gerencia
Descripción	Genera reportes estratégicos en el formato elegido: pantalla, CSV (Excel) o PDF.
Precondición	Existen envíos registrados en el período.
Flujo básico	1. El gerente elige el reporte (incidencias por zona, cumplimiento de tiempos o productividad). 2. Selecciona el período y el formato de salida. 3. El sistema obtiene los datos de las vistas de la base de datos. 4. El sistema genera el reporte en el formato elegido.
Flujos alternos	3a. Sin datos en el período: se informa al usuario.
Postcondición	Reporte generado.

Elemento	Descripción
RF asociados	RF17, RF18, RF19

Trazabilidad requerimientos – casos de uso – patrones. La Tabla 16 relaciona los casos de uso con los patrones que los soportan en el diseño; esta matriz se usa como hilo conductor de los capítulos siguientes.

Tabla 16
Matriz de trazabilidad entre casos de uso y decisiones de diseño

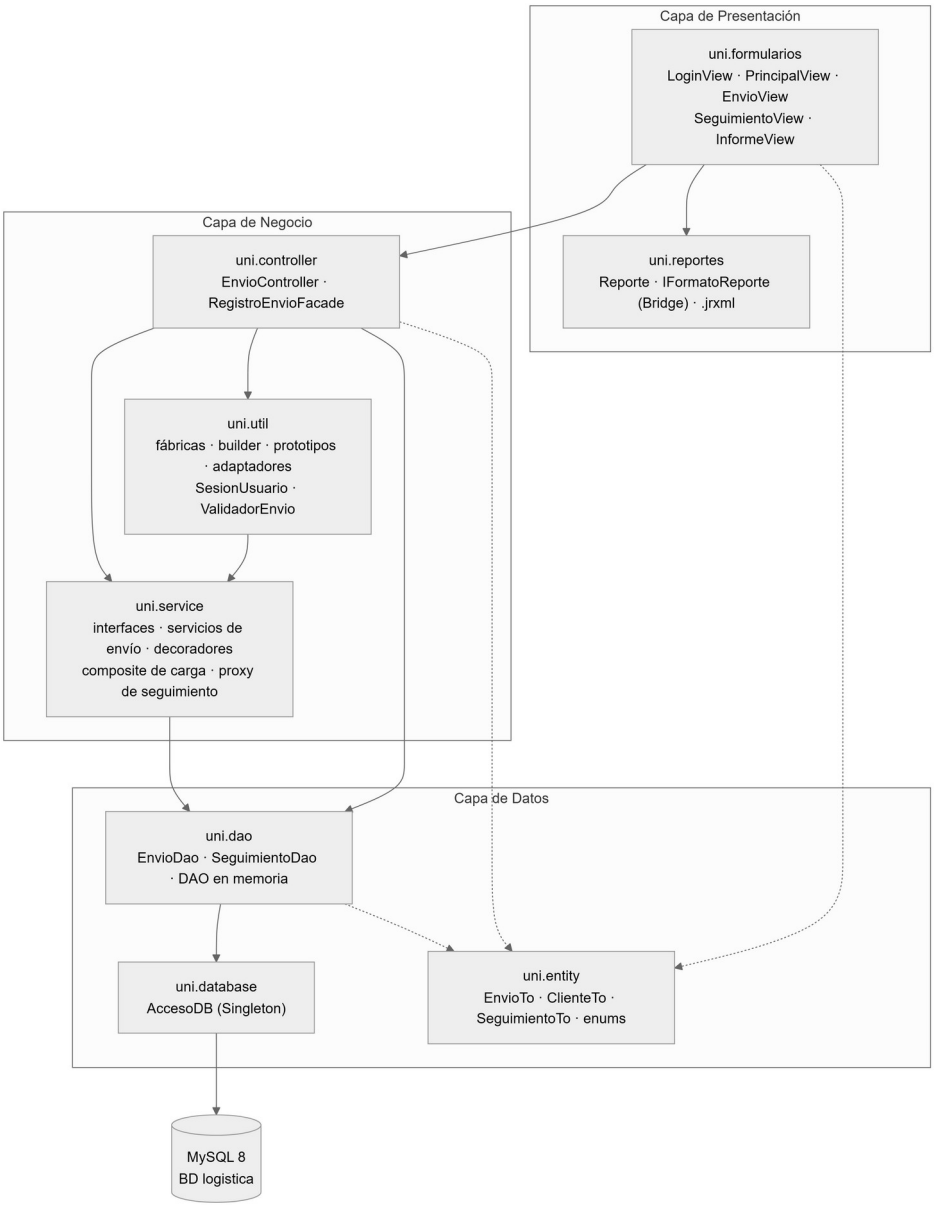
Caso de uso	Principios SOLID	Patrones creacionales	Patrones estructurales
CU00 Iniciar sesión	SRP	Singleton (`SesionUsuario`)	—
CU02 Registrar envío	SRP, OCP, DIP	Factory, Builder, Abstract Factory, Singleton (`AccesoDB`)	Facade, Decorator, Adapter
CU03 Registrar envíos masivos	SRP	Prototype, Builder	Facade, Adapter
CU04 Clasificar y consolidar	OCP	—	Composite
CU05 Actualizar estado	ISP, DIP	Singleton	Proxy, Adapter
CU06 Consultar seguimiento	ISP	—	Proxy (caché)
CU07 Gestionar incidencias	LSP	Factory	Adapter
CU08 Consultar reportes	OCP, DIP	—	Bridge

CAPÍTULO III. APLICACIÓN DE PRINCIPIOS SOLID

Los principios SOLID, formulados por Robert C. Martin, son cinco reglas de diseño orientado a objetos que buscan sistemas fáciles de mantener, extender y probar. En este capítulo cada principio se aplica a un problema concreto de LogiRuta Perú, partiendo tanto del sistema heredado de la empresa como del proyecto de referencia AppEmpresa, cuyo molde se reutiliza.

Contexto arquitectónico. AppLogistica mantiene las capas y paquetes de AppEmpresa (Figura 4). La capa de presentación (`uni.formularios`, `uni.reportes`) solo interactúa con la capa de negocio (`uni.controller`, `uni.service`, `uni.util`), y esta accede a los datos únicamente a través de `uni.dao`, que a su vez obtiene la conexión de `uni.database`. Las entidades de `uni.entity` (Transfer Objects con sufijo *To*, igual que `ClienteTo` o `VentaTo` de AppEmpresa) viajan entre capas.

Figura 4
Arquitectura en capas y estructura de paquetes de AppLogistica (molde AppEmpresa)



Nota. Elaboración propia del grupo de trabajo.

Tabla 17
Responsabilidad de cada paquete en AppLogistica

Paquete	Responsabilidad	Clases principales
`uni.formularios`	Interfaz gráfica Swing (JFrame / JInternalFrame)	LoginView, PrincipalView, EnvioView, SeguimientoView, InformeView
`uni.controller`	Coordinar los casos de uso; punto de entrada desde las vistas	EnvioController, RegistroEnvioFacade
`uni.service`	Contratos (interfaces) y reglas de negocio	IServicioEnvio y servicios, ITarifa y decoradores, IUnidadCarga, ISeguimientoService y proxy
`uni.util`	Componentes transversales: fábricas, constructores, adaptadores, sesión, validación	ServicioEnvioFactory, EnvioBuilder, RegistroPlantillas, adaptadores, SesionUsuario

Paquete	Responsabilidad	Clases principales
`uni.dao`	Acceso a datos con JDBC y procedimientos almacenados	EnvioDao, SeguimientoDao, EnvioDaoMemoria, SeguimientoDaoMemoria
`uni.database`	Conexión a MySQL	AccesoDB (Singleton)
`uni.entity`	Transfer Objects y enumeraciones	EnvioTo, DireccionTo, ClienteTo, EmpleadoTo, SeguimientoTo, IncidenciaTo, EstadoEnvio, TipoServicio, RolUsuario
`uni.reportes`	Reportes estratégicos y plantillas Jasper	Reporte, IFormatoReporte y formatos

3.1 Principio de Responsabilidad Única (SRP)

> **Enunciado:** “Una clase debe tener una sola razón para cambiar.”

Problema identificado

En el sistema heredado de LogiRuta —y del mismo modo en el formulario `VentasView` de AppEmpresa, que supera las 600 líneas— el formulario de registro concentra todas las responsabilidades: lee los campos de la pantalla, valida los datos, calcula la tarifa con condicionales, arma la sentencia SQL, abre la conexión y envía el SMS. El formulario tiene, por tanto, **cinco razones para cambiar**: un cambio en el diseño de la pantalla, en las reglas de validación, en el tarifario, en la base de datos o en el proveedor de SMS obliga a modificar la misma clase, con el riesgo de introducir errores en funcionalidades no relacionadas. Además, ninguna de estas reglas puede probarse sin abrir la ventana.

Código 1
Situación inicial: el formulario EnvioView con múltiples responsabilidades (código del sistema heredado)

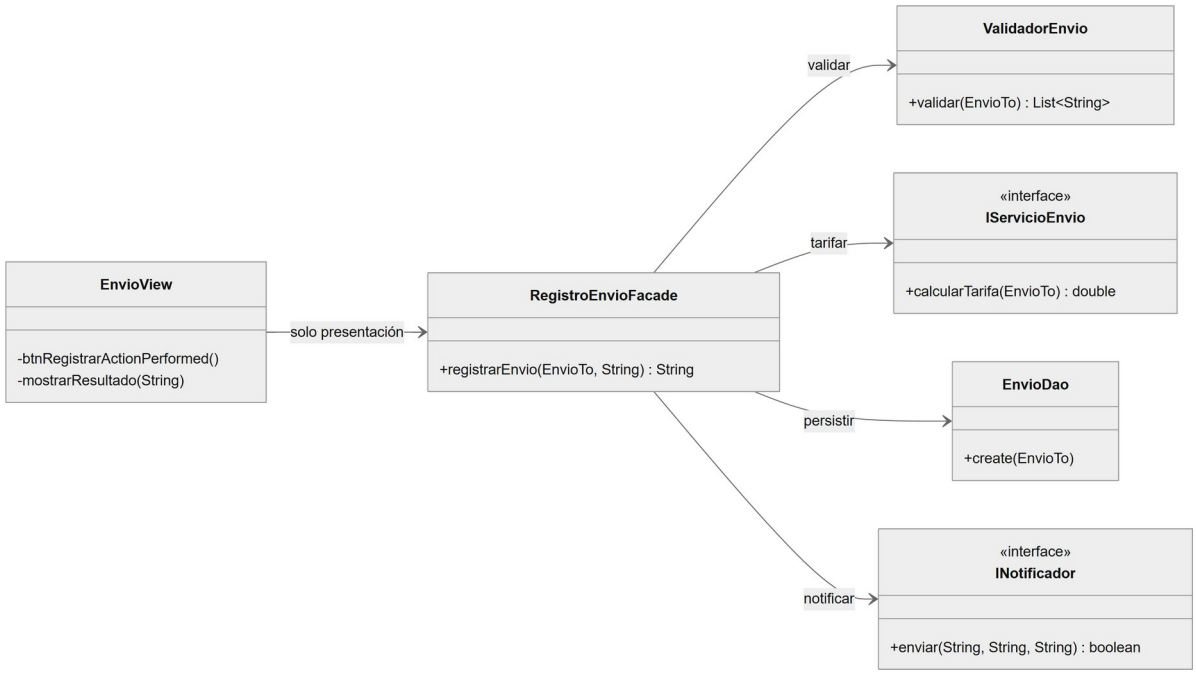
```
private void btnRegistrarActionPerformed(java.awt.event.ActionEvent evt) {
    // 1) Validación mezclada con la vista
    if (txtPeso.getText().isEmpty() || Double.parseDouble(txtPeso.getText()) <= 0) {
        JOptionPane.showMessageDialog(this, "Peso inválido"); return;
    }
    // 2) Regla de negocio (tarifa) dentro del formulario
    double peso = Double.parseDouble(txtPeso.getText()), costo;
    if (cboServicio.getSelectedItem().equals("EXPRESS")) costo = 18 + peso * 3.5;
    else if (cboServicio.getSelectedItem().equals("ESTANDAR")) costo = 10 + peso * 1.8;
    else costo = 25 + peso * 2.5;
    // 3) SQL y conexión dentro del formulario
    Connection cn = DriverManager.getConnection(URL, "root", "1234");
    Statement st = cn.createStatement();
    st.executeUpdate("insert into envios values('" + txtCodigo.getText() + "', ... )");
    // 4) Integración externa dentro del formulario
    new SmsProveedorX().send(txtCelular.getText(), "Su envio fue registrado");
}
```

Solución aplicada

Se separaron las responsabilidades en clases cohesionadas, cada una con un único motivo de cambio: `EnvioView` solo presenta datos; `ValidadorEnvio` (en `uni.util`) solo valida; la jerarquía `IServicioEnvio` solo calcula reglas de servicio; `EnvioDao` solo persiste;

`INotificador` solo notifica; y `RegistroEnvioFacade` (en `uni.controller`) coordina el caso de uso. Como beneficio adicional, la sentencia SQL concatenada (vulnerable a inyección SQL) se reemplaza por procedimientos almacenados con parámetros.

Figura 5
Distribución de responsabilidades del registro de envío aplicando SRP



Nota. Elaboración propia del grupo de trabajo.

Código ejemplo

Código 2
uni.util.ValidadorEnvio – única responsabilidad: validar un envío

```
package uni.util;

/**
 * SRP: única responsabilidad = validar los datos de un envío.
 * Antes esta lógica estaba mezclada dentro del formulario Swing.
 */
public class ValidadorEnvio {

    public List<String> validar(EnvioTo e) {
        List<String> errores = new ArrayList<>();
        if (e.getIdcliente() == null || e.getIdcliente().trim().isEmpty()) {
            errores.add("El cliente remitente es obligatorio.");
        }
        if (e.getServicio() == null) {
            errores.add("Debe seleccionar un tipo de servicio.");
        }
        if (e.getPesoKg() <= 0) {
            errores.add("El peso debe ser mayor que cero.");
        }
        if (e.getNombreDestinatario() == null || e.getNombreDestinatario().trim().isEmpty())
{
            errores.add("El nombre del destinatario es obligatorio.");
        }
        if (e.getTelefonoDestinatario() != null && !e.getTelefonoDestinatario().matches("9\\
d{8}")) {
            errores.add("El celular del destinatario debe tener 9 dígitos y empezar con
9.");
        }
        return errores;
```

```
    }
}
```

Código 3
uni.formularios.EnvioView – el formulario solo delega y muestra el resultado

```
private void btnRegistrarActionPerformed(java.awt.event.ActionEvent evt) {
    try {
        EnvioTo envio = leerFormulario();                    // solo lectura de campos
        String idEmpleado = SesionUsuario.getInstancia().getUsuario().getIdempleado();
        String resumen = fachada.registrarEnvio(envio, idEmpleado);
        JOptionPane.showMessageDialog(this, "Envío registrado: " + resumen);
        limpiarFormulario();
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
    }
}
```

3.2 Principio Abierto/Cerrado (OCP)

> **Enunciado:** “Las entidades de software deben estar abiertas a la extensión, pero cerradas a la modificación.”

Problema identificado

LogiRuta ofrece cinco servicios con reglas diferentes de costo, plazo, prioridad y flujo de estados, y el área comercial planea lanzar la **entrega en el mismo día**. En el sistema heredado, cada regla está implementada mediante cadenas `if/else` o `switch` sobre el nombre del servicio, repetidas en el cálculo de tarifa, en la fecha estimada, en las alertas y en la secuencia de estados. Agregar un servicio exige **modificar código ya probado** en al menos cuatro lugares, lo que contradice la necesidad del caso de “manejar estas variantes sin duplicar lógica innecesariamente”.

Código 4
Situación inicial: la lógica de cada servicio se modifica en cada cambio comercial

```
public double calcularTarifa(String servicio, double peso) {
    switch (servicio) {
        case "EXPRESS":         return 18 + peso * 3.5;
        case "ESTANDAR":        return 10 + peso * 1.8;
        case "PROGRAMADO":      return 12 + peso * 2.2;
        case "RECOJO_TIENDA":   return 6 + peso * 1.2;
        case "INTERPROVINCIAL": return 25 + peso * 2.5;
        // ¿MISMO_DIA? -> hay que editar este método y todos los similares
        default: throw new IllegalArgumentException(servicio);
    }
}
```

Solución aplicada

Se definió la abstracción `IServicioEnvio`, que declara todo lo que varía entre servicios, y una clase abstracta `ServicioEnvioBase` con el algoritmo común (cargo fijo + costo por kilogramo). Cada servicio es una subclase que solo declara sus parámetros y particularidades. El código cliente (fachada, reportes, controladores) trabaja con la interfaz, por lo que **queda**

cerrado a la modificación; para lanzar la entrega en el mismo día basta con **extender** el sistema con una nueva clase `ServicioMismoDia` y registrarla en la fábrica (Capítulo IV).

Figura 6
Jerarquía de servicios de envío abierta a extensión (OCP) y sustituible (LSP)



Nota. Elaboración propia del grupo de trabajo.

Código ejemplo

Código 5
uni.service.IServicioEnvio – abstracción estable

```
package uni.service;

/**
 * Producto del patrón Factory. Cada tipo de servicio encapsula sus reglas
 * de tiempo, costo, prioridad y flujo de estados (OCP + LSP).
 */
public interface IServicioEnvio {

    TipoServicio getTipo();

    /** Precondición explícita del contrato: ¿el servicio acepta este envío? */
    boolean admite(EnvioTo envio);

    double calcularTarifa(EnvioTo envio);

    LocalDate calcularFechaEstimada(EnvioTo envio, LocalDate fechaRegistro);

    List<EstadoEnvio> getFlujoEstados();

    /** 1 = máxima prioridad. */
    int getPrioridad();
}
```

Código 6
uni.service.ServicioEnvioBase – comportamiento común

```
package uni.service;

/**
 * Clase base de los servicios de envío. Define el algoritmo común de tarifa
 * (cargo fijo + costo por kilo) y deja a cada subclase sus parámetros.
 * Ninguna subclase lanza excepciones en estos métodos: las restricciones se
```

```
 * expresan en admite(), que forma parte del contrato (LSP).
 */
public abstract class ServicioEnvioBase implements IServicioEnvio {

    protected abstract double getCargoFijo();

    protected abstract double getCostoPorKg();

    protected abstract int getDiasEntrega();

    @Override
    public boolean admite(EnvioTo envio) {
        return envio != null && envio.getPesoKg() > 0 && envio.getDestino() != null;
    }

    @Override
    public double calcularTarifa(EnvioTo envio) {
        return redondear(getCargoFijo() + envio.getPesoKg() * getCostoPorKg());
    }

    @Override
    public LocalDate calcularFechaEstimada(EnvioTo envio, LocalDate fechaRegistro) {
        return fechaRegistro.plusDays(getDiasEntrega());
    }

    protected static double redondear(double monto) {
        return Math.round(monto * 100.0) / 100.0;
    }
}
```

Código 7
uni.service.ServicioExpress – extensión sin modificar el código existente

```
package uni.service;

/** Entrega express: flujo corto, prioridad máxima, hasta 30 kg. */
public class ServicioExpress extends ServicioEnvioBase {

    public static final double PESO_MAXIMO = 30.0;

    @Override
    public TipoServicio getTipo() { return TipoServicio.EXPRESS; }

    @Override
    protected double getCargoFijo() { return 18.00; }

    @Override
    protected double getCostoPorKg() { return 3.50; }

    @Override
    protected int getDiasEntrega() { return 1; }

    @Override
    public int getPrioridad() { return 1; }

    @Override
    public boolean admite(EnvioTo envio) {
        return super.admite(envio) && envio.getPesoKg() <= PESO_MAXIMO;
    }

    @Override
    public List<EstadoEnvio> getFlujoEstados() {
        // Sin etapa de consolidación: va directo a ruta.
        return Arrays.asList(EstadoEnvio.REGISTRADO, EstadoEnvio.RECOGIDO,
                EstadoEnvio.EN_RUTA, EstadoEnvio.ENTREGADO);
    }
}
```

Evidencia. El mismo envío de 4 kg se cotiza recorriendo todos los servicios con un único bucle polimórfico; no existe ningún `switch` sobre el tipo de servicio en el código cliente.

Salida en consola (ejecución de uni.util.PruebaPatrones):

```
Mismo envío de 4 kg cotizado con cada servicio (polimorfismo):
  EXPRESS          admite=true  tarifa=S/  32.00  prioridad=1  etapas=4
  ESTANDAR         admite=true  tarifa=S/  17.20  prioridad=3  etapas=6
  PROGRAMADO       admite=true  tarifa=S/  20.80  prioridad=2  etapas=5
  RECOJO_TIENDA    admite=true  tarifa=S/  10.80  prioridad=4  etapas=5
  INTERPROVINCIAL  admite=true  tarifa=S/  35.00  prioridad=3  etapas=7
```

3.3 Principio de Sustitución de Liskov (LSP)

> **Enunciado:** “Los objetos de una subclase deben poder reemplazar a los de su superclase sin alterar el correcto funcionamiento del programa.”

Problema identificado

En el sistema heredado, el servicio de *recojo en tienda* se implementó heredando del servicio estándar y sobrescribiendo el método de asignación de ruta para lanzar una excepción, ya que en ese servicio el paquete no sale a reparto. Del mismo modo, en AppEmpresa la clase `ClienteDao` implementa `ICrudDao` pero sus métodos `create`, `update`, `delete` y `find` lanzan `UnsupportedOperationException`. En ambos casos, **el subtipo no cumple el contrato del supertipo**: cualquier código que recorra la lista de servicios o que reciba un `ICrudDao` falla en tiempo de ejecución con objetos que, en apariencia, eran válidos.

Código 8
Violación de LSP en el sistema heredado

```
public class EnvioRecojoTienda extends EnvioEstandar {
    @Override
    public void asignarRuta(Ruta r) {
        throw new UnsupportedOperationException("El recojo en tienda no tiene ruta");
    }
}
// Código cliente que se rompe con un subtipo "legal":
for (EnvioEstandar e : enviosDelDia) {
    e.asignarRuta(rutaZonaSur);    // falla si e es EnvioRecojoTienda
}
```

Solución aplicada

Se rediseñó el contrato para que **cada subtipo pueda cumplirlo completamente**:

- Las diferencias de flujo se expresan como datos (`getFlujoEstados()`), no como métodos prohibidos: el recojo en tienda simplemente no incluye el estado EN_RUTA en su flujo.
- Las restricciones de cada servicio (peso máximo de 30 kg en express, fecha obligatoria en programado, tienda obligatoria en recojo) forman parte del contrato mediante el método `admite(EnvioTo)`. El cliente consulta la precondición antes de usar el servicio; ninguna subclase endurece precondiciones lanzando excepciones inesperadas en `calcularTarifa()` ni en `calcularFechaEstimada()`.
- En la capa de datos, `EnvioDao` (MySQL) y `EnvioDaoMemoria` (pruebas) implementan **todas** las operaciones de `ICrudDao<EnvioTo>` y son intercambiables. Las clases que

no necesitan el contrato completo ya no se ven obligadas a implementarlo gracias a ISP (sección 3.4).

Código ejemplo

Código 9
uni.service.ServicioRecojoTienda – subtipo que respeta el contrato

```
package uni.service;

/**
 * Recojo en tienda: el destinatario retira el paquete en una agencia.
 * No tiene etapa EN_RUTA, pero cumple el mismo contrato que los demás
 * servicios (no lanza UnsupportedOperationException): LSP.
 */
public class ServicioRecojoTienda extends ServicioEnvioBase {

    @Override
    public TipoServicio getTipo() { return TipoServicio.RECOJO_TIENDA; }

    @Override
    protected double getCargoFijo() { return 6.00; }

    @Override
    protected double getCostoPorKg() { return 1.20; }

    @Override
    protected int getDiasEntrega() { return 2; }

    @Override
    public int getPrioridad() { return 4; }

    @Override
    public boolean admite(EnvioTo envio) {
        return envio != null && envio.getPesoKg() > 0 && envio.getIdtiendaRecojo() != null;
    }

    @Override
    public List<EstadoEnvio> getFlujoEstados() {
        return Arrays.asList(EstadoEnvio.REGISTRADO, EstadoEnvio.RECOGIDO,
                EstadoEnvio.EN_TRANSITO, EstadoEnvio.CLASIFICADO, EstadoEnvio.ENTREGADO);
    }
}
```

Evidencia. En la salida de la sección 3.2 los cinco subtipos se usan a través de la misma referencia `IServicioEnvio`, todos responden `admite`, `calcularTarifa`, `getPrioridad` y `getFlujoEstados` sin excepciones, y cada uno devuelve un flujo distinto (4 a 7 etapas). Asimismo, en la sección 3.5 el controlador funciona con `EnvioDaoMemoria` en lugar de `EnvioDao` sin modificar su código.

3.4 Principio de Segregación de Interfaces (ISP)

> **Enunciado:** “Ningún cliente debe verse obligado a depender de métodos que no utiliza.”

Problema identificado

AppEmpresa define una sola interfaz `ICrudDao<T>` con cinco operaciones. Las clases que solo necesitan leer (como el catálogo de tarifas) o que no deben permitir modificaciones quedan forzadas a implementar métodos vacíos o que lanzan excepciones, como ocurre en

`ClienteDao`. En el dominio logístico el problema es más grave: el **historial de seguimiento** debe ser inmutable para garantizar la trazabilidad exigida por el negocio. Si `SeguimientoDao` implementara `ICrudDao`, tendría métodos `update()` y `delete()` que permitirían alterar el historial, abriendo la puerta a la manipulación de evidencias ante un reclamo.

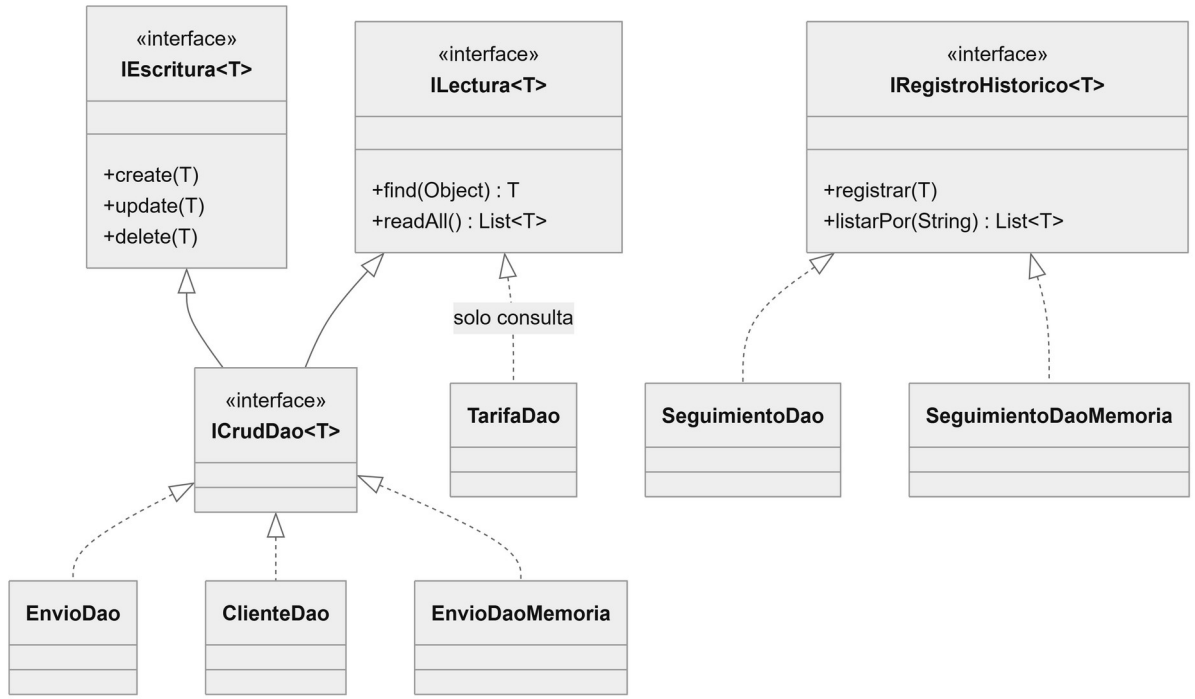
Solución aplicada

Se segregó el contrato de AppEmpresa en interfaces pequeñas y específicas:

- `ILectura<T>`: `find()` y `readAll()`.
- `IEscritura<T>`: `create()`, `update()` y `delete()`.
- `ICrudDao<T>`: se conserva por compatibilidad con el molde, pero ahora es la unión `ILectura<T> + IEscritura<T>`; la implementan solo los DAO que realmente usan las cinco operaciones (clientes, empleados, envíos).
- `IRegistroHistorico<T>`: solo `registrar()` y `listarPor()`; la implementan `SeguimientoDao` y la bitácora de auditoría.

La inmutabilidad se refuerza en la base de datos con los triggers `tg_seguimientos_no_update` y `tg_seguimientos_no_delete`, que rechazan cualquier modificación del historial.

Figura 7
Segregación del contrato ICrudDao en interfaces específicas



Nota. Elaboración propia del grupo de trabajo.

Código ejemplo

Código 10
uni.service.ILectura

```
package uni.service;
```

```
/** ISP: operaciones de solo lectura. */
public interface ILectura<T> {

    T find(Object id) throws Exception;

    List<T> readAll() throws Exception;
}
```

Código 11
uni.service.IEscritura

```
package uni.service;

/** ISP: operaciones de modificación. */
public interface IEscritura<T> {

    void create(T o) throws Exception;

    void update(T o) throws Exception;

    void delete(T o) throws Exception;
}
```

Código 12
uni.service.ICrudDao – compatibilidad con AppEmpresa

```
package uni.service;

/**
 * Se conserva el contrato de AppEmpresa, pero ahora es la unión de dos
 * interfaces pequeñas (ISP). Solo lo implementan los DAO que realmente
 * necesitan las cinco operaciones (clientes, empleados, envíos...).
 */
public interface ICrudDao<T> extends ILectura<T>, IEscritura<T> {
}
```

Código 13
uni.service.IRegistroHistorico – historial inmutable

```
package uni.service;

/**
 * ISP: contrato para historiales inmutables (seguimiento de estados,
 * bitácora de auditoría). Solo permite agregar y consultar:
 * no existe update() ni delete(), porque la trazabilidad no se altera.
 */
public interface IRegistroHistorico<T> {

    void registrar(T o) throws Exception;

    List<T> listarPor(String codigo) throws Exception;
}
```

Código 14
uni.dao.SeguimientoDao – implementa solo lo que necesita

```
package uni.dao;

/**
 * ISP: el historial de seguimiento solo se inserta y se consulta.
 * A diferencia de ClienteDao de AppEmpresa (que implementaba ICrudDao y
 * lanzaba UnsupportedOperationException), aquí no existen métodos vacíos.
 */
public class SeguimientoDao implements IRegistroHistorico<SeguimientoTo> {

    @Override
    public void registrar(SeguimientoTo o) throws Exception {
        Connection cn = AccesoDB.getInstancia().getConexion();
```

```
        try (CallableStatement cs = cn.prepareCall("{call
sp_Seguimiento_Registrar(?,?,?,?,?)}")) {
            cs.setString(1, o.getCodigoEnvio());
            cs.setString(2, o.getEstado().name());
            cs.setString(3, o.getUbicacion());
            cs.setString(4, o.getObservacion());
            cs.setString(5, o.getIdempleado());
            cs.executeUpdate();
        }
    }

    @Override
    public List<SeguimientoTo> listarPor(String codigo) throws Exception {
        List<SeguimientoTo> lista = new ArrayList<>();
        Connection cn = AccesoDB.getInstancia().getConexion();
        String sql = "SELECT codigo, estado, ubicacion, observacion, idempleado, fechahora "
                + "FROM seguimientos WHERE codigo = ? ORDER BY fechahora";
        try (PreparedStatement ps = cn.prepareStatement(sql)) {
            ps.setString(1, codigo);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    lista.add(new SeguimientoTo(rs.getString(1),
EstadoEnvio.valueOf(rs.getString(2)),
                            rs.getString(3), rs.getString(4), rs.getString(5),
                            rs.getTimestamp(6).toLocalDateTime()));
                }
            }
        }
        return lista;
    }
}
```

3.5 Principio de Inversión de Dependencias (DIP)

> **Enunciado:** “Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones.”

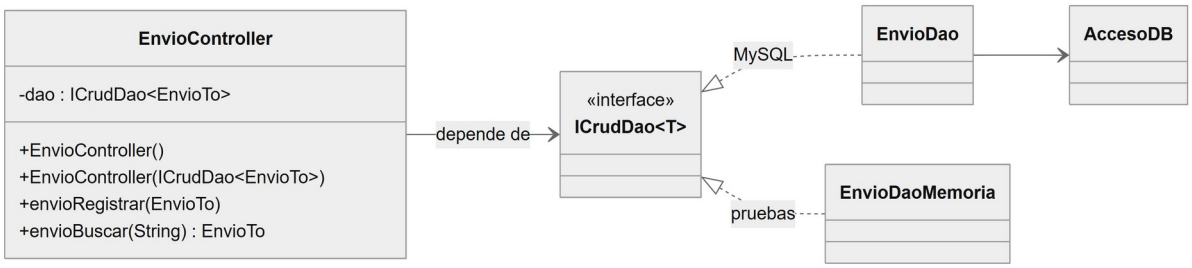
Problema identificado

En AppEmpresa, cada controlador crea directamente su DAO concreto en el constructor (`dao = new ClienteDao();`) y cada DAO llama de forma estática a `AccesoDB.getConnection()`. Las reglas de negocio (alto nivel) quedan atadas a MySQL/Oracle y a JDBC (bajo nivel). Para LogiRuta esto tiene dos consecuencias: (1) no es posible probar la lógica de registro o de seguimiento sin un servidor de base de datos en funcionamiento, y (2) no es posible sustituir un proveedor externo (mapas, SMS) sin modificar las clases de negocio que lo invocan, problema que dejó a la empresa sin notificaciones durante la caída de su proveedor de SMS.

Solución aplicada

Los módulos de alto nivel dependen ahora de **interfaces definidas en `uni.service`**: `EnvioController` y `RegistroEnvioFacade` reciben un `ICrudDao<EnvioTo>` y un `IRegistroHistorico<SeguimientoTo>` por constructor (inyección de dependencias), y la fachada obtiene sus integraciones mediante `IFabricaIntegraciones`, `IGeolocalizador` e `INotificador`. Para no romper el estilo de AppEmpresa, el controlador conserva un constructor sin parámetros que usa la implementación MySQL por defecto, y añade uno que permite inyectar otra implementación.

Figura 8
Inversión de dependencias en EnvioController



Nota. Elaboración propia del grupo de trabajo.

Código ejemplo

Código 15
uni.controller.EnvioController – depende de la abstracción

```
package uni.controller;

/**
 * DIP: el controlador depende de la abstracción ICrudDao<EnvioTo>,
 * no de la clase concreta EnvioDao (en AppEmpresa: dao = new ClienteDao()).
 */
public class EnvioController {

    private final ICrudDao<EnvioTo> dao;

    /** Constructor usado por los formularios Swing: MySQL por defecto. */
    public EnvioController() {
        this(new EnvioDao());
    }

    /** Inyección por constructor: permite usar otra implementación (pruebas). */
    public EnvioController(ICrudDao<EnvioTo> dao) {
        this.dao = dao;
    }

    public void envioRegistrar(EnvioTo e) throws Exception {
        dao.create(e);
    }

    public EnvioTo envioBuscar(String codigo) throws Exception {
        return dao.find(codigo);
    }

    public List<EnvioTo> envioListar() throws Exception {
        return dao.readAll();
    }
}
```

Código 16
uni.dao.EnvioDaoMemoria – implementación alternativa para pruebas

```
package uni.dao;

/**
 * Implementación en memoria del mismo contrato ICrudDao<EnvioTo>.
 * Sirve para pruebas unitarias y para demostrar DIP/LSP: los controladores
 * funcionan igual con MySQL o con esta clase.
 */
public class EnvioDaoMemoria implements ICrudDao<EnvioTo> {

    private final Map<String, EnvioTo> tabla = new LinkedHashMap<>();
    private int correlativo = 1;

    @Override
    public void create(EnvioTo o) {
        o.setCodigo(String.format("LR%08d", correlativo++));
```

```
        tabla.put(o.getCodigo(), o);
    }

    @Override
    public void update(EnvioTo o) {
        tabla.put(o.getCodigo(), o);
    }

    @Override
    public void delete(EnvioTo o) {
        tabla.remove(o.getCodigo());
    }

    @Override
    public EnvioTo find(Object id) {
        return tabla.get(id.toString());
    }

    @Override
    public List<EnvioTo> readAll() {
        return new ArrayList<>(tabla.values());
    }
}
```

Evidencia. El mismo controlador que en producción trabaja con MySQL se ejecuta con el repositorio en memoria, sin modificar una sola línea de su código:

Salida en consola (ejecución de uni.util.PruebaPatrones):

```
Controlador usando EnvioDaoMemoria sin modificar su código.
Registrado: Envio[LR00000001 | ESTANDAR | - -> Jesús María | 2.00 kg | REGISTRADO]
Total en repositorio: 1
```

Beneficios obtenidos

Tabla 18
Beneficios obtenidos con la aplicación de SOLID

Principio	Antes	Después	Beneficio para LogiRuta
SRP	Formulario con validación, tarifa, SQL y SMS	Vista, validador, servicio, DAO, notificador y fachada separados	Cambios aislados; reglas probables sin interfaz gráfica; se elimina la inyección SQL
OCP	`switch` por servicio repetido en varios métodos	Jerarquía `IServicioEnvio`	Nuevo servicio (entrega el mismo día) = una clase nueva; menor costo de cambio
LSP	Subtipos que lanzan `UnsupportedOperationException`	Contrato con `admite()` y flujo como datos	Procesos masivos y reportes recorren cualquier servicio sin fallas
ISP	`ICrudDao` único con métodos no usados	`ILectura`, `IEscritura`, `IRegistroHistorico`	Historial de seguimiento inmutable; trazabilidad y confianza del mercado
DIP	`new ClienteDao()` y llamadas estáticas	Inyección de interfaces	Pruebas sin base de datos; cambio de proveedor externo sin recompilar el negocio

CAPÍTULO IV. PATRONES CREACIONALES

Los patrones creacionales abstraen el proceso de instanciación: hacen que el sistema sea independiente de cómo se crean, componen y representan sus objetos, y delegan la responsabilidad de creación en clases especializadas (Gamma et al., 1994). En AppLogistica la creación de objetos es crítica porque (1) existen recursos que deben ser únicos —la conexión a la base de datos y la sesión del usuario—, (2) los servicios y las integraciones varían según el contexto y (3) los envíos son objetos complejos que se registran en grandes volúmenes. Para cada patrón se responde: **¿qué problema tengo?, ¿por qué este patrón lo resuelve? y ¿cómo lo implementé?**

4.1 Patrón Singleton

Problema

En AppEmpresa, `AccesoDB.getConnection()` carga el driver y abre una **conexión nueva en cada invocación**. En la operación de LogiRuta, un registro de envío ejecuta varias operaciones (correlativo, direcciones, envío, historial), y en campaña se registran más de 16 000 envíos al día. Abrir una conexión por operación provocó la saturación del servidor y tiempos de respuesta superiores a 8 segundos. Además, la configuración de conexión (URL, usuario, clave) estaba repetida y podía divergir entre clases. Un segundo problema es la **sesión del usuario**: el Proxy de seguridad, la auditoría y los formularios necesitan conocer al usuario autenticado y su rol, pero este dato se pasaba de formulario en formulario como parámetro.

Justificación

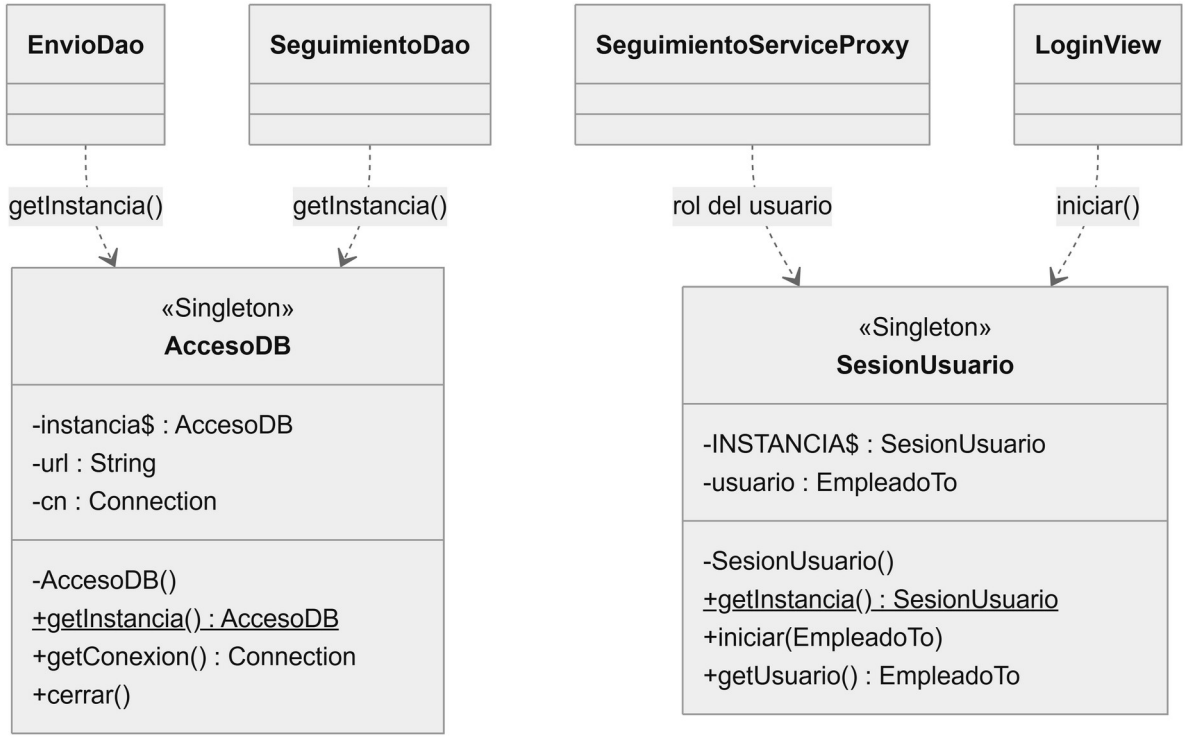
El patrón Singleton garantiza que una clase tenga **una única instancia** y ofrece un **punto de acceso global** a ella. Es adecuado porque la configuración de acceso a datos y la sesión del usuario son, por naturaleza del negocio, únicas dentro de la aplicación de escritorio. Se aplican las dos variantes vistas en clase:

- `AccesoDB`: **instanciación perezosa con doble verificación** (*double-checked locking*) y variable `volatile`, segura en entornos multihilo (la interfaz Swing y las tareas en segundo plano de notificación).
- `SesionUsuario`: **instanciación temprana**, pues la sesión se usa desde el inicio de la aplicación.

Se consideraron las desventajas señaladas en clase (estado global y dificultad para pruebas): por ello la lógica de negocio no llama a `AccesoDB` directamente, sino que depende de interfaces DAO (DIP), y las pruebas usan DAO en memoria.

Diagrama UML

Figura 9
Diagrama de clases del patrón Singleton en AppLogistica



Nota. Elaboración propia del grupo de trabajo.

Implementación

Código 17
uni.database.AccesoDB – Singleton perezoso y thread-safe

```
package uni.database;

/**
 * PATRÓN SINGLETON.
 * Punto único de acceso a la base de datos "logistica".
 * Evolución de uni.database.AccesoDB de AppEmpresa: en lugar de abrir una
 * conexión nueva en cada llamada, existe una sola instancia que administra
 * la configuración y reutiliza la conexión mientras esté abierta.
 */
public final class AccesoDB {

    // Instancia única. volatile garantiza visibilidad entre hilos.
    private static volatile AccesoDB instancia;

    private final String driver = "com.mysql.cj.jdbc.Driver";
    private final String url = "jdbc:mysql://localhost:3306/logistica?
useSSL=false&serverTimezone=America/Lima";
    private final String usuario = "logistica";
    private final String clave = "logistica";
    private Connection cn;

    // Constructor privado: nadie fuera de la clase puede hacer "new AccesoDB()".
    private AccesoDB() {
        System.out.println("[AccesoDB] Configuración de conexión cargada (una sola vez).");
    }

    // Instanciación perezosa con doble verificación (thread-safe).
    public static AccesoDB getInstancia() {
        if (instancia == null) {
            synchronized (AccesoDB.class) {
                if (instancia == null) {
```

```
                    instancia = new AccesoDB();
                }
            }
        }
        return instancia;
    }

    /** Devuelve la conexión activa; si no existe o fue cerrada, la reabre. */
    public synchronized Connection getConexion() throws SQLException, ClassNotFoundException
{
        if (cn == null || cn.isClosed()) {
            Class.forName(driver);
            cn = DriverManager.getConnection(url, usuario, clave);
        }
        return cn;
    }

    /** Se invoca una sola vez al cerrar la aplicación (PrincipalView). */
    public synchronized void cerrar() throws SQLException {
        if (cn != null && !cn.isClosed()) {
            cn.close();
        }
    }

    public String getUrl() {
        return url;
    }
}
```

Código 18
uni.util.SesionUsuario – Singleton con instanciación temprana

```
package uni.util;

/**
 * PATRÓN SINGLETON (instanciación temprana).
 * Guarda el usuario autenticado en LoginView para que cualquier capa
 * (Proxy de seguridad, auditoría, formularios) conozca quién opera.
 */
public final class SesionUsuario {

    private static final SesionUsuario INSTANCIA = new SesionUsuario();

    private EmpleadoTo usuario;

    private SesionUsuario() {
    }

    public static SesionUsuario getInstancia() {
        return INSTANCIA;
    }

    public void iniciar(EmpleadoTo usuario) {
        this.usuario = usuario;
    }

    public void cerrar() {
        this.usuario = null;
    }

    public EmpleadoTo getUsuario() {
        if (usuario == null) {
            throw new IllegalStateException("No hay sesión iniciada.");
        }
        return usuario;
    }
}
```

Uso desde la capa de datos (fragmento de `EnvioDao`): `Connection cn = AccesoDB.getInstancia().getConexion();`

Resultado

Las dos referencias obtenidas apuntan al mismo objeto (mismo `identityHashCode`) y el mensaje de carga de configuración aparece **una sola vez**, aunque se solicite la instancia varias veces:

Salida en consola (ejecución de uni.util.PruebaPatrones):

```
[AccesoDB] Configuración de conexión cargada (una sola vez).
a.hashCode() = 1625635731
b.hashCode() = 1625635731
¿Misma instancia? true
URL configurada: jdbc:mysql://localhost:3306/logistica?useSSL=false&serverTimezone=America/
Lima
Usuario en sesión (Singleton SesionUsuario): Rosa - AGENTE_VENTANILLA
```

4.2 Patrón Factory

Problema

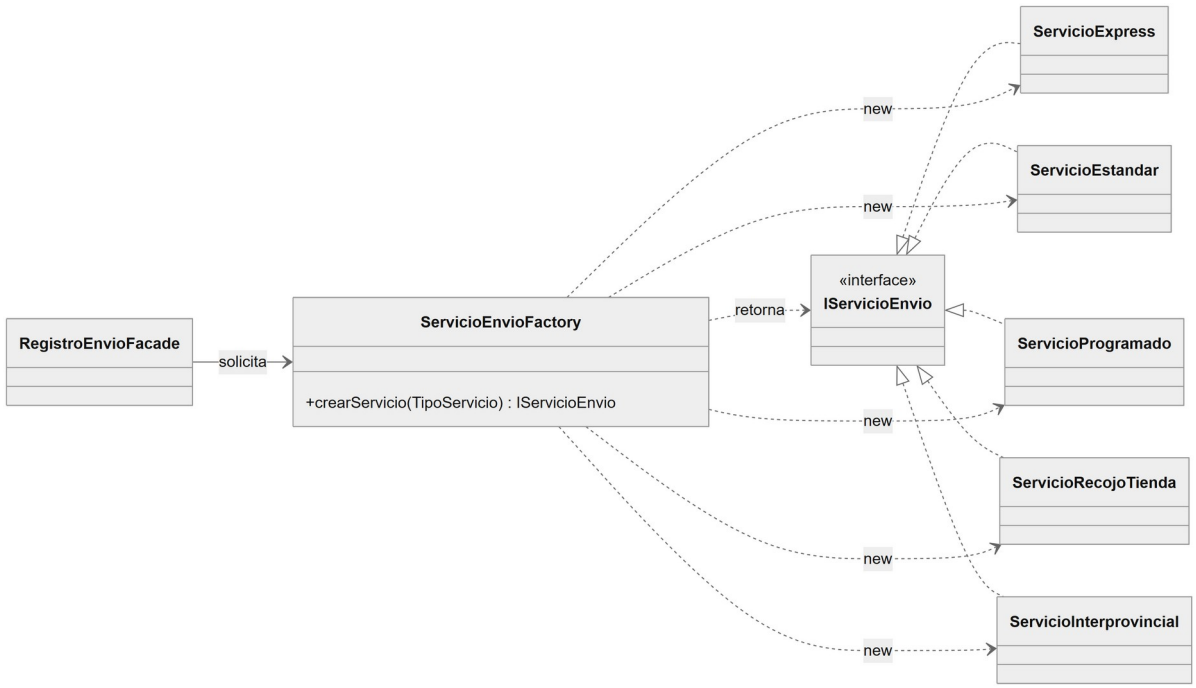
El registro de un envío necesita el objeto de servicio adecuado (`ServicioExpress`, `ServicioEstandar`, etc.) según lo que el agente selecciona en pantalla. Si cada clase cliente (fachada de registro, importación masiva, cotizador web, reprogramación) instancia directamente las clases concretas con `new`, la decisión de **qué clase crear** se repite en muchos lugares y todos ellos deben modificarse al agregar un servicio, contradiciendo el OCP del Capítulo III.

Justificación

El patrón Factory (Factory Method en su variante parametrizada, como se trabajó en la sesión 07) centraliza en un único método la creación de objetos de una misma familia de productos, devolviendo la abstracción `IServicioEnvio`. Los clientes **no usan `new`** sobre clases concretas ni conocen su existencia: solo piden un servicio por su tipo. Agregar la entrega en el mismo día implica modificar únicamente la fábrica; el resto del sistema permanece intacto.

Diagrama UML

Figura 10
Diagrama de clases del patrón Factory para servicios de envío



Nota. Elaboración propia del grupo de trabajo.

Implementación

Código 19
uni.util.ServicioEnvioFactory – creación centralizada de servicios

```
package uni.util;

/**
 * PATRÓN FACTORY (Factory Method parametrizado).
 * Único lugar del sistema que conoce las clases concretas de servicio.
 * Controladores y fachadas piden un IServicioEnvio sin usar "new".
 */
public class ServicioEnvioFactory {

    public IServicioEnvio crearServicio(TipoServicio tipo) {
        if (tipo == null) {
            throw new IllegalArgumentException("Tipo de servicio no indicado.");
        }
        switch (tipo) {
            case EXPRESS:
                return new ServicioExpress();
            case ESTANDAR:
                return new ServicioEstandar();
            case PROGRAMADO:
                return new ServicioProgramado();
            case RECOJO_TIENDA:
                return new ServicioRecojoTienda();
            case INTERPROVINCIAL:
                return new ServicioInterprovincial();
            default:
                throw new IllegalArgumentException("Servicio no soportado: " + tipo);
        }
    }
}
```

Uso en la fachada: `IServicioEnvio servicio = fabricaServicios.crearServicio(envio.getServicio());`

Resultado

Para cada tipo de servicio la fábrica devuelve la clase concreta correspondiente, cada una con su propio flujo de estados:

Salida en consola (ejecución de uni.util.PruebaPatrones):

```
  crearServicio(EXPRESS        ) -> ServicioExpress          flujo=[REGISTRADO, RECOGIDO,
EN_RUTA, ENTREGADO]
  crearServicio(ESTANDAR       ) -> ServicioEstandar         flujo=[REGISTRADO, RECOGIDO,
EN_TRANSITO, CLASIFICADO, EN_RUTA, ENTREGADO]
  crearServicio(PROGRAMADO     ) -> ServicioProgramado       flujo=[REGISTRADO, RECOGIDO,
CLASIFICADO, EN_RUTA, ENTREGADO]
  crearServicio(RECOJO_TIENDA  ) -> ServicioRecojoTienda     flujo=[REGISTRADO, RECOGIDO,
EN_TRANSITO, CLASIFICADO, ENTREGADO]
  crearServicio(INTERPROVINCIAL) -> ServicioInterprovincial  flujo=[REGISTRADO, RECOGIDO,
CLASIFICADO, EN_TRANSITO, CLASIFICADO, EN_RUTA, ENTREGADO]
```

4.3 Patrón Abstract Factory

Problema

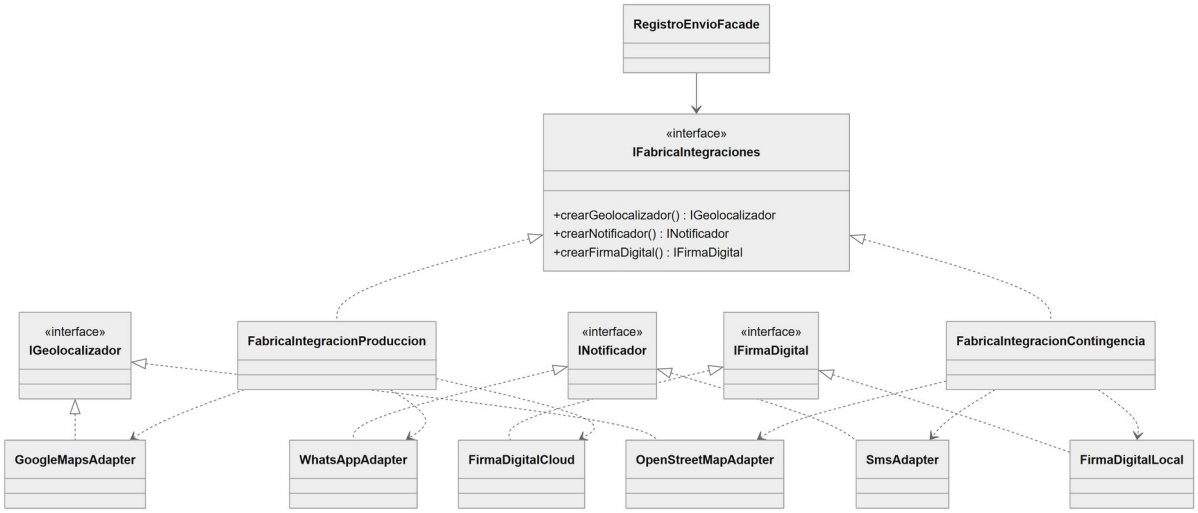
LogiRuta depende de tres integraciones externas que trabajan juntas: **geolocalización**, **notificación** y **firma digital**. El caso empresarial exige una arquitectura “preparada para integrarse con múltiples servicios sin generar dependencia excesiva de uno solo”, fundamental para la continuidad operativa. La empresa opera con dos combinaciones de proveedores: la familia principal en la nube (mapas de Google, WhatsApp Business y firma electrónica en la nube) y una familia de **contingencia** (OpenStreetMap, gateway SMS del operador y firma local en el dispositivo, sincronizada luego) para caídas del proveedor principal o zonas sin cobertura de datos. Si cada clase crea sus integraciones de forma independiente, es posible **mezclar proveedores incompatibles** (por ejemplo, firma local con notificación por WhatsApp sin conexión), y cambiar de familia exige modificar muchas clases.

Justificación

El patrón Abstract Factory proporciona una interfaz para crear **familias de objetos relacionados** sin especificar sus clases concretas. Cada fábrica concreta garantiza que los productos que entrega sean coherentes entre sí. Para conmutar a contingencia basta con inyectar `FabricaIntegracionContingencia` en lugar de `FabricaIntegracionProduccion` (decisión que puede leerse desde la tabla `proveedores_integracion`), sin modificar la fachada ni ninguna otra clase de negocio. Los productos concretos son, a su vez, adaptadores de las librerías externas (patrón Adapter, sección 5.1).

Diagrama UML

Figura 11
Diagrama de clases del patrón Abstract Factory de integraciones



Nota. Elaboración propia del grupo de trabajo.

Implementación

Código 20
uni.service.IFabricaIntegraciones – fábrica abstracta

```
package uni.service;

/**
 * PATRÓN ABSTRACT FACTORY.
 * Crea una familia coherente de integraciones externas
 * (geolocalización + notificación + firma digital).
 */
public interface IFabricaIntegraciones {

    IGeolocalizador crearGeolocalizador();

    INotificador crearNotificador();

    IFirmaDigital crearFirmaDigital();

    String getNombreFamilia();
}
```

Código 21
uni.util.FabricaIntegracionProduccion – familia principal

```
package uni.util;

/** PATRÓN ABSTRACT FACTORY - Fábrica concreta: proveedores principales en la nube. */
public class FabricaIntegracionProduccion implements IFabricaIntegraciones {

    @Override
    public IGeolocalizador crearGeolocalizador() {
        return new GoogleMapsAdapter(new ExtGoogleMapsClient());
    }

    @Override
    public INotificador crearNotificador() {
        return new WhatsAppAdapter(new ExtWhatsAppBusinessApi());
    }

    @Override
    public IFirmaDigital crearFirmaDigital() {
        return new FirmaDigitalCloud();
```

```
    }

    @Override
    public String getNombreFamilia() {
        return "PRODUCCIÓN (nube)";
    }
}
```

Código 22
uni.util.FabricaIntegracionContingencia – familia de contingencia

```
package uni.util;

/**
 * PATRÓN ABSTRACT FACTORY - Fábrica concreta: proveedores alternos para
 * continuidad operativa (caída de proveedor principal o zonas sin datos).
 */
public class FabricaIntegracionContingencia implements IFabricaIntegraciones {

    @Override
    public IGeolocalizador crearGeolocalizador() {
        return new OpenStreetMapAdapter(new ExtNominatimClient());
    }

    @Override
    public INotificador crearNotificador() {
        return new SmsAdapter(new ExtSmsGatewayLegacy());
    }

    @Override
    public IFirmaDigital crearFirmaDigital() {
        return new FirmaDigitalLocal();
    }

    @Override
    public String getNombreFamilia() {
        return "CONTINGENCIA (proveedores alternos)";
    }
}
```

Resultado

El mismo código cliente recorre ambas familias; cada una produce un geolocalizador, un notificador y una firma digital consistentes entre sí:

Salida en consola (ejecución de uni.util.PruebaPatrones):

```
Familia: PRODUCCIÓN (nube)
  Geolocalizador -> Google Maps (-12.01710, -77.06710)
  Notificador    -> WHATSAPP
   [WA-API] +51987654321 template=estado_envio body="LogiRuta - Prueba de canal"
  Firma digital  -> Firma electrónica en la nube | código CLOUD-63845003
Familia: CONTINGENCIA (proveedores alternos)
  Geolocalizador -> OpenStreetMap (-12.06000, -77.04000)
  Notificador    -> SMS
   [SMS-GW] 987654321 <- "LogiRuta: Prueba de canal"
  Firma digital  -> Firma local en dispositivo (sincroniza luego) | código LOCAL-15619
```

4.4 Patrón Builder

Problema

`EnvioTo` es un objeto complejo con **17 atributos**, muchos de ellos opcionales y dependientes del servicio: la fecha programada solo aplica a la mensajería programada, la tienda de recojo solo al recojo en tienda, el valor declarado solo si se contrata seguro. Siguiendo el estilo de

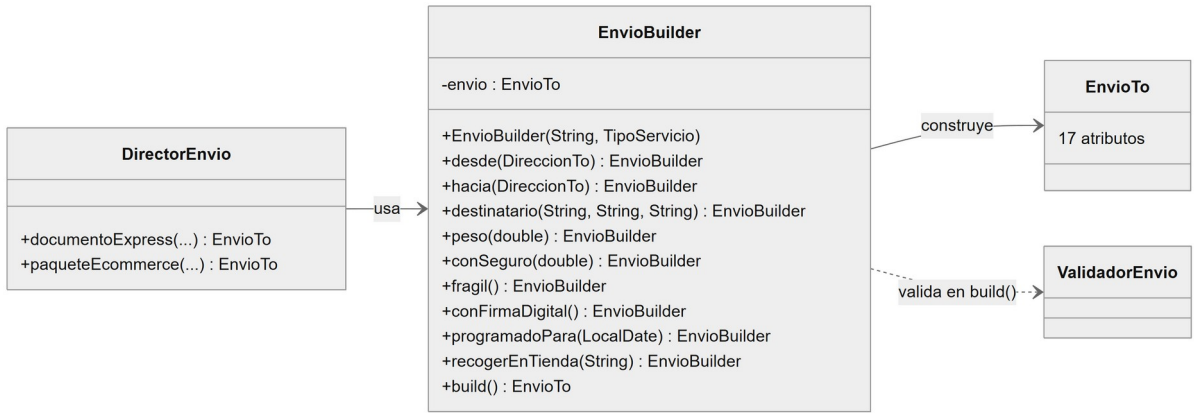
AppEmpresa (constructor con todos los parámetros, como en `VentaTo`), se tendría un **constructor telescópico** de 17 parámetros, ilegible y propenso a errores de orden (por ejemplo, intercambiar peso y valor declarado, ambos `double`); con *setters* sueltos, en cambio, se pueden crear envíos incompletos o inconsistentes (un envío programado sin fecha).

Justificación

El patrón Builder separa la construcción de un objeto complejo de su representación y permite construirlo **paso a paso** con métodos expresivos (`peso()`, `conSeguro()`, `programadoPara()`), produciendo distintas configuraciones con el mismo proceso. El método `build()` valida el objeto completo antes de entregarlo, por lo que **no pueden existir envíos inválidos** en el sistema. El `DirectorEnvio` encapsula las recetas frecuentes de ventanilla (sobre de documentos express con firma, paquete de comercio electrónico asegurado).

Diagrama UML

Figura 12
Diagrama de clases del patrón Builder para EnvioTo



Nota. Elaboración propia del grupo de trabajo.

Implementación

Código 23
uni.util.EnvioBuilder – construcción paso a paso con validación

```
package uni.util;

/**
 * PATRÓN BUILDER.
 * Construye un EnvioTo paso a paso. Evita el "constructor telescópico" de
 * 17 parámetros y valida el objeto completo antes de entregarlo.
 */
public class EnvioBuilder {

    private final EnvioTo envio = new EnvioTo();

    public EnvioBuilder(String idcliente, TipoServicio servicio) {
        envio.setIdcliente(idcliente);
        envio.setServicio(servicio);
    }

    public EnvioBuilder desde(DireccionTo origen) {
        envio.setOrigen(origen);
        return this;
```

```
    }

    public EnvioBuilder hacia(DireccionTo destino) {
        envio.setDestino(destino);
        return this;
    }

    public EnvioBuilder destinatario(String nombre, String celular, String email) {
        envio.setNombreDestinatario(nombre);
        envio.setTelefonoDestinatario(celular);
        envio.setEmailDestinatario(email);
        return this;
    }

    public EnvioBuilder peso(double kg) {
        envio.setPesoKg(kg);
        return this;
    }

    public EnvioBuilder conSeguro(double valorDeclarado) {
        envio.setConSeguro(true);
        envio.setValorDeclarado(valorDeclarado);
        return this;
    }

    public EnvioBuilder fragil() {
        envio.setFragil(true);
        return this;
    }

    public EnvioBuilder conFirmaDigital() {
        envio.setRequiereFirma(true);
        return this;
    }

    public EnvioBuilder programadoPara(LocalDate fecha) {
        envio.setFechaProgramada(fecha);
        return this;
    }

    public EnvioBuilder recogerEnTienda(String idtienda) {
        envio.setIdtiendaRecojo(idtienda);
        return this;
    }

    /** Valida y entrega el producto terminado. */
    public EnvioTo build() {
        List<String> errores = new ValidadorEnvio().validar(envio);
        if (envio.getServicio() == TipoServicio.PROGRAMADO && envio.getFechaProgramada() ==
null) {
            errores.add("La mensajería programada requiere fecha.");
        }
        if (envio.getServicio() == TipoServicio.RECOJO_TIENDA && envio.getIdtiendaRecojo()
== null) {
            errores.add("El recojo en tienda requiere indicar la agencia.");
        }
        if (envio.getServicio() != TipoServicio.RECOJO_TIENDA && envio.getDestino() == null)
{
            errores.add("La dirección de destino es obligatoria.");
        }
        if (!errores.isEmpty()) {
            throw new IllegalStateException("Envío inválido: " + errores);
        }
        return envio;
    }
}
```

Código 24
uni.util.DirectorEnvio – recetas de construcción

```
package uni.util;

/**
```

```
 * PATRÓN BUILDER - Director.
 * Encapsula "recetas" de construcción frecuentes en ventanilla.
 */
public class DirectorEnvio {

    /** Sobre de documentos express con firma de cargo. */
    public EnvioTo documentoExpress(String idcliente, DireccionTo origen, DireccionTo
destino,
                                    String receptor, String celular) {
        return new EnvioBuilder(idcliente, TipoServicio.EXPRESS)
                .desde(origen).hacia(destino)
                .destinatario(receptor, celular, null)
                .peso(0.5)
                .conFirmaDigital()
                .build();
    }

    /** Paquete de comercio electrónico estándar asegurado. */
    public EnvioTo paqueteEcommerce(String idcliente, DireccionTo origen, DireccionTo
destino,
                                    String receptor, String celular, double kg, double
valor) {
        return new EnvioBuilder(idcliente, TipoServicio.ESTANDAR)
                .desde(origen).hacia(destino)
                .destinatario(receptor, celular, null)
                .peso(kg)
                .conSeguro(valor)
                .build();
    }
}
```

Resultado

Se construye un envío programado con seguro y contenido frágil, un sobre express mediante el Director, y se comprueba que el Builder rechaza un envío programado sin fecha. El código aparece como `null` porque se asigna recién al registrarse en la base de datos.

Salida en consola (ejecución de uni.util.PruebaPatrones):

```
Construido: Envio[null | PROGRAMADO | Callao -> San Isidro | 1.20 kg | REGISTRADO]
  seguro=true fragil=true fechaProgramada=2026-11-05
Director (documentoExpress): Envio[null | EXPRESS | Callao -> Surco | 0.50 kg | REGISTRADO]
firma=true
Validación del Builder: Envío inválido: [La mensajería programada requiere fecha.]
```

4.5 Patrón Prototype

Problema

Un cliente corporativo como *Tienda Virtual Andina S.A.C.* despacha cada día entre 300 y 800 pedidos casi idénticos: mismo almacén de origen, mismo servicio estándar, peso típico de 1.5 kg y seguro por S/ 120; solo cambian el destinatario y la dirección. Construir cada envío desde cero repite el mismo trabajo cientos de veces y obliga al cliente a ingresar los mismos datos en cada pedido, que es justamente el proceso manual que se quiere eliminar (RF05).

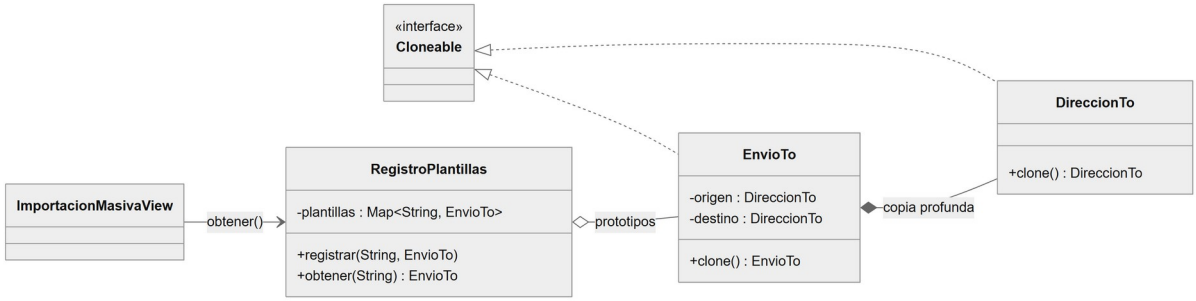
Justificación

El patrón Prototype crea objetos **copiando una instancia existente** (prototipo) en lugar de construirlos desde cero, y es útil cuando la creación es costosa o repetitiva. Cada cliente corporativo registra sus plantillas en un `RegistroPlantillas`; el registro masivo clona la

plantilla y solo completa los datos variables. Se implementó una **copia profunda**: `EnvioTo.clone()` clona también sus objetos `DireccionTo`, de modo que modificar la dirección de un clon **no altera el prototipo** ni a los otros clones (error típico de la copia superficial). Además, el clon reinicia el código y el estado, porque un envío clonado todavía no ha sido registrado.

Diagrama UML

Figura 13
Diagrama de clases del patrón Prototype para plantillas de envío



Nota. Elaboración propia del grupo de trabajo.

Implementación

Código 25
uni.entity.EnvioTo – método clone() con copia profunda (fragmento)

```
public class EnvioTo implements Cloneable {
    // ... 17 atributos, getters y setters ...

    /** Copia profunda: las direcciones también se clonan. */
    @Override
    public EnvioTo clone() {
        try {
            EnvioTo copia = (EnvioTo) super.clone();
            copia.origen = (origen == null) ? null : origen.clone();
            copia.destino = (destino == null) ? null : destino.clone();
            copia.codigo = null;                 // la copia aún no está registrada
            copia.estado = EstadoEnvio.REGISTRADO;
            return copia;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);
        }
    }
}
```

Código 26
uni.util.RegistroPlantillas – registro de prototipos

```
package uni.util;

/**
 * PATRÓN PROTOTYPE - Registro de prototipos.
 * Un cliente corporativo guarda plantillas de envío (origen, servicio,
 * peso típico, seguro...) y genera cientos de envíos clonándolas.
 */
public class RegistroPlantillas {

    private final Map<String, EnvioTo> plantillas = new HashMap<>();

    public void registrar(String clave, EnvioTo prototipo) {
        plantillas.put(clave, prototipo);
    }
```

```
    /** Devuelve una copia profunda; el prototipo nunca se modifica. */
    public EnvioTo obtener(String clave) {
        EnvioTo prototipo = plantillas.get(clave);
        if (prototipo == null) {
            throw new IllegalArgumentException("No existe la plantilla " + clave);
        }
        return prototipo.clone();
    }
}
```

Resultado

Se generan tres envíos a partir de la plantilla “ECOMMERCE_STD”. Cada clon recibe un código propio al registrarse, y se verifica que el prototipo permanece intacto y que los clones no comparten la dirección con él:

Salida en consola (ejecución de uni.util.PruebaPatrones):

```
  Clon -> Envio[LR00000001 | ESTANDAR | Callao -> San Martín de Porres | 1.50 kg |
REGISTRADO] para Ana Torres
  Clon -> Envio[LR00000002 | ESTANDAR | Callao -> Breña | 1.50 kg | REGISTRADO] para Jorge
Lima
  Clon -> Envio[LR00000003 | ESTANDAR | Callao -> Miraflores | 1.50 kg | REGISTRADO] para
Elena Vega
Prototipo intacto: destino=- | ¿destino compartido con clon? false
```

Comparación de patrones creacionales

Tabla 19
Comparación de los patrones creacionales aplicados en AppLogistica

Patrón	Intención	Problema de LogiRuta que resuelve	Clases en AppLogistica	Cuándo usarlo
Singleton	Única instancia con acceso global	Saturación por conexiones repetidas; sesión del usuario dispersa	`AccesoDB`, `SesionUsuario`	Recursos que deben ser únicos en la aplicación
Factory	Delegar la creación de un producto según un parámetro	Selección del servicio de envío repetida en muchas clases	`ServicioEnvioFactory`	Una jerarquía de productos con creación condicional
Abstract Factory	Crear familias de productos relacionados	Combinaciones coherentes de proveedores externos y contingencia	`IFabricaIntegraciones` y fábricas concretas	Varias familias de objetos que deben usarse juntos
Builder	Construir objetos complejos paso a paso	Envío con 17 atributos opcionales y reglas por servicio	`EnvioBuilder`, `DirectorEnvio`	Muchos parámetros opcionales y validación del objeto completo
Prototype	Crear copiando un prototipo	Registro masivo de pedidos casi idénticos	`EnvioTo.clone()`, `RegistroPlantillas`	Objetos similares en gran cantidad o costosos de crear

Los patrones creacionales se complementan: el Builder valida y construye cada envío; el Prototype reutiliza envíos ya construidos; la Factory decide qué regla de servicio aplicar; la Abstract Factory decide con qué familia de proveedores trabajar; y el Singleton garantiza recursos únicos para todos ellos. En conjunto, eliminan el uso indiscriminado de `new` en las capas de presentación y negocio, lo que reduce el acoplamiento y facilita las pruebas.

CAPÍTULO V. PATRONES ESTRUCTURALES

Los patrones estructurales explican cómo ensamblar clases y objetos en estructuras más grandes manteniendo la flexibilidad y la eficiencia. Su mecanismo principal es la **composición de objetos** (agregación y delegación) en lugar de la herencia, con el objetivo de reducir el acoplamiento y reutilizar código existente. En AppLogistica resuelven tres grandes necesidades del caso empresarial: **integrarse con proveedores externos heterogéneos** (Adapter), **simplificar procesos que involucran muchos subsistemas** (Facade, Proxy) y **modelar variantes y jerarquías del negocio sin explosión de clases** (Decorator, Composite, Bridge).

5.1 Patrón Adapter

Problema

La empresa notifica a los destinatarios mediante tres proveedores externos, cada uno con una interfaz distinta e incompatible, y que **no pueden modificarse** porque son librerías de terceros:

- La librería de correo (`ExtSmtpMailer`) recibe un arreglo de destinatarios y cuerpo HTML, y lanza `IOException` ante errores.
- El gateway SMS heredado del operador (`ExtSmsGatewayLegacy`) solo acepta números de 9 dígitos sin prefijo, mensajes de hasta 160 caracteres, y responde con códigos numéricos (0 = OK, 21 = número inválido, 34 = mensaje largo).
- La API de WhatsApp Business (`ExtWhatsAppBusinessApi`) recibe un mapa clave-valor con el número en formato internacional E.164 (+51...) y una plantilla aprobada, y responde con un texto JSON.

Si la lógica de negocio invoca directamente estas librerías, queda acoplada a cada proveedor: en la caída del proveedor de SMS, reemplazarlo implicó modificar varios formularios. El mismo problema ocurre con los proveedores de mapas, que devuelven las coordenadas en formatos distintos (texto “lat,lng” frente a un arreglo `{longitud, latitud}` en orden inverso).

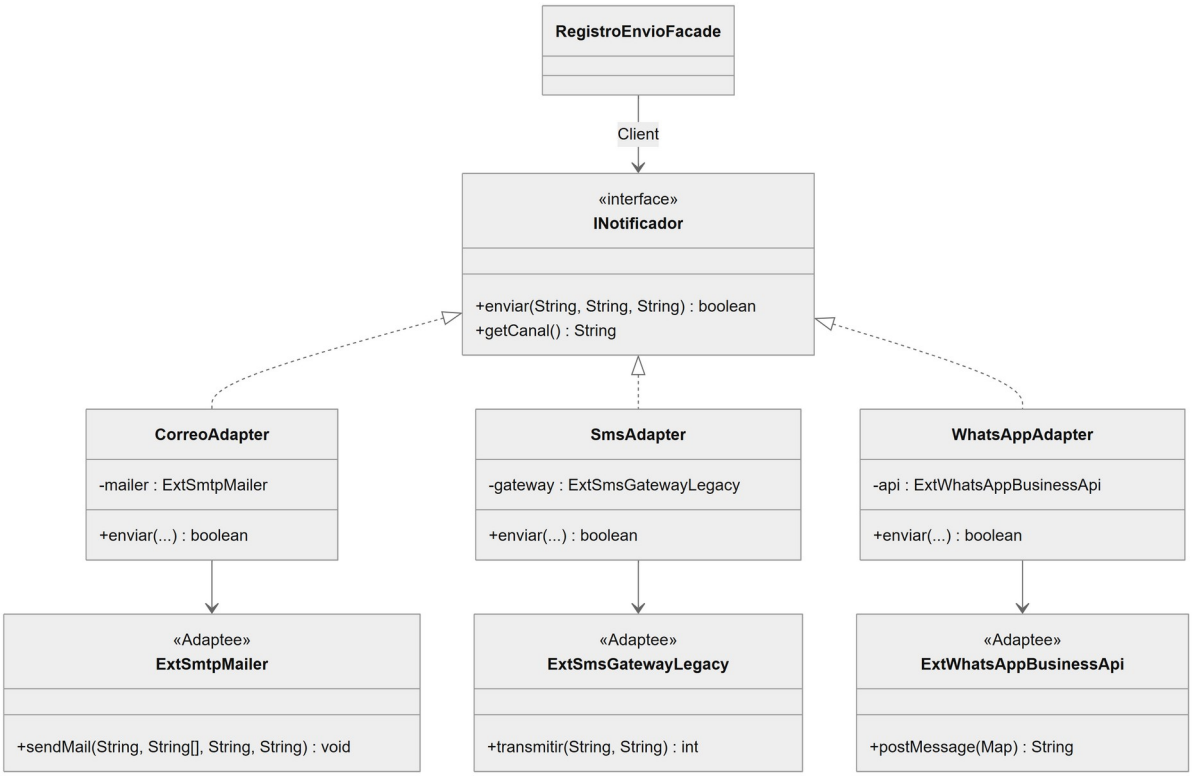
Justificación

El patrón Adapter actúa como un **traductor** que convierte la interfaz de una clase existente (*Adaptee*) en la interfaz que espera el cliente (*Target*). Se definió el *Target* propio `INotificador` con un único método `enviar(destinatario, asunto, mensaje) : boolean`, y un adaptador por proveedor que realiza las conversiones necesarias: texto a HTML y excepción a booleano (correo); eliminación del prefijo +51, truncado a 160 caracteres y código numérico a booleano (SMS); construcción del *payload* E.164 y respuesta JSON a booleano (WhatsApp). La fachada, el proxy y los procesos masivos solo conocen `INotificador`; cambiar o agregar

un proveedor consiste en escribir un adaptador nuevo, sin tocar el negocio (OCP y DIP). El mismo enfoque se aplicó a la geolocalización con `GoogleMapsAdapter` y `OpenStreetMapAdapter`, que implementan `IGeolocalizador`.

UML

Figura 14
Diagrama de clases del patrón Adapter para proveedores de notificación



Nota. Elaboración propia del grupo de trabajo.

Código

Código 27
uni.service.INotificador – Target

```
package uni.service;

/**
 * PATRÓN ADAPTER - Target.
 * Interfaz que el sistema espera para enviar notificaciones, sin importar
 * el proveedor externo (correo, SMS, WhatsApp...).
 */
public interface INotificador {

    /** @return true si el proveedor aceptó el mensaje. */
    boolean enviar(String destinatario, String asunto, String mensaje);

    String getCanal();
}
```

Código 28
uni.util.ExtSmsGatewayLegacy – Adaptee (librería externa simulada)

```
package uni.util;

/**
```

```
 * ADAPTEE (simula el gateway SMS heredado del operador).
 * Acepta solo números de 9 dígitos, textos de hasta 160 caracteres y
 * responde con códigos numéricos (0 = OK).
 */
public class ExtSmsGatewayLegacy {

    public int transmitir(String numero9, String texto) {
        if (numero9 == null || !numero9.matches("9\\d{8}")) {
            return 21;                      // número inválido
        }
        if (texto.length() > 160) {
            return 34;                      // mensaje demasiado largo
        }
        System.out.println("   [SMS-GW] " + numero9 + " <- \"" + texto + "\"");
        return 0;
    }
}
```

Código 29

uni.util.SmsAdapter – Adapter del gateway SMS

```
package uni.util;

/** PATRÓN ADAPTER: adapta ExtSmsGatewayLegacy a INotificador. */
public class SmsAdapter implements INotificador {

    private final ExtSmsGatewayLegacy gateway;

    public SmsAdapter(ExtSmsGatewayLegacy gateway) {
        this.gateway = gateway;
    }

    @Override
    public boolean enviar(String destinatario, String asunto, String mensaje) {
        String numero = destinatario.replace("+51", "").replace(" ", "");   // +51 999.. ->
999..
        String texto = asunto + ": " + mensaje;
        if (texto.length() > 160) {
            texto = texto.substring(0, 157) + "...";                        // límite del
gateway
        }
        int codigo = gateway.transmitir(numero, texto);
        return codigo == 0;                                                 // código ->
boolean
    }

    @Override
    public String getCanal() {
        return "SMS";
    }
}
```

Código 30

uni.util.WhatsAppAdapter – Adapter de WhatsApp Business

```
package uni.util;

/** PATRÓN ADAPTER: adapta ExtWhatsAppBusinessApi a INotificador. */
public class WhatsAppAdapter implements INotificador {

    private final ExtWhatsAppBusinessApi api;

    public WhatsAppAdapter(ExtWhatsAppBusinessApi api) {
        this.api = api;
    }

    @Override
    public boolean enviar(String destinatario, String asunto, String mensaje) {
        Map<String, String> payload = new HashMap<>();
        payload.put("to", destinatario.startsWith("+51") ? destinatario : "+51" +
destinatario);
        payload.put("template", "estado_envio");
```

```
        payload.put("body", asunto + " - " + mensaje);
        String respuesta = api.postMessage(payload);
        return respuesta.contains("\"sent\"");                              // JSON ->
boolean
    }

    @Override
    public String getCanal() {
        return "WHATSAPP";
    }
}
```

Código 31
uni.util.CorreoAdapter – Adapter de correo

```
package uni.util;

/** PATRÓN ADAPTER: adapta ExtSmtpMailer a la interfaz INotificador. */
public class CorreoAdapter implements INotificador {

    private static final String REMITENTE = "notificaciones@logiruta.pe";
    private final ExtSmtpMailer mailer;

    public CorreoAdapter(ExtSmtpMailer mailer) {
        this.mailer = mailer;
    }

    @Override
    public boolean enviar(String destinatario, String asunto, String mensaje) {
        try {
            String html = "<p>" + mensaje + "</p>";                      // texto -> HTML
            mailer.sendMail(REMITENTE, new String[]{destinatario}, asunto, html);
            return true;
        } catch (IOException ex) {                                       // excepción ->
boolean
            System.out.println("   [CorreoAdapter] rechazo: " + ex.getMessage());
            return false;
        }
    }

    @Override
    public String getCanal() {
        return "CORREO";
    }
}
```

Código 32
uni.util.OpenStreetMapAdapter – Adapter de geolocalización (corrige el orden de coordenadas)

```
package uni.util;

/** PATRÓN ADAPTER: adapta ExtNominatimClient a IGeolocalizador. */
public class OpenStreetMapAdapter implements IGeolocalizador {

    private final ExtNominatimClient cliente;

    public OpenStreetMapAdapter(ExtNominatimClient cliente) {
        this.cliente = cliente;
    }

    @Override
    public DireccionTo geocodificar(DireccionTo d) {
        double[] lonLat = cliente.search(d.getCalle(), d.getDistrito(), "PE");
        d.setLongitud(lonLat[0]);                                        // se corrige el orden
        d.setLatitud(lonLat[1]);
        return d;
    }

    @Override
    public String getProveedor() {
        return "OpenStreetMap";
    }
```

```
}
```

Evidencia de funcionamiento

El cliente recorre una lista de `INotificador` sin conocer el proveedor concreto. Cada adaptador traduce la solicitud al formato de su proveedor (líneas `[SMTP]`, `[SMS-GW]` y `[WA-API]`). El último caso muestra cómo el adaptador de correo convierte la excepción del proveedor en un resultado `false`, evitando que el error se propague al negocio:

Salida en consola (ejecución de uni.util.PruebaPatrones):

```
   [SMTP] from=notificaciones@logiruta.pe to=carla.romero@correo.pe subject="Envio
LR00000007"
  CORREO -> enviado=true
   [SMS-GW] 912345678 <- "Envio LR00000007: Su paquete está EN_RUTA"
  SMS -> enviado=true
   [WA-API] +51912345678 template=estado_envio body="Envio LR00000007 - Su paquete está
EN_RUTA"
  WHATSAPP -> enviado=true
   [CorreoAdapter] rechazo: 550 Invalid recipient
  CORREO con destinatario inválido -> enviado=false
```

5.2 Patrón Facade

Problema

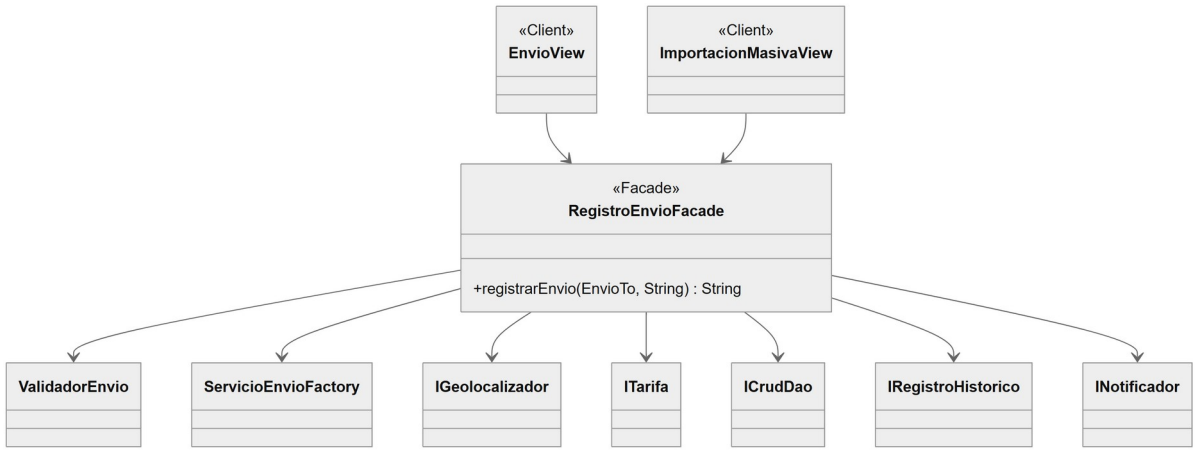
Registrar un envío (CU02) involucra **ocho subsistemas**: validación de datos, selección de la regla de servicio, geolocalización del destino, cálculo de la tarifa con adicionales, generación del correlativo y persistencia transaccional, registro del primer estado en el historial, notificación al destinatario y armado del comprobante. Ese mismo proceso es invocado desde el formulario de ventanilla (`EnvioView`), la importación masiva del cliente corporativo y, a futuro, la integración con tiendas virtuales. Si cada cliente coordina los subsistemas por su cuenta, la lógica se **duplica** (problema de duplicidad identificado en el Capítulo I), los formularios se acoplan a ocho clases y cualquier cambio en el orden del proceso debe replicarse en todos ellos.

Justificación

El patrón Facade proporciona una **interfaz unificada y simplificada** a un conjunto de interfaces de un subsistema. `RegistroEnvioFacade` (en `uni.controller`, en la misma línea que los controladores de AppEmpresa, que ya actúan como intermediarios entre vista y DAO) expone una sola operación, `registrarEnvio(envio, idempleado)`, que encapsula el orden correcto de los pasos. Los clientes dependen de una sola clase, se garantiza que todos los canales registran los envíos de la misma manera y la complejidad interna puede evolucionar sin afectar a las vistas. A diferencia del Adapter (que traduce una interfaz), el Facade **simplifica** el acceso a muchas.

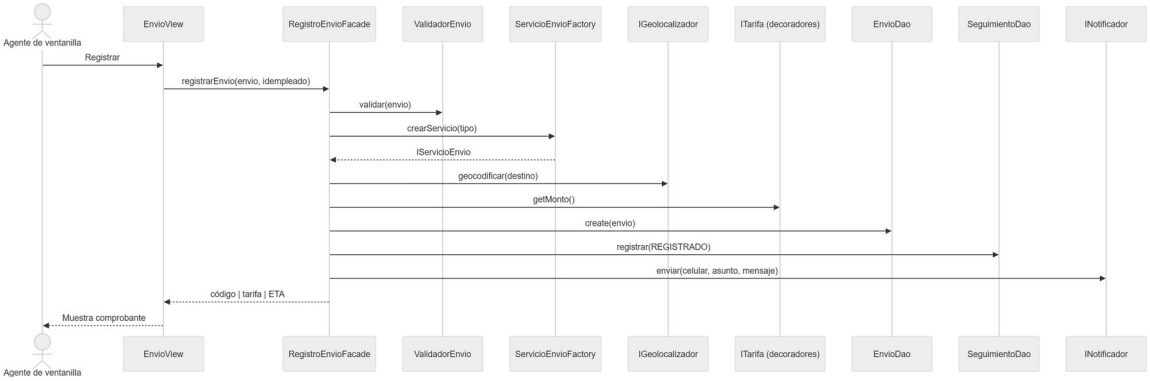
UML

Figura 15
Diagrama de clases del patrón Facade de registro de envíos



Nota. Elaboración propia del grupo de trabajo.

Figura 16
Diagrama de secuencia de la operación registrarEnvio()



Nota. Elaboración propia del grupo de trabajo.

Código

Código 33
uni.controller.RegistroEnvioFacade – fachada del registro de envíos

```
package uni.controller;

/**
 * PATRÓN FACADE.
 * Un único método registrarEnvio() oculta a los formularios (EnvioView,
 * importación masiva) los 8 subsistemas que intervienen en el registro.
 */
public class RegistroEnvioFacade {

    private final ValidadorEnvio validador = new ValidadorEnvio();
    private final ServicioEnvioFactory fabricaServicios = new ServicioEnvioFactory();
    private final ICrudDao<EnvioTo> envioDao;
    private final IRegistroHistorico<SeguimientoTo> historialDao;
    private final IGeolocalizador geolocalizador;
    private final INotificador notificador;

    public RegistroEnvioFacade(ICrudDao<EnvioTo> envioDao,
                               IRegistroHistorico<SeguimientoTo> historialDao,
                               IFabricaIntegraciones integraciones) {
        this.envioDao = envioDao;
        this.historialDao = historialDao;
        this.geolocalizador = integraciones.crearGeolocalizador();
        this.notificador = integraciones.crearNotificador();
```

```
    }

    /** Operación simplificada que usa la capa de presentación. */
    public String registrarEnvio(EnvioTo envio, String idempleado) throws Exception {
        // 1. Validación de datos
        List<String> errores = validador.validar(envio);
        if (!errores.isEmpty()) {
            throw new IllegalArgumentException(errores.toString());
        }
        // 2. Regla del servicio (Factory)
        IServicioEnvio servicio = fabricaServicios.crearServicio(envio.getServicio());
        if (!servicio.admite(envio)) {
            throw new IllegalArgumentException("El servicio " + servicio.getTipo()
                    + " no admite este envío.");
        }
        // 3. Geolocalización del destino (Adapter vía Abstract Factory)
        if (envio.getDestino() != null) {
            geolocalizador.geocodificar(envio.getDestino());
        }
        // 4. Tarifa con servicios adicionales (Decorator)
        ITarifa tarifa = armarTarifa(servicio, envio);
        envio.setCosto(tarifa.getMonto());
        // 5. Persistencia del envío (DAO)
        envioDao.create(envio);
        // 6. Primer registro de trazabilidad
        historialDao.registrar(new SeguimientoTo(envio.getCodigo(), EstadoEnvio.REGISTRADO,
                "Agencia de origen", "Envío registrado", idempleado, LocalDateTime.now()));
        // 7. Notificación al destinatario (Adapter)
        LocalDate eta = servicio.calcularFechaEstimada(envio, LocalDate.now());
        notificador.enviar(envio.getTelefonoDestinatario(), "LogiRuta " + envio.getCodigo(),
                "Su envio fue registrado. Llegada estimada: " + eta);
        // 8. Resultado para la vista
        return envio.getCodigo() + " | " + tarifa.getDescripcion()
                + " | S/ " + String.format("%.2f", tarifa.getMonto()) + " | ETA " + eta;
    }

    private ITarifa armarTarifa(IServicioEnvio servicio, EnvioTo envio) {
        ITarifa t = new TarifaBase(servicio, envio);
        if (envio.isConSeguro()) {
            t = new SeguroDecorator(t, envio.getValorDeclarado());
        }
        if (envio.isFragil()) {
            t = new EmbalajeFragilDecorator(t);
        }
        if (envio.isRequiereFirma()) {
            t = new FirmaDigitalDecorator(t);
        }
        return t;
    }
}
```

Evidencia de funcionamiento

Con una sola llamada a la fachada se valida el envío, se geolocaliza el destino, se calcula la tarifa con los adicionales (S/ 26.75 base express + S/ 8.00 seguro + S/ 4.50 firma = S/ 39.25), se registra el envío y su historial, y se notifica al destinatario por WhatsApp con la fecha estimada:

Salida en consola (ejecución de uni.util.PruebaPatrones):

```
   [WA-API] +51976543210 template=estado_envio body="LogiRuta LR00000001 - Su envio fue
registrado. Llegada estimada: 2026-10-27"
Resultado: LR00000001 | Servicio EXPRESS (2.5 kg) + Seguro + Firma digital | S/ 39.25 | ETA
2026-10-27
Destino geolocalizado: (-12.04150, -77.00150)
```

5.3 Patrón Decorator

Problema

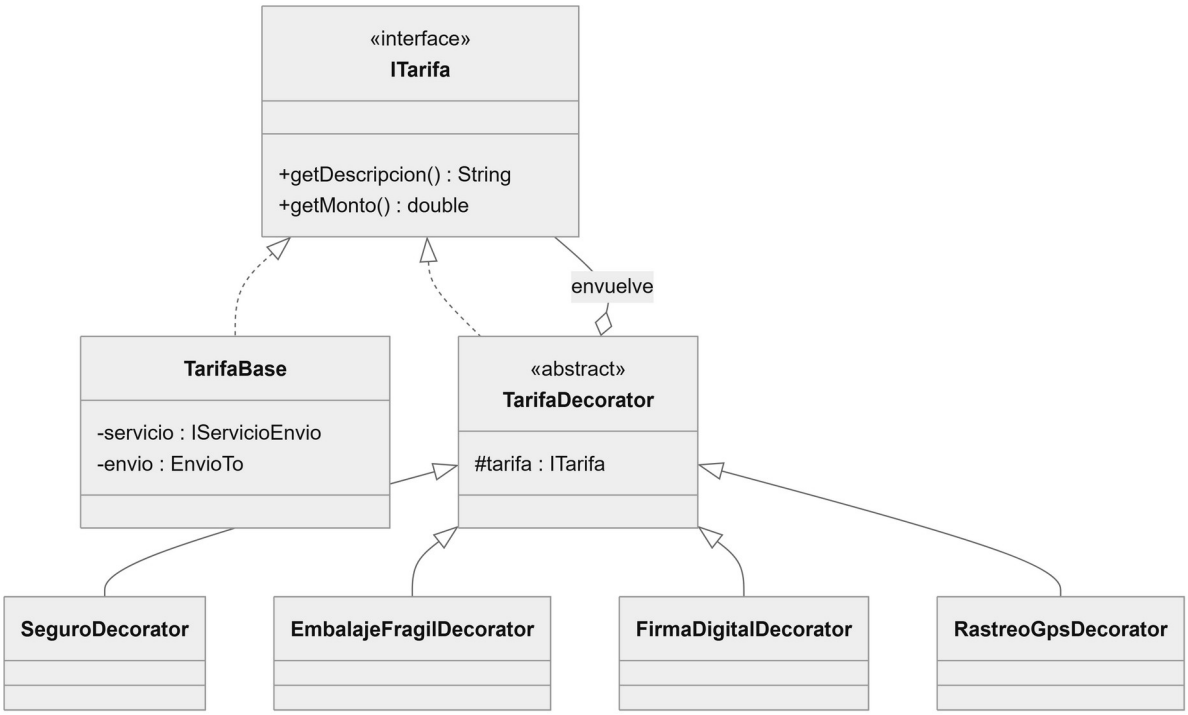
Sobre la tarifa de cualquier servicio, el cliente puede contratar **servicios adicionales combinables**: seguro contra pérdida (2 % del valor declarado, mínimo S/ 5.00), embalaje para contenido frágil (S/ 8.00), confirmación con firma digital (S/ 4.50) y rastreo GPS en tiempo real con alertas inmediatas (+10 % de la tarifa acumulada). Con herencia, cada combinación requeriría una subclase (`EstandarConSeguro`, `EstandarConSeguroYFragil`, `ExpressConSeguroFragilYGPS`...): con 5 servicios y 4 adicionales resultan $5 \times 2^4 =$ **80 clases**, y cada nuevo adicional duplica esa cantidad. Además, las combinaciones se eligen en tiempo de ejecución, en la ventanilla.

Justificación

El patrón Decorator agrega responsabilidades a un objeto **de forma dinámica**, envolviéndolo en objetos decoradores que comparten su misma interfaz. Es la misma idea del ejemplo de transporte con seguro y rastreo visto en la sesión 09, aplicada al tarifario real de LogiRuta. `TarifaBase` (componente concreto) calcula el precio del servicio; cada decorador envuelve una `ITarifa`, añade su costo y su descripción, y delega el resto. Los adicionales se apilan en cualquier orden y cantidad: con 4 decoradores se cubren todas las combinaciones, y un nuevo adicional (por ejemplo, “entrega nocturna”) es una sola clase nueva. El orden importa cuando un adicional es porcentual: el rastreo GPS se aplica sobre el monto acumulado, por eso se coloca al final.

UML

Figura 17
Diagrama de clases del patrón Decorator para tarifas



Nota. Elaboración propia del grupo de trabajo.

Código

Código 34
uni.service.ITarifa – Component

```
package uni.service;

/** PATRÓN DECORATOR - Component: tarifa de un envío. */
public interface ITarifa {

    String getDescripcion();

    double getMonto();
}
```

Código 35
uni.service.TarifaBase – ConcreteComponent

```
package uni.service;

/** PATRÓN DECORATOR - ConcreteComponent: tarifa del servicio contratado. */
public class TarifaBase implements ITarifa {

    private final IServicioEnvio servicio;
    private final EnvioTo envio;

    public TarifaBase(IServicioEnvio servicio, EnvioTo envio) {
        this.servicio = servicio;
        this.envio = envio;
    }

    @Override
    public String getDescripcion() {
        return "Servicio " + servicio.getTipo() + " (" + envio.getPesoKg() + " kg)";
    }
```

```
    @Override
    public double getMonto() {
        return servicio.calcularTarifa(envio);
    }
}
```

Código 36
uni.service.TarifaDecorator – Decorator abstracto

```
package uni.service;

/** PATRÓN DECORATOR - Decorator abstracto: envuelve otra ITarifa. */
public abstract class TarifaDecorator implements ITarifa {

    protected final ITarifa tarifa;

    protected TarifaDecorator(ITarifa tarifa) {
        this.tarifa = tarifa;
    }

    @Override
    public String getDescripcion() {
        return tarifa.getDescripcion();
    }

    @Override
    public double getMonto() {
        return tarifa.getMonto();
    }

    protected static double redondear(double monto) {
        return Math.round(monto * 100.0) / 100.0;
    }
}
```

Código 37
uni.service.SeguroDecorator – decorador concreto

```
package uni.service;

/** Seguro contra pérdida: 2 % del valor declarado (mínimo S/ 5.00). */
public class SeguroDecorator extends TarifaDecorator {

    private final double valorDeclarado;

    public SeguroDecorator(ITarifa tarifa, double valorDeclarado) {
        super(tarifa);
        this.valorDeclarado = valorDeclarado;
    }

    @Override
    public String getDescripcion() {
        return tarifa.getDescripcion() + " + Seguro";
    }

    @Override
    public double getMonto() {
        return redondear(tarifa.getMonto() + Math.max(5.00, valorDeclarado * 0.02));
    }
}
```

Código 38
uni.service.RastreoGpsDecorator – decorador concreto porcentual

```
package uni.service;

/** Rastreo GPS en tiempo real con alertas inmediatas: +10 % de la tarifa. */
public class RastreoGpsDecorator extends TarifaDecorator {

    public RastreoGpsDecorator(ITarifa tarifa) {
```

```
        super(tarifa);
    }

    @Override
    public String getDescripcion() {
        return tarifa.getDescripcion() + " + Rastreo GPS";
    }

    @Override
    public double getMonto() {
        return redondear(tarifa.getMonto() * 1.10);
    }
}
```

Los decoradores `EmbalajeFragilDecorator` y `FirmaDigitalDecorator` siguen la misma estructura con montos fijos de S/ 8.00 y S/ 4.50 respectivamente.

Evidencia de funcionamiento

Para un envío estándar de 5 kg con valor declarado de S/ 1 500: tarifa base S/ 10.00 + 5 × S/ 1.80 = S/ 19.00; con seguro (2 % de 1 500 = S/ 30.00) = S/ 49.00; con todos los adicionales (49.00 + 8.00 + 4.50 = 61.50, y luego × 1.10 por GPS) = S/ 67.65:

Salida en consola (ejecución de uni.util.PruebaPatrones):

```
  S/   19.00  <- Servicio ESTANDAR (5.0 kg)
  S/   49.00  <- Servicio ESTANDAR (5.0 kg) + Seguro
  S/   67.65  <- Servicio ESTANDAR (5.0 kg) + Seguro + Embalaje frágil + Firma digital +
Rastreo GPS
```

5.4 Patrón Composite

Problema

El caso empresarial indica que el envío estándar “puede pasar por más etapas de consolidación”. En el almacén del Callao, los paquetes se agrupan en **sacos** por zona; los sacos y los paquetes voluminosos se colocan en **paletas**; y paletas, sacos y paquetes sueltos se cargan en el **manifiesto** de un vehículo. Es una estructura parte-todo de profundidad variable (un saco puede ir dentro de una paleta). El sistema necesita calcular el peso total del manifiesto para validar la capacidad del vehículo, contar los paquetes y, sobre todo, **cambiar el estado de todos los paquetes** cuando el vehículo sale a ruta. Tratar por separado paquetes, sacos y paletas obliga a escribir bucles anidados con verificación de tipos (`instanceof`) que se rompen cuando cambia la forma de consolidar.

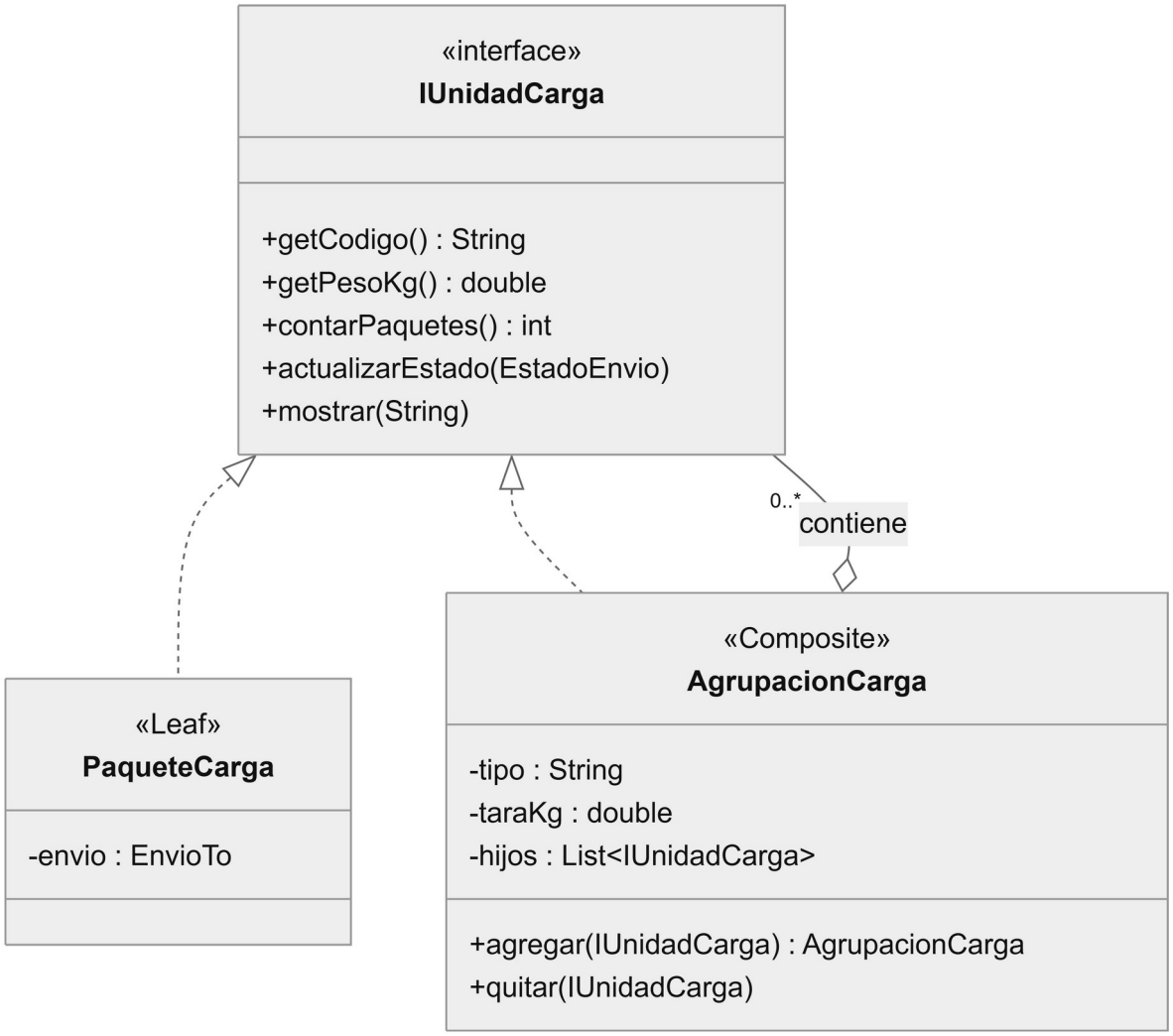
Justificación

El patrón Composite compone objetos en **estructuras de árbol** y permite tratar de manera uniforme a los objetos individuales (hojas) y a las composiciones. `IUnidadCarga` define las operaciones comunes; `PaqueteCarga` es la hoja y `AgrupacionCarga` el compuesto, que contiene una lista de `IUnidadCarga` y resuelve cada operación recorriendo recursivamente a sus hijos. Así, `manifiesto.getPesoKg()` o `manifiesto.actualizarEstado(EN_RUTA)`

funcionan con una sola llamada, cualquiera sea la profundidad del árbol, y un nuevo tipo de agrupación (por ejemplo, un contenedor interprovincial) se incorpora sin modificar el código cliente. En la base de datos el árbol se persiste mediante la auto-relación `unidades_carga.idpadre`.

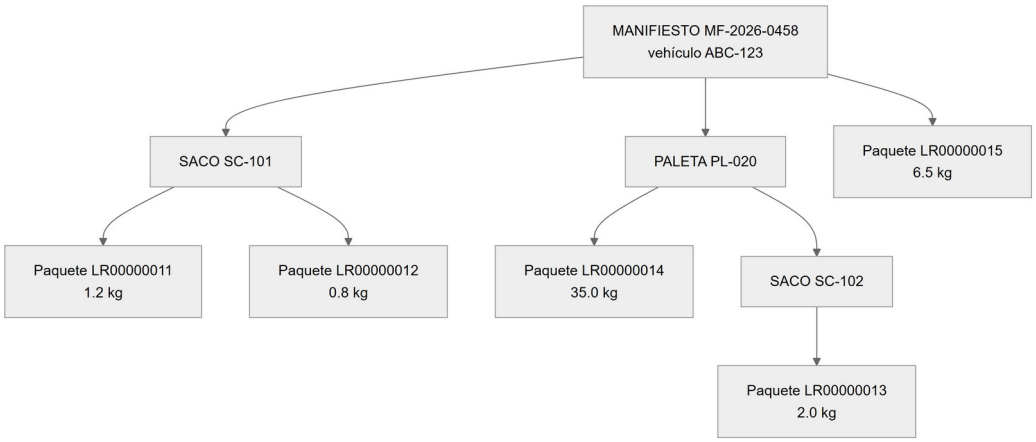
UML

Figura 18
Diagrama de clases del patrón Composite de unidades de carga



Nota. Elaboración propia del grupo de trabajo.

Figura 19
Estructura de objetos del manifiesto MF-2026-0458



Nota. Elaboración propia del grupo de trabajo.

Código

Código 39
uni.service.IUnidadCarga – Component

```
package uni.service;

/**
 * PATRÓN COMPOSITE - Component.
 * Un paquete individual y una agrupación (saco, paleta, manifiesto de
 * vehículo) se manipulan con las mismas operaciones.
 */
public interface IUnidadCarga {

    String getCodigo();

    double getPesoKg();

    int contarPaquetes();

    /** Cambia el estado y lo propaga a todo lo que contiene. */
    void actualizarEstado(EstadoEnvio estado);

    void mostrar(String sangria);
}
```

Código 40
uni.service.PaqueteCarga – Leaf

```
package uni.service;

/** PATRÓN COMPOSITE - Leaf: un envío físico individual. */
public class PaqueteCarga implements IUnidadCarga {

    private final EnvioTo envio;

    public PaqueteCarga(EnvioTo envio) {
        this.envio = envio;
    }

    @Override
    public String getCodigo() { return envio.getCodigo(); }

    @Override
    public double getPesoKg() { return envio.getPesoKg(); }

    @Override
    public int contarPaquetes() { return 1; }
```

```
    @Override
    public void actualizarEstado(EstadoEnvio estado) {
        envio.setEstado(estado);
    }

    @Override
    public void mostrar(String sangria) {
        System.out.printf("%s- Paquete %s  %.1f kg  [%s]%n",
                sangria, getCodigo(), getPesoKg(), envio.getEstado());
    }
}
```

Código 41
uni.service.AgrupacionCarga – Composite

```
package uni.service;

/**
 * PATRÓN COMPOSITE - Composite: saco, paleta o manifiesto de vehículo.
 * Puede contener paquetes u otras agrupaciones (estructura de árbol).
 */
public class AgrupacionCarga implements IUnidadCarga {

    private final String codigo;
    private final String tipo;      // SACO | PALETA | MANIFIESTO
    private final double taraKg;    // peso del contenedor vacío
    private final List<IUnidadCarga> hijos = new ArrayList<>();

    public AgrupacionCarga(String codigo, String tipo, double taraKg) {
        this.codigo = codigo;
        this.tipo = tipo;
        this.taraKg = taraKg;
    }

    public AgrupacionCarga agregar(IUnidadCarga unidad) {
        hijos.add(unidad);
        return this;
    }

    public void quitar(IUnidadCarga unidad) {
        hijos.remove(unidad);
    }

    public List<IUnidadCarga> getHijos() {
        return Collections.unmodifiableList(hijos);
    }

    @Override
    public String getCodigo() { return codigo; }

    @Override
    public double getPesoKg() {
        double total = taraKg;
        for (IUnidadCarga u : hijos) {
            total += u.getPesoKg();          // recursión polimórfica
        }
        return total;
    }

    @Override
    public int contarPaquetes() {
        int total = 0;
        for (IUnidadCarga u : hijos) {
            total += u.contarPaquetes();
        }
        return total;
    }

    @Override
    public void actualizarEstado(EstadoEnvio estado) {
        for (IUnidadCarga u : hijos) {
            u.actualizarEstado(estado);      // se propaga a todo el árbol
        }
```

```
    }

    @Override
    public void mostrar(String sangria) {
        System.out.printf("%s+ %s %s  (%d paquete(s), %.1f kg)%n",
                sangria, tipo, codigo, contarPaquetes(), getPesoKg());
        for (IUnidadCarga u : hijos) {
            u.mostrar(sangria + "    ");
        }
    }
}
```

Evidencia de funcionamiento

El manifiesto contiene un saco, una paleta (que a su vez contiene otro saco) y un paquete suelto. El peso total (58.3 kg) incluye la tara de los contenedores. Una sola llamada a `actualizarEstado(EN_RUTA)` sobre el manifiesto actualiza los cinco paquetes en todos los niveles:

Salida en consola (ejecución de uni.util.PruebaPatrones):

```
Estructura de carga del vehículo:
  + MANIFIESTO MF-2026-0458  (5 paquete(s), 58.3 kg)
      + SACO SC-101  (2 paquete(s), 2.4 kg)
          - Paquete LR00000011  1.2 kg  [CLASIFICADO]
          - Paquete LR00000012  0.8 kg  [CLASIFICADO]
      + PALETA PL-020  (2 paquete(s), 49.4 kg)
          - Paquete LR00000014  35.0 kg  [CLASIFICADO]
          + SACO SC-102  (1 paquete(s), 2.4 kg)
              - Paquete LR00000013  2.0 kg  [CLASIFICADO]
      - Paquete LR00000015  6.5 kg  [CLASIFICADO]
Tras manifiesto.actualizarEstado(EN_RUTA):
  + MANIFIESTO MF-2026-0458  (5 paquete(s), 58.3 kg)
      + SACO SC-101  (2 paquete(s), 2.4 kg)
          - Paquete LR00000011  1.2 kg  [EN_RUTA]
          - Paquete LR00000012  0.8 kg  [EN_RUTA]
      + PALETA PL-020  (2 paquete(s), 49.4 kg)
          - Paquete LR00000014  35.0 kg  [EN_RUTA]
          + SACO SC-102  (1 paquete(s), 2.4 kg)
              - Paquete LR00000013  2.0 kg  [EN_RUTA]
      - Paquete LR00000015  6.5 kg  [EN_RUTA]
```

5.5 Patrón Proxy

Problema

El servicio de seguimiento es el componente más sensible y más consultado del sistema:

- **Seguridad:** en el sistema actual cualquier usuario autenticado puede cambiar el estado de cualquier envío. El negocio exige que solo el personal operativo modifique estados, que un repartidor solo actualice los envíos **asignados a su ruta** y que el destinatario solo consulte.
- **Rendimiento:** los destinatarios consultan el mismo código de seguimiento repetidas veces (en promedio 6 consultas por envío), lo que genera carga innecesaria sobre la base de datos en campaña.
- **Trazabilidad:** cada intento de cambio, permitido o no, debe quedar auditado.

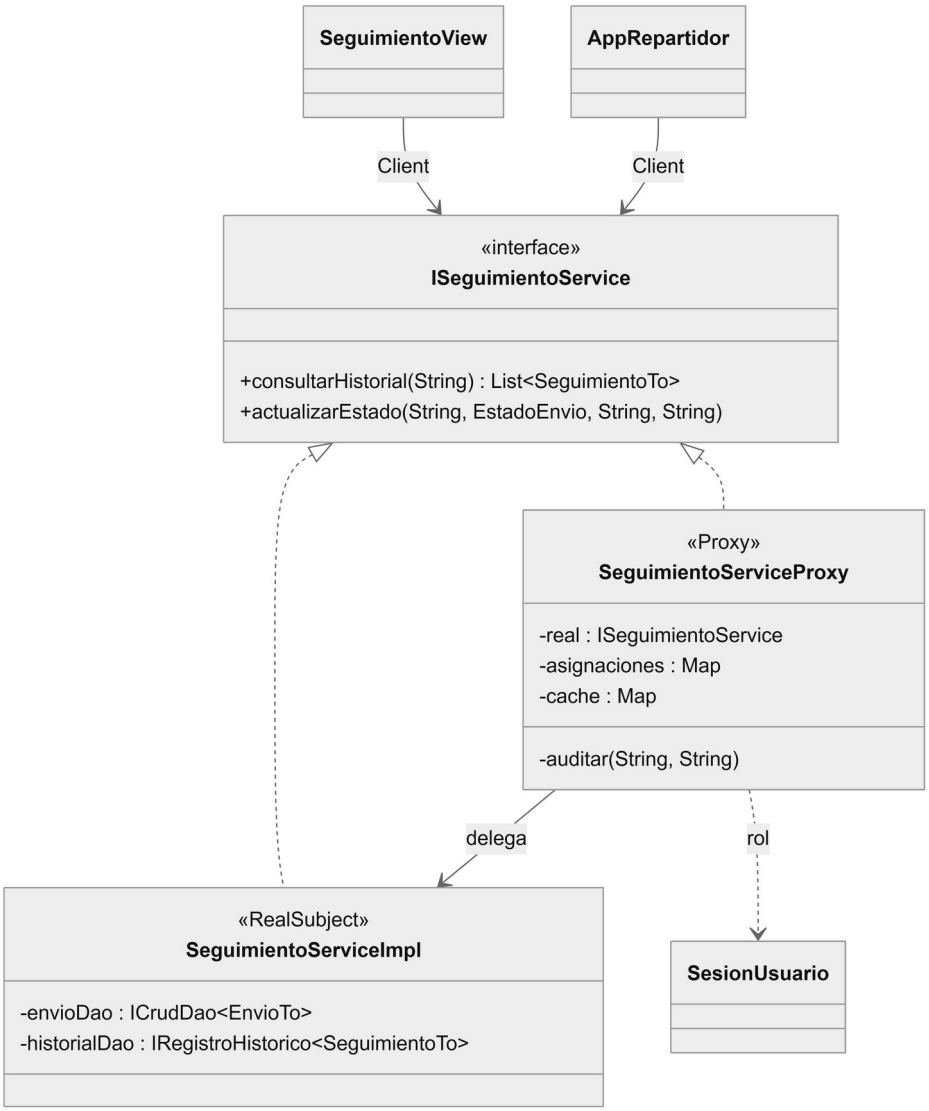
Incorporar estos controles dentro de `SeguimientoServiceImpl` mezclaría seguridad, caché y auditoría con la lógica de seguimiento (violando SRP), y obligaría a modificar el servicio real cada vez que cambie una política de acceso.

Justificación

El patrón Proxy proporciona un **sustituto que controla el acceso** a otro objeto implementando su misma interfaz. `SeguimientoServiceProxy` implementa `ISeguimientoService`, por lo que los clientes (formulario de seguimiento, aplicación del repartidor) no distinguen entre el proxy y el servicio real. Combina tres de los tipos de proxy vistos en clase: **proxy de protección** (verifica rol y asignación usando el Singleton `SesionUsuario`), **proxy inteligente** (registra la auditoría de cada operación) y un mecanismo de **caché** (guarda el historial consultado y lo invalida al actualizar el estado, evitando mostrar datos desactualizados). El servicio real queda enfocado solo en la lógica de seguimiento. Aunque estructuralmente es similar al Decorator, la intención es distinta: el Proxy **controla el acceso** al objeto, mientras que el Decorator **agrega comportamiento**.

UML

Figura 20
Diagrama de clases del patrón Proxy de seguimiento



Nota. Elaboración propia del grupo de trabajo.

Código

Código 42
uni.service.ISeguimientoService – Subject

```
package uni.service;

/** PATRÓN PROXY - Subject: contrato común del servicio real y de su proxy. */
public interface ISeguimientoService {

    List<SeguimientoTo> consultarHistorial(String codigoEnvio) throws Exception;

    void actualizarEstado(String codigoEnvio, EstadoEnvio nuevo,
                          String ubicacion, String observacion) throws Exception;
}
```

Código 43
uni.service.SeguimientoServiceImpl – RealSubject

```
package uni.service;
```

```
/** PATRÓN PROXY - RealSubject: lógica real de seguimiento (accede a datos). */
public class SeguimientoServiceImpl implements ISeguimientoService {

    private final ICrudDao<EnvioTo> envioDao;
    private final IRegistroHistorico<SeguimientoTo> historialDao;

    public SeguimientoServiceImpl(ICrudDao<EnvioTo> envioDao,
                                  IRegistroHistorico<SeguimientoTo> historialDao) {
        this.envioDao = envioDao;
        this.historialDao = historialDao;
    }

    @Override
    public List<SeguimientoTo> consultarHistorial(String codigoEnvio) throws Exception {
        System.out.println("   (consulta a la base de datos: historial de " + codigoEnvio +
")");
        return historialDao.listarPor(codigoEnvio);
    }

    @Override
    public void actualizarEstado(String codigoEnvio, EstadoEnvio nuevo,
                                 String ubicacion, String observacion) throws Exception {
        EnvioTo envio = envioDao.find(codigoEnvio);
        if (envio == null) {
            throw new IllegalArgumentException("Envío no existe: " + codigoEnvio);
        }
        envio.setEstado(nuevo);
        envioDao.update(envio);
        String idempleado = SesionUsuario.getInstancia().getUsuario().getIdempleado();
        historialDao.registrar(new SeguimientoTo(codigoEnvio, nuevo, ubicacion,
                observacion, idempleado, LocalDateTime.now()));
    }
}
```

Código 44

uni.service.SeguimientoServiceProxy – Proxy de protección, caché y auditoría

```
package uni.service;

/**
 * PATRÓN PROXY (de protección + caché + auditoría).
 * Tiene la misma interfaz que el servicio real y controla el acceso a él:
 *  1) Protección: solo ciertos roles cambian estados; un repartidor solo
 *     modifica los envíos que tiene asignados.
 *  2) Caché: las consultas repetidas de tracking no vuelven a la BD.
 *  3) Auditoría: cada intento queda registrado.
 */
public class SeguimientoServiceProxy implements ISeguimientoService {

    private static final Set<RolUsuario> ROLES_OPERATIVOS = EnumSet.of(
            RolUsuario.REPARTIDOR, RolUsuario.OPERADOR_ALMACEN,
            RolUsuario.SUPERVISOR_RUTA, RolUsuario.ADMINISTRADOR);

    private final ISeguimientoService real;
    private final Map<String, String> asignaciones;   // codigoEnvio -> idrepartidor
    private final Map<String, List<SeguimientoTo>> cache = new HashMap<>();

    public SeguimientoServiceProxy(ISeguimientoService real, Map<String, String>
asignaciones) {
        this.real = real;
        this.asignaciones = asignaciones;
    }

    @Override
    public List<SeguimientoTo> consultarHistorial(String codigoEnvio) throws Exception {
        auditar("CONSULTA", codigoEnvio);
        if (cache.containsKey(codigoEnvio)) {
            System.out.println("   (respuesta desde caché del proxy)");
            return cache.get(codigoEnvio);
        }
        List<SeguimientoTo> historial = real.consultarHistorial(codigoEnvio);
        cache.put(codigoEnvio, historial);
```

```
        return historial;
    }

    @Override
    public void actualizarEstado(String codigoEnvio, EstadoEnvio nuevo,
                                 String ubicacion, String observacion) throws Exception {
        EmpleadoTo usuario = SesionUsuario.getInstancia().getUsuario();
        auditar("ACTUALIZAR->" + nuevo, codigoEnvio);

        if (!ROLES_OPERATIVOS.contains(usuario.getRol())) {
            throw new SecurityException("El rol " + usuario.getRol()
                    + " no puede modificar estados.");
        }
        if (usuario.getRol() == RolUsuario.REPARTIDOR
                && !usuario.getIdempleado().equals(asignaciones.get(codigoEnvio))) {
            throw new SecurityException("El envío " + codigoEnvio
                    + " no está asignado al repartidor " + usuario.getIdempleado());
        }
        real.actualizarEstado(codigoEnvio, nuevo, ubicacion, observacion);
        cache.remove(codigoEnvio);   // invalida la caché para no mostrar datos viejos
    }

    private void auditar(String accion, String codigoEnvio) {
        EmpleadoTo u = SesionUsuario.getInstancia().getUsuario();
        System.out.printf("[AUDITORÍA] %s (%s) -> %s sobre %s%n",
                u.getIdempleado(), u.getRol(), accion, codigoEnvio);
    }
}
```

Evidencia de funcionamiento

Tres usuarios intentan actualizar el mismo envío: el destinatario es rechazado por su rol, un repartidor que no tiene asignado el envío es rechazado, y el repartidor asignado actualiza correctamente. Luego, de dos consultas consecutivas, solo la primera accede a la base de datos y la segunda se responde desde la caché. Todas las operaciones quedan auditadas:

Salida en consola (ejecución de uni.util.PruebaPatrones):

```
[AUDITORÍA] D0001 (DESTINATARIO) -> ACTUALIZAR->ENTREGADO sobre LR00000001
   DENEGADO: El rol DESTINATARIO no puede modificar estados.
[AUDITORÍA] E0011 (REPARTIDOR) -> ACTUALIZAR->EN_RUTA sobre LR00000001
   DENEGADO: El envío LR00000001 no está asignado al repartidor E0011
[AUDITORÍA] E0010 (REPARTIDOR) -> ACTUALIZAR->EN_RUTA sobre LR00000001
   OK: Repartidor asignado actualizó a EN_RUTA
Consulta de tracking (2 veces):
[AUDITORÍA] E0010 (REPARTIDOR) -> CONSULTA sobre LR00000001
   (consulta a la base de datos: historial de LR00000001)
[AUDITORÍA] E0010 (REPARTIDOR) -> CONSULTA sobre LR00000001
   (respuesta desde caché del proxy)
   EN_RUTA      | Barranco               | Actualización desde app móvil
```

5.6 Patrón Bridge

Problema

La gerencia necesita reportes estratégicos (incidencias por zona, cumplimiento de tiempos prometidos y productividad de reparto) en **tres formatos de salida**: tabla en pantalla (`InformeView`), CSV para analizar en Excel y PDF generado con JasperReports (como `listaEmpleados.jrxml` de AppEmpresa). Si cada combinación se implementa como una subclase (`ReporteIncidenciasPdf`, `ReporteIncidenciasCsv`, `ReporteCumplimientoPdf`...), se obtienen 3 × 3 = **9 clases**, con la lógica de cálculo duplicada

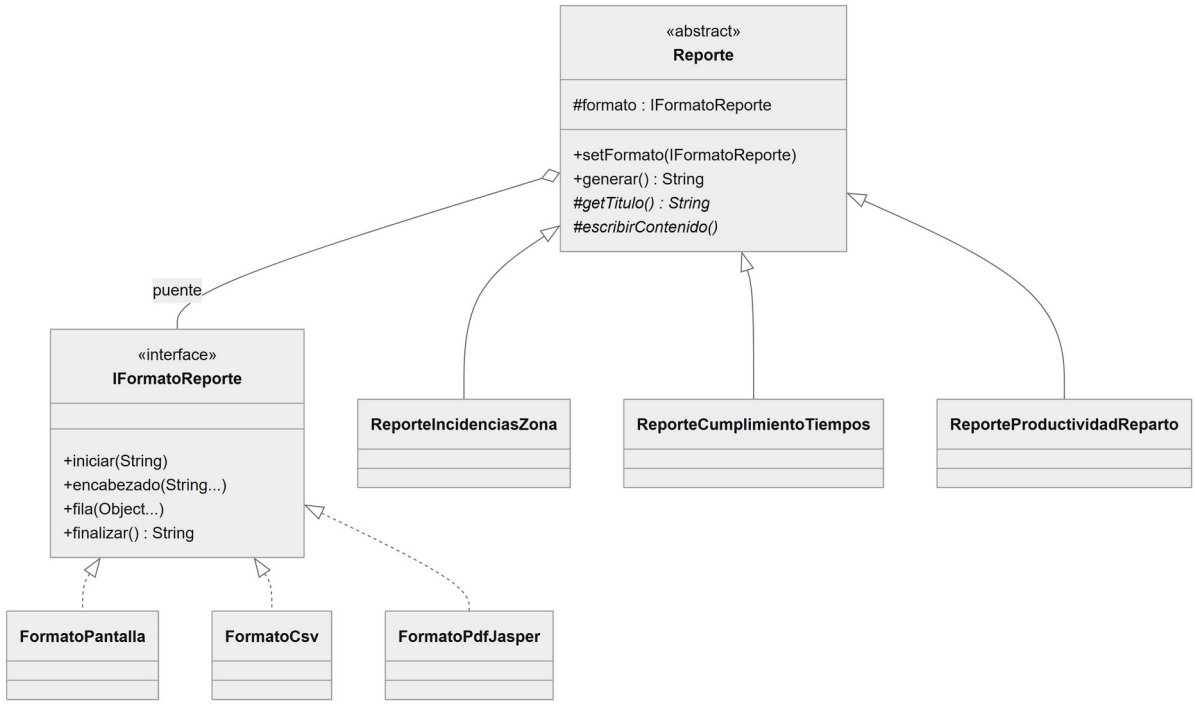
en cada formato; agregar un reporte o un formato (por ejemplo, un *dashboard* web) multiplica nuevamente las clases. Este fenómeno es la “explosión de clases” que el patrón busca evitar.

Justificación

El patrón Bridge **separa una abstracción de su implementación** para que ambas varíen de forma independiente. Se identificaron dos dimensiones de cambio: *qué información contiene el reporte* (abstracción `Reporte` y sus refinamientos) y *cómo se presenta* (implementador `IFormatoReporte` y sus formatos concretos). La clase `Reporte` mantiene una referencia —el “puente”— a un `IFormatoReporte` y le delega el dibujo, mientras ella solo calcula los datos. Con esta estructura, 3 reportes + 3 formatos = **6 clases** en lugar de 9, y cada nueva variante suma una sola clase. El formato puede cambiarse incluso en tiempo de ejecución con `setFormato()`. Este diseño cumple el objetivo del caso de “convertir la información en ventaja competitiva”.

UML

Figura 21
Diagrama de clases del patrón Bridge para reportes estratégicos



Nota. Elaboración propia del grupo de trabajo.

Código

Código 45
uni.reportes.Reporte – Abstraction

```
package uni.reportes;

/**
 * PATRÓN BRIDGE - Abstraction.
 * Mantiene una referencia (el "puente") al formato de salida.
 */
```

```
public abstract class Reporte {

    protected IFormatoReporte formato;

    protected Reporte(IFormatoReporte formato) {
        this.formato = formato;
    }

    /** El formato puede cambiarse en tiempo de ejecución. */
    public void setFormato(IFormatoReporte formato) {
        this.formato = formato;
    }

    public final String generar() {
        formato.iniciar(getTitulo());
        escribirContenido();
        return formato.finalizar();
    }

    protected abstract String getTitulo();

    protected abstract void escribirContenido();
}
```

Código 46
uni.reportes.IFormatoReporte – Implementor

```
package uni.reportes;

/**
 * PATRÓN BRIDGE - Implementor.
 * Define CÓMO se dibuja un reporte (pantalla, CSV para Excel, PDF Jasper),
 * independiente de QUÉ información contiene.
 */
public interface IFormatoReporte {

    void iniciar(String titulo);

    void encabezado(String... columnas);

    void fila(Object... valores);

    String finalizar();
}
```

Código 47
uni.reportes.ReporteIncidenciasZona – RefinedAbstraction

```
package uni.reportes;

/** RefinedAbstraction: zonas con más incidencias (decisión estratégica). */
public class ReporteIncidenciasZona extends Reporte {

    private final List<IncidenciaTo> incidencias;

    public ReporteIncidenciasZona(IFormatoReporte formato, List<IncidenciaTo> incidencias) {
        super(formato);
        this.incidencias = incidencias;
    }

    @Override
    protected String getTitulo() {
        return "Incidencias por zona";
    }

    @Override
    protected void escribirContenido() {
        Map<String, Integer> conteo = new TreeMap<>();
        for (IncidenciaTo i : incidencias) {
            conteo.merge(i.getZona(), 1, Integer::sum);
        }
        formato.encabezado("Zona", "Incidencias", "% del total");
```

```
        for (Map.Entry<String, Integer> e : conteo.entrySet()) {
            double pct = 100.0 * e.getValue() / incidencias.size();
            formato.fila(e.getKey(), e.getValue(), String.format("%.1f %%", pct));
        }
    }
}
```

Código 48
uni.reportes.FormatoCsv – ConcreteImplementor

```
package uni.reportes;

/** ConcreteImplementor: CSV separado por ";" para abrir en Excel. */
public class FormatoCsv implements IFormatoReporte {

    private final StringBuilder sb = new StringBuilder();

    @Override
    public void iniciar(String titulo) {
        sb.setLength(0);
        sb.append("# ").append(titulo).append("\n");
    }

    @Override
    public void encabezado(String... columnas) {
        sb.append(String.join(";", columnas)).append("\n");
    }

    @Override
    public void fila(Object... valores) {
        for (int i = 0; i < valores.length; i++) {
            sb.append(i > 0 ? ";" : "").append(valores[i]);
        }
        sb.append("\n");
    }

    @Override
    public String finalizar() {
        return sb.toString();
    }
}
```

Código 49
uni.reportes.FormatoPdfJasper – ConcreteImplementor (JasperReports)

```
package uni.reportes;

/**
 * ConcreteImplementor: PDF con JasperReports (como listaEmpleados.jrxml de
 * AppEmpresa). En la demo se simula el llenado del datasource.
 */
public class FormatoPdfJasper implements IFormatoReporte {

    private String plantilla;
    private final List<Object[]> datasource = new ArrayList<>();

    @Override
    public void iniciar(String titulo) {
        plantilla = titulo.toLowerCase().replace(' ', '_') + ".jasper";
        datasource.clear();
    }

    @Override
    public void encabezado(String... columnas) {
        // Las columnas están definidas en el diseño .jrxml
    }

    @Override
    public void fila(Object... valores) {
        datasource.add(valores);
    }
```

```
    @Override
    public String finalizar() {
        // Real: JasperFillManager.fillReport(plantilla, params, new
JRBeanArrayDataSource(...))
        //       JasperExportManager.exportReportToPdfFile(print, ruta);
        return "PDF generado con plantilla " + plantilla + " (" + datasource.size() + "
filas)";
    }
}
```

Evidencia de funcionamiento

El mismo objeto `ReporteIncidenciasZona` se genera en pantalla y luego en CSV cambiando solo el formato; el reporte de cumplimiento de tiempos se muestra en pantalla y se exporta a PDF. La lógica de cálculo de cada reporte no se duplica:

Salida en consola (ejecución de uni.util.PruebaPatrones):

```
==== INCIDENCIAS POR ZONA ====
Zona               Incidencias         % del total
------------------------------------------------------------
Lima Norte         3                   60.0 %
Lima Sur           1                   20.0 %
Sierra Central     1                   20.0 %
# Incidencias por zona
Zona;Incidencias;% del total
Lima Norte;3;60.0 %
Lima Sur;1;20.0 %
Sierra Central;1;20.0 %
==== CUMPLIMIENTO DE TIEMPOS ====
Servicio           Entregados          A tiempo            Cumplimiento
--------------------------------------------------------------------------------
EXPRESS            420                 401                 95.5 %
ESTANDAR           1310                1189                90.8 %
INTERPROVINCIAL    275                 236                 85.8 %
PDF generado con plantilla cumplimiento_de_tiempos.jasper (3 filas)
```

Comparación de patrones estructurales

Tabla 20
Comparación de los patrones estructurales aplicados en AppLogistica

Patrón	Intención	Problema de LogiRuta que resuelve	Clases en AppLogistica	Requerimientos
Adapter	Traducir una interfaz incompatible a la esperada	Proveedores de notificación y mapas con interfaces distintas	`INotificador`, `CorreoAdapter`, `SmsAdapter`, `WhatsAppAdapter`, `GoogleMapsAdapter`, `OpenStreetMapAdapter`	RF06, RF12, RF21, RNF11
Facade	Simplificar el acceso a un subsistema complejo	Registro de envío con ocho pasos repetidos en varios canales	`RegistroEnvioFacade`	RF03, RF05
Decorator	Agregar responsabilidades dinámicamente	Combinaciones de servicios adicionales sobre la tarifa	`ITarifa`, `TarifaBase`, cuatro decoradores	RF04

Patrón	Intención	Problema de LogiRuta que resuelve	Clases en AppLogistica	Requerimientos
Composite	Tratar de forma uniforme partes y todos	Consolidación de paquetes en sacos, paletas y manifiestos	`IUnidadCarga`, `PaqueteCarga`, `AgrupacionCarga`	RF07, RF09
Proxy	Controlar el acceso a un objeto	Seguridad por rol, caché de consultas y auditoría	`SeguimientoServiceProxy`	RF09, RF11, RF20, RNF01, RNF07
Bridge	Separar abstracción e implementación	Explosión de clases reporte × formato	`Reporte`, `IFormatoReporte` y sus variantes	RF17, RF18, RF19

Relación entre los patrones. Los patrones del diseño no actúan de forma aislada, sino que colaboran: la **Facade** de registro utiliza internamente la **Factory** de servicios, la **Abstract Factory** de integraciones (cuyos productos son **Adapters**) y la cadena de **Decorators** de tarifa; el **Proxy** de seguimiento consulta al **Singleton** `SesionUsuario`; el **Composite** propaga los cambios de estado que luego se registran en el historial segregado por **ISP**; y el **Bridge** permite presentar los datos de las vistas SQL en cualquier formato. Asimismo, se distinguen patrones con estructura parecida pero intención diferente: Adapter (traduce) frente a Facade (simplifica), y Decorator (agrega comportamiento) frente a Proxy (controla el acceso).

Enlace con el Capítulo VI. El diseño ya preparaba el terreno para los patrones de comportamiento: el ciclo de vida de la Figura 2 se implementa con el patrón **State** (eliminando las validaciones de transición basadas en condicionales), las notificaciones automáticas ante cada cambio de estado con **Observer**, las acciones del repartidor con posibilidad de deshacer con **Command**, y la restauración de un envío antes de una reprogramación con **Memento**.

CAPÍTULO VI. PATRONES DE COMPORTAMIENTO

Este capítulo corresponde a la **Unidad 4 del curso (semanas 12 y 13)**: introducción a los patrones de comportamiento, patrón **State**, patrón **Observer**, patrón **Command** y patrón **Memento**. Mientras los patrones creacionales resolvieron *cómo se crean* los objetos y los estructurales *cómo se componen*, los patrones de comportamiento resuelven *cómo se reparten las responsabilidades y cómo se comunican los objetos* cuando el sistema está en ejecución (Gamma et al., 1994). Siguiendo el material de la Sesión 12, cada patrón incluye su ficha (definición, intención, ventajas y desventajas) y, además, se replica el ejercicio de clase de aplicar State y Observer a la clase Empleado (sección 6.5). En el caso de LogiRuta Perú este es precisamente el núcleo del negocio: el **ciclo de vida del envío** cambia de estado decenas de veces, cada cambio debe quedar trazado y comunicado, y el personal operativo necesita corregir errores sin perder la trazabilidad.

En el Avance 1 se dejaron identificados cuatro problemas que los patrones creacionales y estructurales no resolvían. La Tabla 21 los relaciona con el patrón de comportamiento elegido, las clases que lo implementan y los requerimientos funcionales que satisface.

Tabla 21
Problemas del ciclo de vida del envío y patrón de comportamiento aplicado

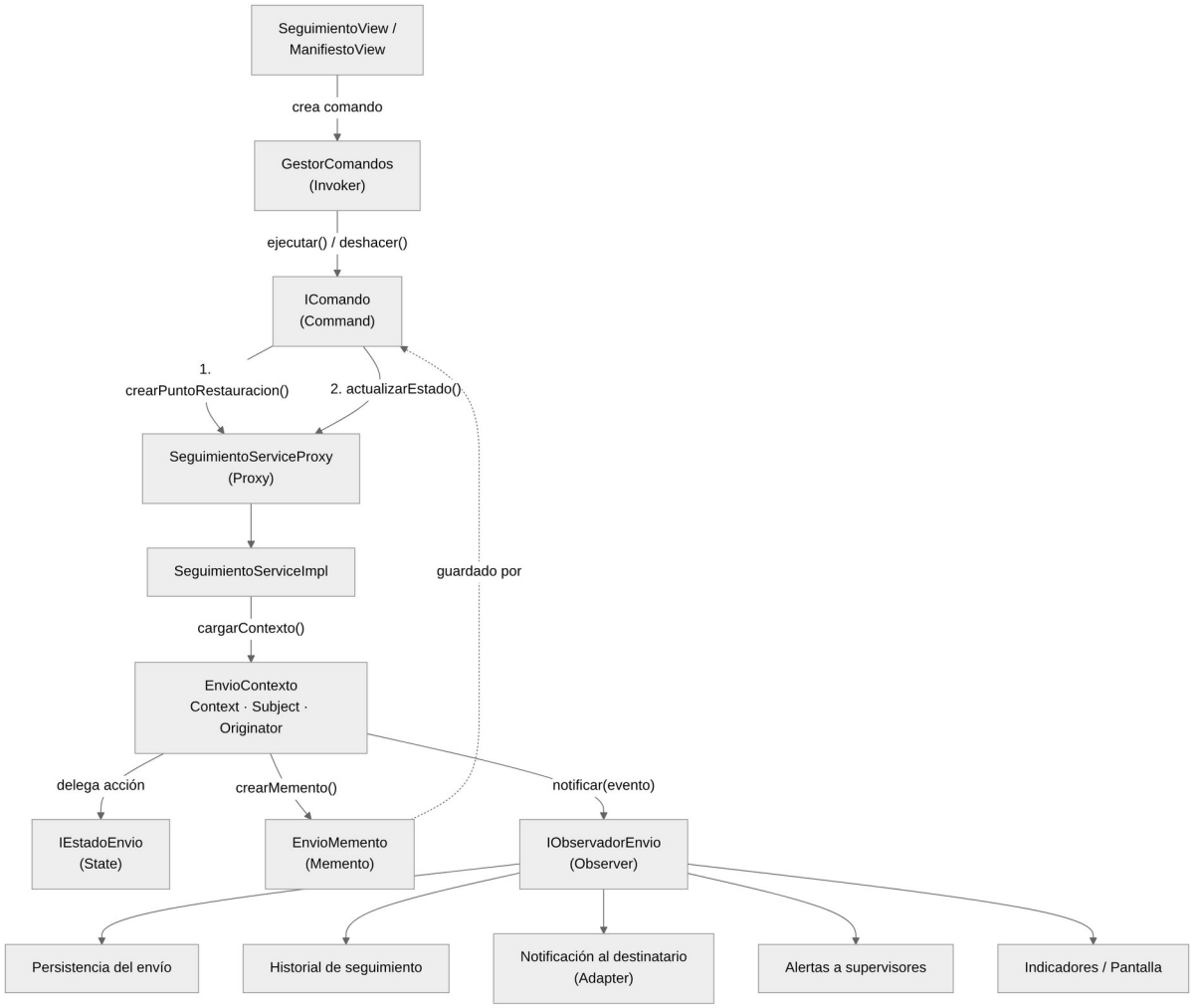
Problema del negocio	Patrón	Clases principales	RF / CU
Cada cambio de estado obliga a guardar, registrar el historial, notificar al destinatario y alertar al supervisor; hoy todo está "cableado" dentro del servicio.	Observer	IObservadorEnvio, EnvioContexto, 5 observadores concretos	RF10, RF12, RF20 CU05
Las reglas de transición dependen del estado y del tipo de servicio (express no consolida, recojo en tienda no sale a ruta) y se validaban con if/switch dispersos.	State	IEstadoEnvio, EstadoEnvioBase, 9 estados, EnvioContexto	RF09, RF13, RF14, RF15 CU05, CU07
El repartidor y el operador se equivocan al escanear y necesitan deshacer; además el móvil pierde señal y las acciones deben guardarse para enviarse luego.	Command	IComando, ComandoEnvioBase, 4 comandos, MacroComando, GestorComandos	RF09, RF13, RF20 CU04, CU05
Antes de reprogramar o de revertir una acción hay que poder volver exactamente al estado anterior sin exponer los datos internos del envío.	Memento	EnvioMemento, EnvioContexto (Originator), HistorialMementos	RF14 CU07

Nota. Elaboración propia del grupo de trabajo.

Los cuatro patrones no se aplicaron de forma aislada: colaboran alrededor de una sola clase, `EnvioContexto`, que es al mismo tiempo el **Context** del patrón State, el **Subject** del patrón Observer y el **Originator** del patrón Memento, mientras que los comandos (Command) usan

los mementos para deshacer. La Figura 22 resume esta colaboración y su relación con los patrones de los capítulos anteriores (Proxy, Adapter, Factory).

Figura 22
Colaboración de los patrones de comportamiento en AppLogistica



Nota. Elaboración propia del grupo de trabajo.

Proyecto funcional. A partir de este avance AppLogistica deja de ser solo una demostración por consola: se implementó la capa `uni.formularios` con los mismos tipos de ventanas de AppEmpresa (`LoginView`, `PrincipalView` con `JDesktopPane` y formularios internos) y los DAO trabajan contra la base de datos MySQL **logistica**. Las clases nuevas respetan la estructura de paquetes del proyecto de referencia, como se detalla en la Tabla 22.

Tabla 22
Clases incorporadas en el Avance 2 según el paquete de AppEmpresa

Paquete	Clases nuevas o modificadas	Responsabilidad
uni.entity	AccionEnvio, EventoEnvio, AlertaTo, ComandoLogTo, ItemCombo; EnvioTo (paso_flujo, intentos, firma)	Transfer Objects y enumeraciones del ciclo de vida

Paquete	Clases nuevas o modificadas	Responsabilidad
uni.service	IObservadorEnvio, ISujetoEnvio, 5 observadores; IEstadoEnvio, EstadoEnvioBase, 9 estados, CatalogoEstados, EnvioContexto; IComando, ComandoEnvioBase, 4 comandos, MacroComando; EnvioMemento	Lógica de negocio y contratos de los cuatro patrones
uni.controller	GestorComandos (Invoker), ClienteController, EmpleadoController	Coordinan las peticiones de los formularios
uni.dao	AlertaDao, ComandoLogDao, NotificacionDao, AuditoriaDao, AsignacionDao, IncidenciaDao, ManifiestoDao, CatalogoDao, ReporteDao, ClienteDao, EmpleadoDao, HistorialLaboralDao	Acceso a MySQL con procedimientos almacenados
uni.formularios	LoginView, PrincipalView, ClienteView, EmpleadoView, EnvioView, SeguimientoView, ManifiestoView, ConsultaEnviosView, ReporteView, AlertasView	Capa de presentación Swing
uni.service (Sesión 12)	IEstadoLaboral, EstadoLaboralBase, EstadoLaboralActivo, EstadoLaboralVacaciones, EstadoLaboralInactivo, EmpleadoContexto, IObservadorEmpleado, RecursosHumanosObserver, OperacionesObserver, GestionLaboralService	State y Observer aplicados al empleado (sección 6.5)
uni.util	HistorialMementos (Caretaker), ConfiguracionSistema, TransicionInvalidaException, PruebaComportamiento	Utilitarios, composición de la aplicación y pruebas
uni.reportes	FormatoHtml, ReporteProductividadReparto	Nuevas variantes del Bridge

Nota. Elaboración propia del grupo de trabajo.

6.1 Patrón Observer

Escenario de aplicación

Tabla 23
Ficha del patrón Observer (formato de la sesión de clase)

Elemento	Descripción
Definición	Establece una relación uno a muchos entre objetos: cuando el sujeto (publicador) cambia, notifica automáticamente a todos sus observadores (suscriptores).
Intención	Desacoplar el sujeto que genera los eventos de los objetos que reaccionan a ellos.
Ventajas	Bajo acoplamiento; nuevas reacciones sin modificar el sujeto (OCP); suscripción y desuscripción en tiempo de ejecución; comunicación tipo difusión.
Desventajas	El orden de notificación no es evidente; un observador lento o con error afecta al resto; puede haber notificaciones en cascada difíciles de depurar.

Nota. Adaptado de Gamma et al. (1994) y del material de clase del curso.

¿Qué problema tengo? Cuando un envío cambia de estado (por ejemplo, de *EN_RUTA* a *OBSERVADO*) el negocio exige varias reacciones simultáneas: (1) persistir el nuevo estado en la tabla `envios`, (2) agregar un registro inmutable en `seguimientos` (RF10), (3) avisar al destinatario por WhatsApp o SMS (RF12), (4) alertar al supervisor si el envío es express o ya acumula tres intentos fallidos, y (5) actualizar los indicadores de productividad que ve la gerencia. En el diseño del Avance 1 `SeguimientoServiceImpl.actualizarEstado()` hacía directamente la persistencia y el historial; agregar las notificaciones y las alertas en ese mismo

método lo convertía en un "método dios" acoplado a cinco subsistemas, y cada nueva reacción (por ejemplo, avisar a la tienda virtual del cliente corporativo) obligaba a modificarlo, violando OCP.

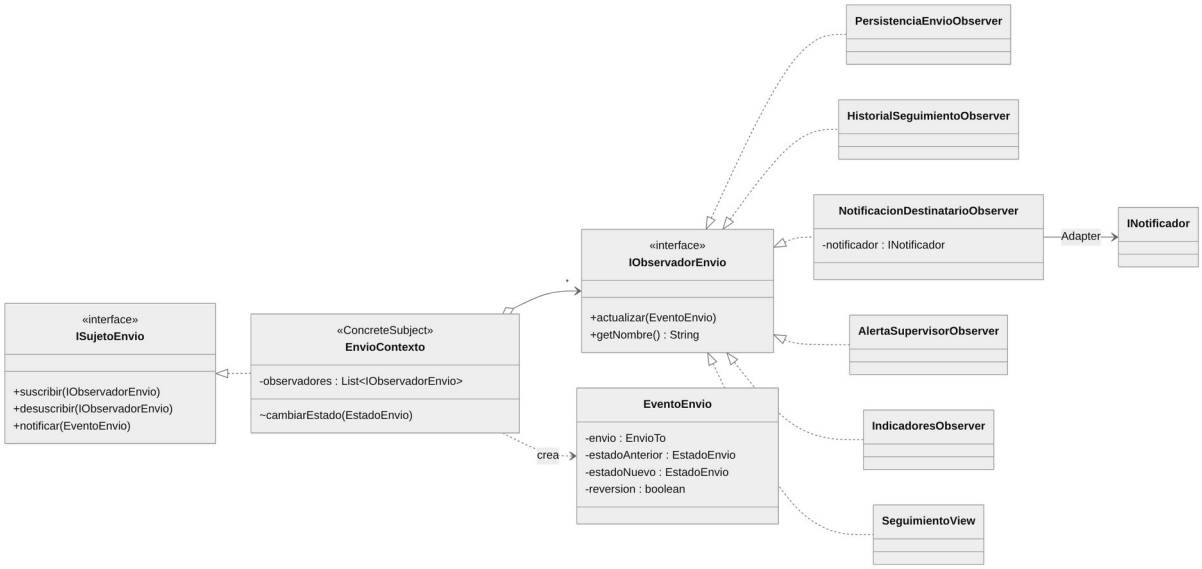
¿Por qué el patrón Observer lo resuelve? Observer define una dependencia de uno a muchos entre objetos: cuando el **sujeto** cambia, todos sus **observadores** son notificados automáticamente, sin que el sujeto conozca sus clases concretas. El envío (sujeto) solo sabe que tiene una lista de `IObservadorEnvio`; cada reacción del negocio es un observador independiente que puede agregarse o retirarse en tiempo de ejecución. Así se cumple OCP (nuevas reacciones = nuevas clases), SRP (cada observador hace una sola cosa) y DIP (el sujeto depende de una interfaz).

- **Subject:** `ISujetoEnvio`, implementado por `EnvioContexto`.
- **Observer:** `IObservadorEnvio` con el método `actualizar(EventoEnvio)`.
- **ConcreteObservers:** `PersistenciaEnvioObserver`, `HistorialSeguimientoObserver`, `NotificacionDestinatarioObserver` (usa el Adapter de notificación del Capítulo V), `AlertaSupervisorObserver`, `IndicadoresObserver` y la propia ventana `SeguimientoView`.
- **Estado notificado:** `EventoEnvio`, objeto inmutable con estado anterior, estado nuevo, ubicación, observación, empleado y un indicador de reversión.

Se eligió el modelo **push** (el sujeto envía el evento completo) porque todos los observadores necesitan los mismos datos y así se evita que vuelvan a consultar la base de datos.

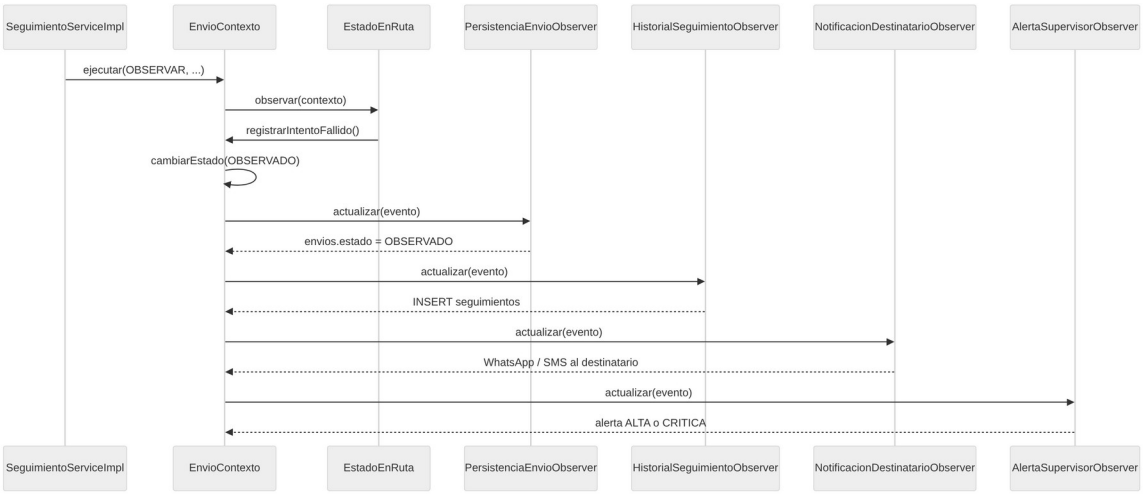
Diagrama UML

Figura 23
Diagrama de clases del patrón Observer en el seguimiento de envíos



Nota. Elaboración propia del grupo de trabajo.

Figura 24
Diagrama de secuencia: un cambio de estado notifica a todos los observadores



Nota. Elaboración propia del grupo de trabajo.

Implementación

Código 50
uni.service.IObservadorEnvio y uni.service.ISujetoEnvio – Observer y Subject

```
package uni.service;

/**
 * PATRÓN OBSERVER - Observer.
 * Todo componente que deba reaccionar cuando un envío cambia de estado
 * (historial, notificación al destinatario, alertas, indicadores, pantallas)
 * implementa esta interfaz. El sujeto no conoce las clases concretas.
 */
public interface IObservadorEnvio {

    void actualizar(EventoEnvio evento);

    /** Nombre legible para la bitácora y para la pantalla de seguimiento. */
    String getNombre();
}

public interface ISujetoEnvio {

    void suscribir(IObservadorEnvio observador);

    void desuscribir(IObservadorEnvio observador);

    void notificar(EventoEnvio evento);
}
```

Código 51
uni.service.EnvioContexto – ConcreteSubject (fragmento)

```
public class EnvioContexto implements ISujetoEnvio {

    private final List<IObservadorEnvio> observadores = new ArrayList<>();
    // ...

    /** Cambia el objeto estado y avisa a los observadores. */
    void cambiarEstado(EstadoEnvio nuevo) {
        EstadoEnvio anterior = envio.getEstado();
        envio.setEstado(nuevo);
        estado = CatalogoEstados.de(nuevo);
        notificar(new EventoEnvio(envio, anterior, nuevo, ubicacion, observacion,
idempleado, false));
    }
```

```
    // ========================================================== OBSERVER
    @Override
    public void suscribir(IObservadorEnvio o) {
        if (!observadores.contains(o)) {
            observadores.add(o);
        }
    }

    @Override
    public void desuscribir(IObservadorEnvio o) {
        observadores.remove(o);
    }

    @Override
    public void notificar(EventoEnvio evento) {
        for (IObservadorEnvio o : new ArrayList<>(observadores)) {
            o.actualizar(evento);
        }
    }
    // ...
}
```

Código 52

uni.service.NotificacionDestinatarioObserver – observador que reutiliza el Adapter

```
public class NotificacionDestinatarioObserver implements IObservadorEnvio {

    private static final Set<EstadoEnvio> HITOS = EnumSet.of(EstadoEnvio.RECOGIDO,
            EstadoEnvio.EN_RUTA, EstadoEnvio.ENTREGADO, EstadoEnvio.OBSERVADO,
            EstadoEnvio.REPROGRAMADO, EstadoEnvio.DEVUELTO);

    private final INotificador notificador;
    private final IBitacoraNotificaciones bitacora;   // puede ser null (pruebas)

    public NotificacionDestinatarioObserver(INotificador notificador,
IBitacoraNotificaciones bitacora) {
        this.notificador = notificador;
        this.bitacora = bitacora;
    }

    @Override
    public void actualizar(EventoEnvio e) {
        EnvioTo envio = e.getEnvio();
        boolean express = envio.getServicio() == TipoServicio.EXPRESS;
        if (!express && !HITOS.contains(e.getEstadoNuevo()) && !e.isReversion()) {
            return;
        }
        String destino = envio.getTelefonoDestinatario();
        if (destino == null || destino.isEmpty()) {
            return;
        }
        String asunto = "LogiRuta " + envio.getCodigo();
        String mensaje = e.isReversion()
                ? "Corregimos el estado de su envío: " + e.getEstadoNuevo()
                : textoPara(e.getEstadoNuevo(), envio);
        boolean ok = notificador.enviar(destino, asunto, mensaje);
        if (bitacora != null) {
            try {
                bitacora.registrar(envio.getCodigo(), notificador.getCanal(), destino,
mensaje, ok);
            } catch (Exception ex) {
                System.out.println("[Notificación] no se pudo registrar: " +
ex.getMessage());
            }
        }
    }
    // textoPara(): mensaje según el estado (RECOGIDO, EN_RUTA, ENTREGADO...)
}
```

Código 53
uni.service.AlertaSupervisorObserver – reglas de escalamiento (fragmento)

```
    /** Reglas de negocio de escalamiento (público para pruebas). */
    public AlertaTo evaluar(EventoEnvio e) {
        EnvioTo envio = e.getEnvio();
        if (e.isReversion()) {
            return e.getEstadoAnterior() == EstadoEnvio.ENTREGADO
                    ? new AlertaTo(envio.getCodigo(), "ALTA", "Se revirtió una entrega: " +
e.getObservacion())
                    : null;
        }
        switch (e.getEstadoNuevo()) {
            case OBSERVADO:
                if (envio.getServicio() == TipoServicio.EXPRESS) {
                    return new AlertaTo(envio.getCodigo(), "CRITICA",
                            "Envío EXPRESS observado: riesgo de incumplir 24 h. " +
texto(e));
                }
                if (envio.getIntentos() >= EstadoObservado.MAX_INTENTOS) {
                    return new AlertaTo(envio.getCodigo(), "CRITICA",
                            "Tercer intento fallido: proceder con la devolución. " +
texto(e));
                }
                return new AlertaTo(envio.getCodigo(), "ALTA", "Envío observado (intento "
                        + envio.getIntentos() + "/" + EstadoObservado.MAX_INTENTOS + "): " +
texto(e));
            case DEVUELTO:
                return new AlertaTo(envio.getCodigo(), "ALTA", "Envío devuelto al
remitente.");
            default:
                return null;
        }
    }
```

Los observadores se registran en un solo lugar, la clase `ConfiguracionSistema` (Singleton de arranque de la aplicación), y `SeguimientoServiceImpl` los suscribe a cada `EnvioContexto` que arma. La ventana de seguimiento se suscribe al abrirse y se desuscribe al cerrarse, demostrando la suscripción dinámica:

Código 54
Registro de observadores en uni.util.ConfiguracionSistema y uni.formularios.SeguimientoView

```
        // Servicio real + observadores (Observer)
        seguimientoReal = new SeguimientoServiceImpl(envioDao, seguimientoDao, new
IncidenciaDao(),
                integraciones.crearFirmaDigital());
        seguimientoReal.agregarObservador(new NotificacionDestinatarioObserver(
                integraciones.crearNotificador(), new NotificacionDao()));
        seguimientoReal.agregarObservador(new AlertaSupervisorObserver(new AlertaDao()));
        seguimientoReal.agregarObservador(indicadores);

// SeguimientoView (implements IObservadorEnvio)
        // --- OBSERVER: la ventana se suscribe al abrir y se desuscribe al cerrar
        ConfiguracionSistema.getInstancia().getPublicadorEventos().agregarObservador(this);
        addInternalFrameListener(new InternalFrameAdapter() {
            @Override
            public void internalFrameClosed(InternalFrameEvent e) {

ConfiguracionSistema.getInstancia().getPublicadorEventos().quitarObservador(SeguimientoView.
this);
            }
        });
```

Pruebas

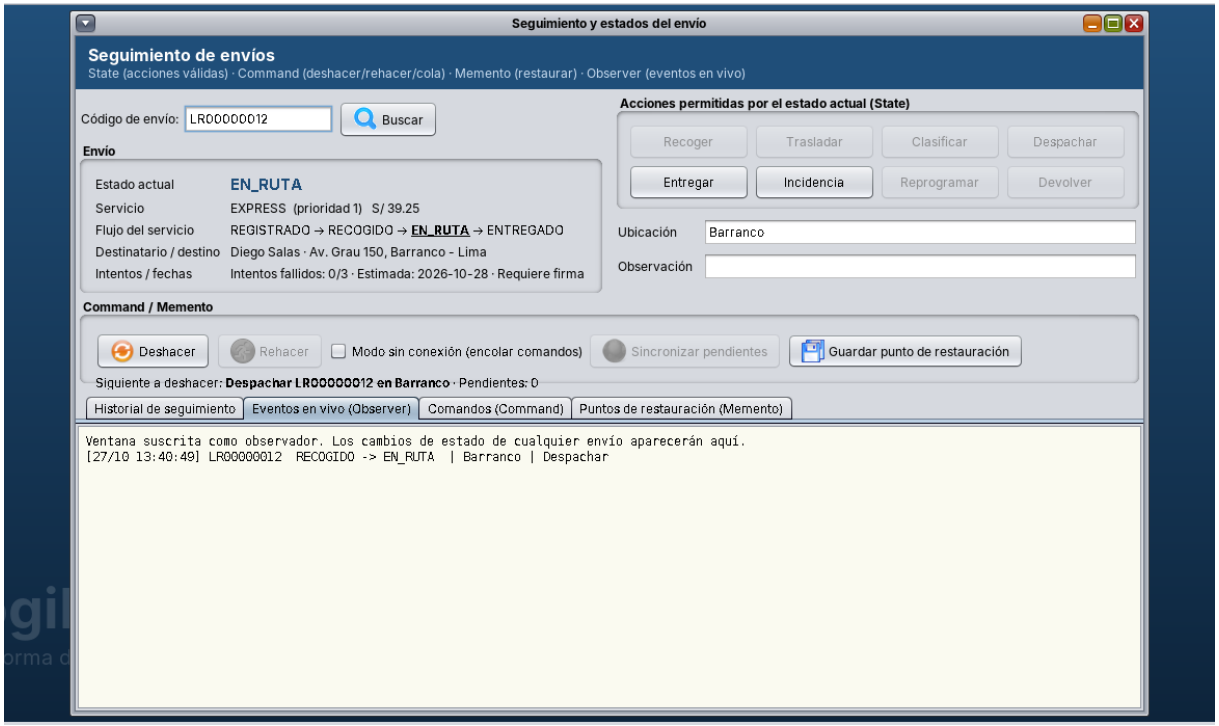
Prueba por consola (`uni.util.PruebaComportamiento`, DAO en memoria): un solo cambio de estado dispara a todos los observadores; luego se desuscribe la pantalla y deja de recibir eventos.

```
Observadores suscritos:
   - Persistencia del envío
   - Historial de seguimiento
   - Notificación al destinatario (WHATSAPP)
   - Alertas a supervisores
   - Indicadores en tiempo real
   - Pantalla
Un solo cambio de estado (RECOGER) en LR00000001:
   [WA-API] +51934555666 template=estado_envio body="LogiRuta LR00000001 - Recogimos su
paquete."
   [Pantalla de seguimiento] LR00000001: REGISTRADO -> RECOGIDO
Despachar y registrar incidencia (express):
   [WA-API] +51934555666 template=estado_envio body="LogiRuta LR00000001 - Su paquete está
en ruta y llegará hoy."
   [Pantalla de seguimiento] LR00000001: RECOGIDO -> EN_RUTA
   [WA-API] +51934555666 template=estado_envio body="LogiRuta LR00000001 - No pudimos
entregar su paquete (intento 1). Lo contactaremos para coordinar."
   [Pantalla de seguimiento] LR00000001: EN_RUTA -> OBSERVADO
Historial registrado por HistorialSeguimientoObserver:
   RECOGIDO     | Agencia Central Lima   | Paquete recogido
   EN_RUTA      | Surco                  | Salida a reparto
   OBSERVADO    | Surco                  | AUSENCIA_DESTINATARIO: Nadie en domicilio
Alertas generadas por AlertaSupervisorObserver: [[CRITICA] LR00000001 - Envío EXPRESS
observado: riesgo de incumplir 24 h. AUSENCIA_DESTINATARIO: Nadie en domicilio]
Indicadores: Sesión: 0 entregados | 1 en ruta | 1 observados | 0 devueltos | 0 reversiones
Se desuscribe la pantalla y se reprograma (la pantalla ya no imprime):
   [WA-API] +51934555666 template=estado_envio body="LogiRuta LR00000001 - Entrega
reprogramada para el 2026-10-28."
Estado final: REPROGRAMADO (paso 2, intentos 1)
```

En la aplicación, la pestaña **Eventos en vivo (Observer)** de `SeguimientoView` muestra los eventos que recibe la ventana como observador, y el formulario **Alertas, notificaciones y bitácora** muestra lo que registraron en MySQL los observadores de notificación y de alertas (Figuras 25 y 26).

Figura 25

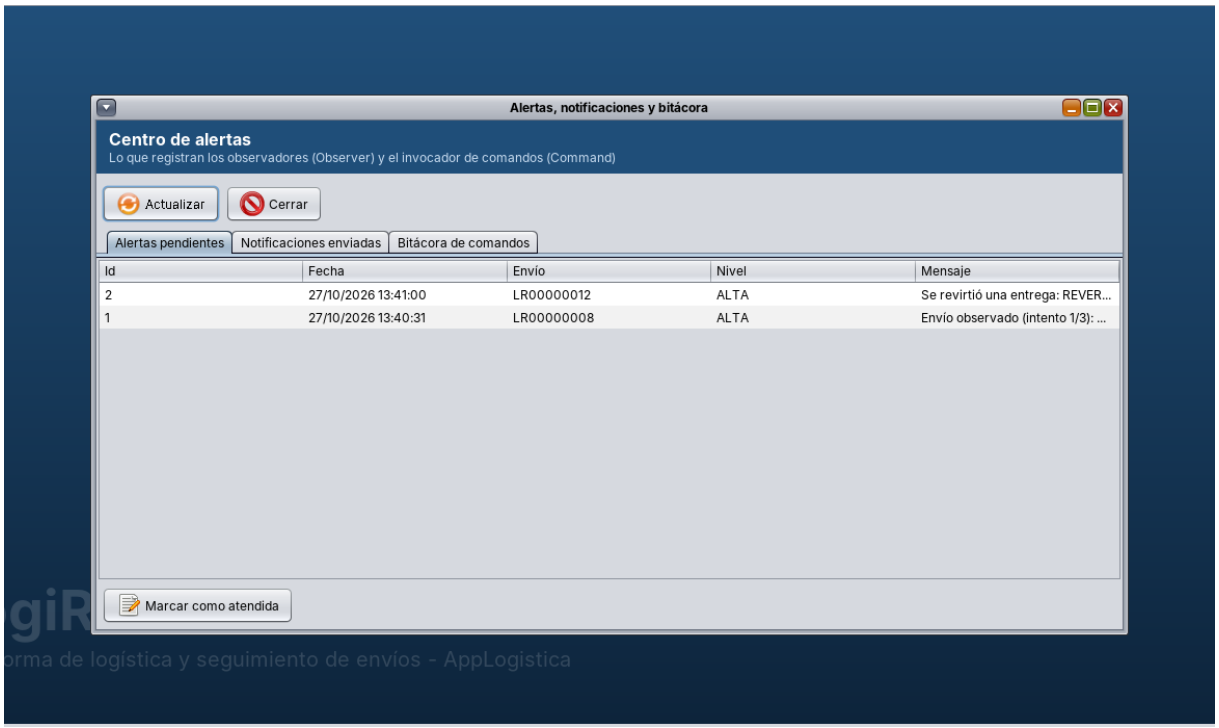
SeguimientoView: la ventana recibe el evento RECOGIDO → EN_RUTA como observador



Nota. Captura de AppLogistica ejecutándose contra MySQL 8 (usuario jtorres, supervisor de ruta).

Figura 26

AlertasView: alertas generadas automáticamente por AlertaSupervisorObserver



Nota. Captura de AppLogistica. Las alertas CRITICA y ALTA se generaron sin código adicional en el servicio.

Tabla 24
Casos de prueba del patrón Observer

Caso	Entrada	Resultado esperado	Resultado obtenido
CP-OBS-01	RECOGER sobre un envío EXPRESS	Se persiste, se registra historial, se envía WhatsApp y la pantalla muestra el evento	Conforme: 6 observadores notificados
CP-OBS-02	Incidencia en envío EXPRESS	Alerta CRITICA "riesgo de incumplir 24 h"	Conforme
CP-OBS-03	Desuscribir la pantalla y reprogramar	La pantalla ya no recibe el evento; los demás sí	Conforme
CP-OBS-04	Clasificar un envío ESTÁNDAR	No se envía WhatsApp (estado interno, no es hito)	Conforme
CP-OBS-05	Deshacer una entrega	Notificación "Corregimos el estado" y alerta ALTA	Conforme

Nota. Elaboración propia del grupo de trabajo.

6.2 Patrón State

Escenario de aplicación

Tabla 25
Ficha del patrón State (formato de la sesión de clase)

Elemento	Descripción
Definición	Permite que un objeto modifique su comportamiento cuando cambia su estado interno; el objeto parece cambiar de clase.
Intención	Evitar muchos if o switch encapsulando cada estado en una clase independiente.
Ventajas	Elimina condicionales complejos; facilita agregar nuevos estados; las transiciones quedan explícitas y se prueban por separado.
Desventajas	Aumenta el número de clases; puede ser excesivo para sistemas con pocos estados o reglas simples.

Nota. Adaptado de Gamma et al. (1994) y del material de clase del curso.

¿Qué problema tengo? El caso empresarial define nueve estados (*registrado, recogido, en tránsito, clasificado, en ruta, entregado, observado, reprogramado y devuelto*) y las transiciones válidas dependen **del estado actual y del tipo de servicio**: un envío express pasa de *RECOGIDO* directamente a *EN_RUTA*; uno estándar debe pasar por tránsito y clasificación; uno interprovincial se clasifica dos veces; el recojo en tienda nunca sale a ruta; un envío solo se reprograma si tiene menos de tres intentos fallidos y no puede marcarse *ENTREGADO* si requiere firma digital y esta no existe. Implementar esto con condicionales produce un `switch` sobre el estado anidado con `if` por servicio que se repite en cada operación (recoger, despachar, entregar...), es difícil de probar y cualquier estado nuevo obliga a modificar todos los métodos.

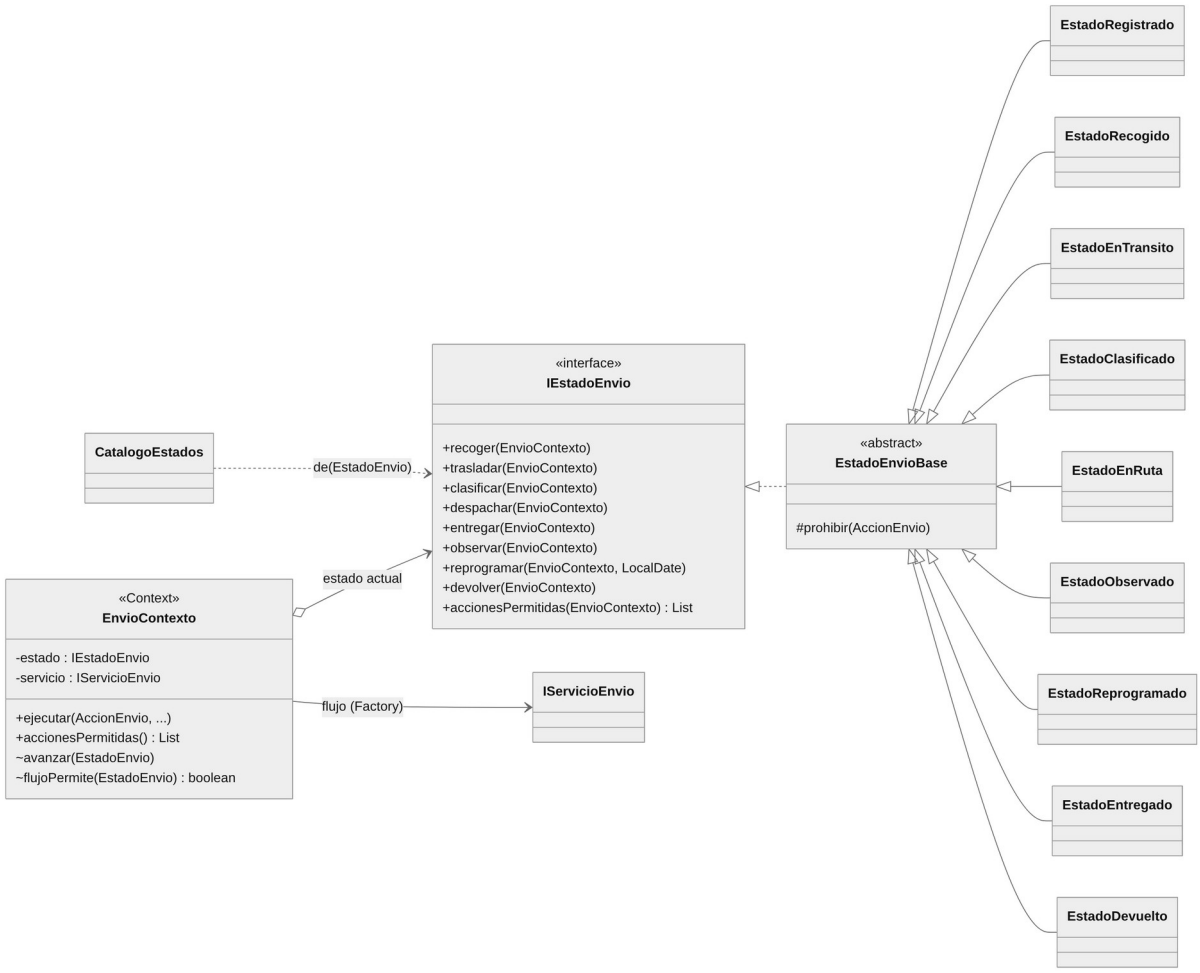
¿Por qué el patrón State lo resuelve? State permite que un objeto altere su comportamiento cuando cambia su estado interno, como si cambiara de clase. Cada estado se convierte en una clase que implementa la interfaz `IEstadoEnvio` y decide qué acciones acepta; el contexto

(`EnvioContexto`) simplemente delega en el estado actual. Las transiciones inválidas se rechazan en un solo lugar (`EstadoEnvioBase.prohibir()`), y el flujo propio de cada servicio se toma de `IServicioEnvio.getFlujoEstados()`, creado por la Factory del Capítulo IV, por lo que **State y Factory colaboran**: el estado define las acciones posibles y el servicio define el orden de las etapas.

- **Context:** `EnvioContexto` (guarda el estado actual, el paso del flujo y el número de intentos).
- **State:** `IEstadoEnvio` con una operación por acción del negocio.
- **ConcreteStates:** `EstadoRegistrado`, `EstadoRecogido`, `EstadoEnTransito`, `EstadoClasificado`, `EstadoEnRuta`, `EstadoObservado`, `EstadoReprogramado`, `EstadoEntregado` y `EstadoDevuelto`.
- **Persistencia del estado:** columnas `estado`, `paso_flujo` e `intentos` de la tabla `envios`; `CatalogoEstados` convierte el valor guardado en el objeto estado correspondiente.

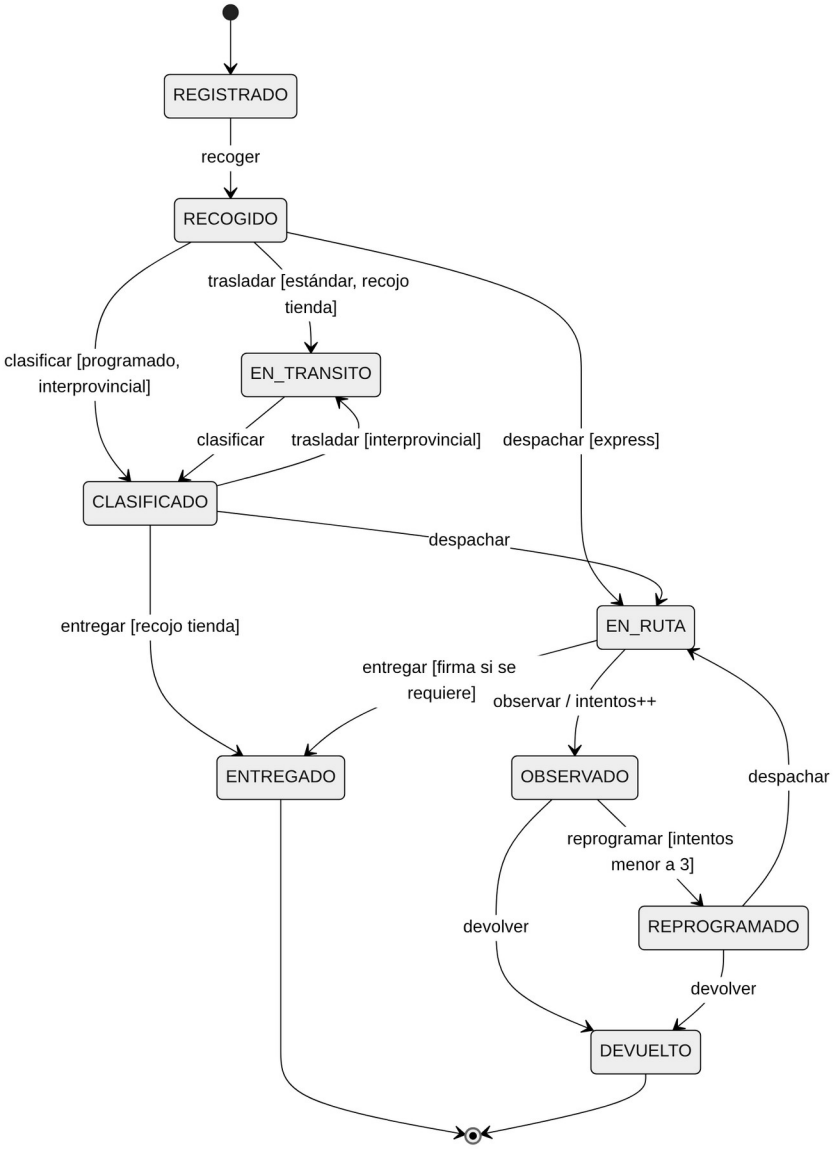
Diagrama UML

Figura 27
Diagrama de clases del patrón State del ciclo de vida del envío



Nota. Elaboración propia del grupo de trabajo.

Figura 28
Máquina de estados implementada (acciones y condiciones de guarda)



Nota. Elaboración propia del grupo de trabajo.

Implementación

Código 55
uni.service.IEstadoEnvio – State

```
package uni.service;

/**
 * PATRÓN STATE - State.
 * Cada estado del ciclo de vida de un envío decide qué acciones acepta y a
 * qué estado conduce cada una. El contexto (EnvioContexto) delega en el
 * estado actual en lugar de usar un switch gigante sobre el enum.
 */
public interface IEstadoEnvio {

    EstadoEnvio getCodigo();

    void recoger(EnvioContexto c);

    void trasladar(EnvioContexto c);

    void clasificar(EnvioContexto c);
```

```
    void despachar(EnvioContexto c);

    void entregar(EnvioContexto c);

    void observar(EnvioContexto c);

    void reprogramar(EnvioContexto c, LocalDate nuevaFecha);

    void devolver(EnvioContexto c);

    /** Acciones válidas en este estado (habilitan los botones de la vista). */
    List<AccionEnvio> accionesPermitidas(EnvioContexto c);

    boolean esFinal();
}
```

Código 56
uni.service.EstadoEnvioBase – comportamiento por defecto (fragmento)

```
public abstract class EstadoEnvioBase implements IEstadoEnvio {

    protected void prohibir(AccionEnvio accion) {
        throw new TransicionInvalidaException(getCodigo(), accion);
    }

    @Override
    public void recoger(EnvioContexto c) { prohibir(AccionEnvio.RECOGER); }    // ... igual
para clasificar, despachar, entregar, observar, reprogramar y devolver

    /** Agrega a la lista las acciones de avance que el flujo del servicio permite. */
    protected void agregarSiFlujoPermite(EnvioContexto c, List<AccionEnvio> lista,
AccionEnvio... acciones) {
        for (AccionEnvio a : acciones) {
            if (c.flujoPermite(a.getEstadoDestino())) {
                lista.add(a);
            }
        }
    }
}
```

Código 57
uni.service.EstadoRecogido – el flujo del servicio decide el siguiente paso

```
/**
 * Recogido: según el servicio pasa a tránsito (estándar, recojo en tienda),
 * a clasificación (programado, interprovincial) o directo a ruta (express).
 * La decisión la toma el flujo definido por cada IServicioEnvio (Factory).
 */
public class EstadoRecogido extends EstadoEnvioBase {

    @Override
    public EstadoEnvio getCodigo() { return EstadoEnvio.RECOGIDO; }

    @Override
    public void trasladar(EnvioContexto c) { c.avanzar(EstadoEnvio.EN_TRANSITO); }

    @Override
    public void clasificar(EnvioContexto c) { c.avanzar(EstadoEnvio.CLASIFICADO); }

    @Override
    public void despachar(EnvioContexto c) { c.avanzar(EstadoEnvio.EN_RUTA); }

    @Override
    public List<AccionEnvio> accionesPermitidas(EnvioContexto c) {
        List<AccionEnvio> l = super.accionesPermitidas(c);
        agregarSiFlujoPermite(c, l, AccionEnvio.TRASLADAR, AccionEnvio.CLASIFICAR,
AccionEnvio.DESPACHAR);
        return l;
    }
}
```

Código 58

uni.service.EstadoEnRuta y uni.service.EstadoObservado – reglas de firma e intentos

```
public class EstadoEnRuta extends EstadoEnvioBase {

    @Override
    public EstadoEnvio getCodigo() { return EstadoEnvio.EN_RUTA; }

    @Override
    public void entregar(EnvioContexto c) {
        c.validarFirma();                       // regla: si requiere firma, debe existir
        c.avanzar(EstadoEnvio.ENTREGADO);
    }

    @Override
    public void observar(EnvioContexto c) {
        c.registrarIntentoFallido();            // intentos++ y pasa a OBSERVADO
    }

    @Override
    public List<AccionEnvio> accionesPermitidas(EnvioContexto c) {
        return Arrays.asList(AccionEnvio.ENTREGAR, AccionEnvio.OBSERVAR);
    }
}

public class EstadoObservado extends EstadoEnvioBase {

    public static final int MAX_INTENTOS = 3;

    @Override
    public EstadoEnvio getCodigo() { return EstadoEnvio.OBSERVADO; }

    @Override
    public void reprogramar(EnvioContexto c, LocalDate nuevaFecha) {
        if (c.getEnvio().getIntentos() >= MAX_INTENTOS) {
            throw new TransicionInvalidaException("El envío ya tuvo " + MAX_INTENTOS
                    + " intentos fallidos: debe devolverse al remitente.");
        }
        if (nuevaFecha == null || nuevaFecha.isBefore(LocalDate.now())) {
            throw new TransicionInvalidaException("La nueva fecha de entrega debe ser hoy o
posterior.");
        }
        c.getEnvio().setFechaProgramada(nuevaFecha);
        c.cambiarEstado(EstadoEnvio.REPROGRAMADO);
    }

    @Override
    public void devolver(EnvioContexto c) {
        c.cambiarEstado(EstadoEnvio.DEVUELTO);
    }

    @Override
    public List<AccionEnvio> accionesPermitidas(EnvioContexto c) {
        List<AccionEnvio> l = super.accionesPermitidas(c);
        if (c.getEnvio().getIntentos() < MAX_INTENTOS) {
            l.add(AccionEnvio.REPROGRAMAR);
        }
        l.add(AccionEnvio.DEVOLVER);
        return l;
    }
}
```

Código 59

uni.service.EnvioContexto – Context: delega en el estado actual (fragmento)

```
    // ============================================================ STATE
    /** Punto de entrada usado por el servicio de seguimiento. */
    public void ejecutar(AccionEnvio accion, String idempleado, String ubicacion,
                         String observacion, LocalDate fecha) {
        this.idempleado = idempleado;
        this.ubicacion = ubicacion;
        this.observacion = observacion;
        switch (accion) {
```

```
            case RECOGER:     estado.recoger(this); break;
            case TRASLADAR:   estado.trasladar(this); break;
            case CLASIFICAR:  estado.clasificar(this); break;
            case DESPACHAR:   estado.despachar(this); break;
            case ENTREGAR:    estado.entregar(this); break;
            case OBSERVAR:    estado.observar(this); break;
            case REPROGRAMAR: estado.reprogramar(this, fecha); break;
            case DEVOLVER:    estado.devolver(this); break;
            default: throw new IllegalArgumentException("Acción no soportada: " + accion);
        }
    }

    public List<AccionEnvio> accionesPermitidas() {
        return estado.accionesPermitidas(this);
    }

    public IEstadoEnvio getEstado() {
        return estado;
    }

    /** ¿El siguiente paso del flujo del servicio es 'destino'? */
    boolean flujoPermite(EstadoEnvio destino) {
        List<EstadoEnvio> flujo = servicio.getFlujoEstados();
        int siguiente = envio.getPasoFlujo() + 1;
        return siguiente < flujo.size() && flujo.get(siguiente) == destino;
    }

    boolean flujoContiene(EstadoEnvio e) {
        return servicio.getFlujoEstados().contains(e);
    }

    /** Avance normal dentro del flujo del servicio. */
    void avanzar(EstadoEnvio destino) {
        if (!flujoPermite(destino)) {
            throw new TransicionInvalidaException("El servicio " + servicio.getTipo()
                    + " no pasa de " + envio.getEstado() + " a " + destino
                    + ". Flujo: " + servicio.getFlujoEstados());
        }
        envio.setPasoFlujo(envio.getPasoFlujo() + 1);
        cambiarEstado(destino);
    }
    // ...
```

Con este diseño, `SeguimientoServiceImpl.actualizarEstado()` —el método que en el Capítulo V asignaba el estado directamente— quedó reducido a cargar el contexto y delegar: `ctx.ejecutar(AccionEnvio.hacia(nuevo), usuario, ubicacion, observacion, null)`. El Proxy de seguridad del Capítulo V no se modificó: sigue controlando *quién* puede actuar, mientras que State controla *qué* acción es válida.

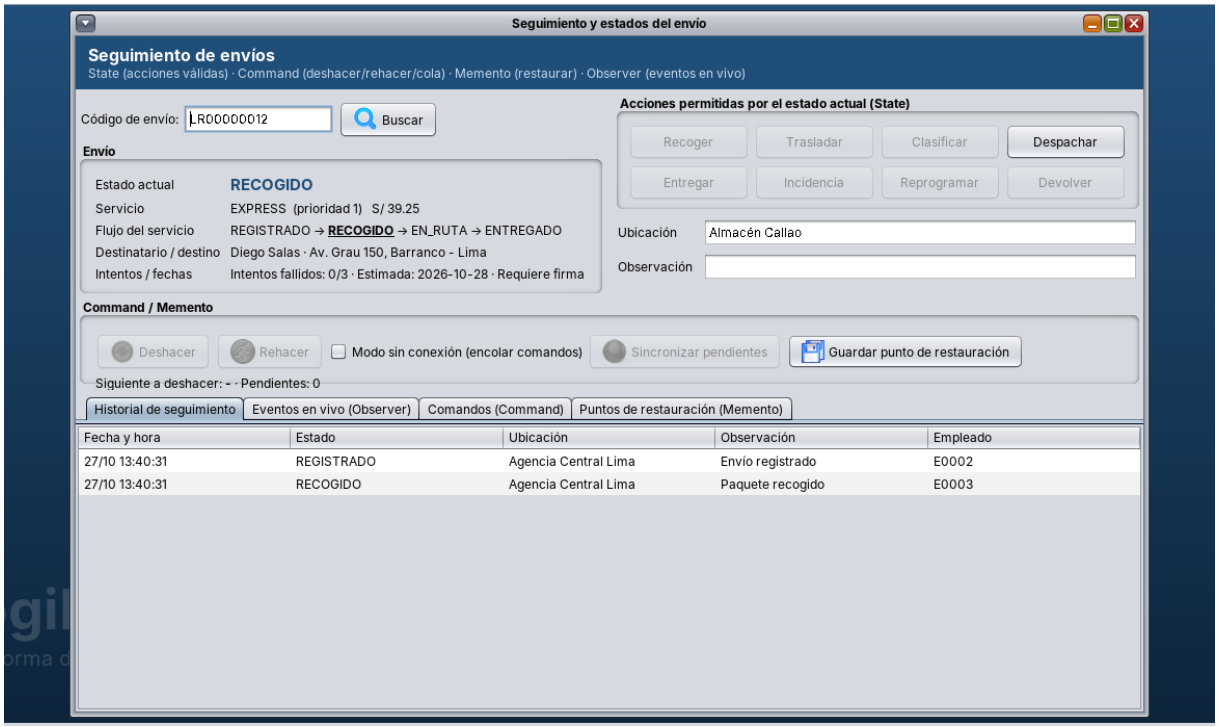
Pruebas

Prueba por consola: recorrido completo de un envío estándar con firma, diferencia entre servicios y tope de tres intentos (se omiten mensajes repetidos de WhatsApp).

```
Recorrido de un envío ESTÁNDAR con firma digital (LR00000001):
   REGISTRADO   permite [RECOGER]                                    -> RECOGER
   [WA-API] +51987111222 template=estado_envio body="LogiRuta LR00000001 - Recogimos su
paquete."
   RECOGIDO     permite [TRASLADAR]                                  -> TRASLADAR
   EN_TRANSITO  permite [CLASIFICAR]                                 -> CLASIFICAR
   CLASIFICADO  permite [DESPACHAR]                                  -> DESPACHAR
   [WA-API] +51987111222 template=estado_envio body="LogiRuta LR00000001 - Su paquete está
en ruta y llegará hoy."
   EN_RUTA sin firma -> ENTREGAR: RECHAZADO -> El envío LR00000001 requiere firma digital
del receptor antes de marcarse ENTREGADO.
   Con firma -> ENTREGADO (paso 5, intentos 0) firma=CLOUD-63845003
   Acciones en ENTREGADO: []
Mismo estado RECOGIDO, distinto servicio:
   EXPRESS  permite [DESPACHAR]
   ESTANDAR permite [TRASLADAR]
   ESTANDAR RECOGIDO -> DESPACHAR: RECHAZADO -> El servicio ESTANDAR no pasa de RECOGIDO a
EN_RUTA. Flujo: [REGISTRADO, RECOGIDO, EN_TRANSITO, CLASIFICADO, EN_RUTA, ENTREGADO]
Tres intentos fallidos de entrega (LR00000002):
   Incidencia 1 -> OBSERVADO (paso 2, intentos 1) permite [REPROGRAMAR, DEVOLVER]
   Incidencia 2 -> OBSERVADO (paso 2, intentos 2) permite [REPROGRAMAR, DEVOLVER]
   Incidencia 3 -> OBSERVADO (paso 2, intentos 3) permite [DEVOLVER]
   Reprogramar por 4.ª vez: RECHAZADO -> El envío ya tuvo 3 intentos fallidos: debe
devolverse al remitente.
   Final -> DEVUELTO (paso 2, intentos 3)
```

Figura 29

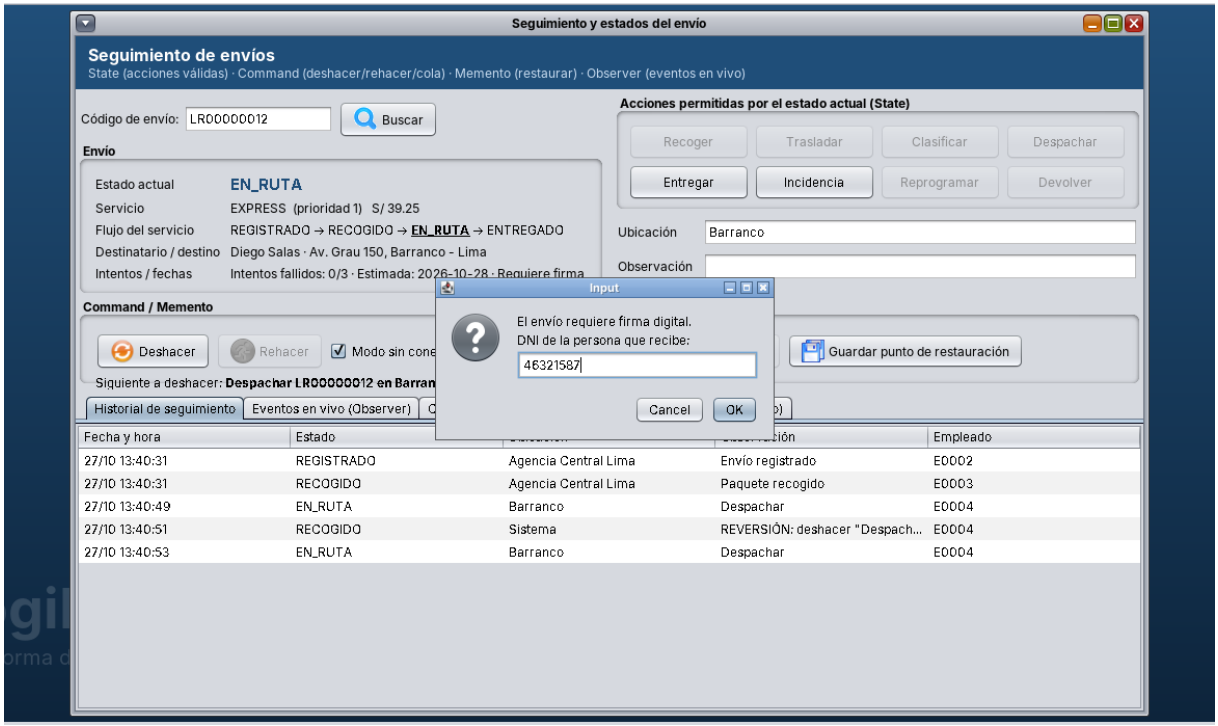
SeguimientoView: en RECOGIDO un envío EXPRESS solo habilita "Despachar" (botones calculados por el estado)



Nota. Captura de AppLogistica. Los botones se habilitan con accionesPermitidas() del estado actual; el flujo del servicio resalta la etapa actual.

Figura 30

Regla del estado EN_RUTA: el envío requiere firma digital, por lo que se solicita el DNI del receptor



Nota. Captura de AppLogistica. El código de firma lo genera IFirmaDigital (Abstract Factory, Capítulo IV).

Tabla 26

Casos de prueba del patrón State

Caso	Entrada	Resultado esperado	Resultado obtenido
CP-STA-01	ESTÁNDAR: recoger, trasladar, clasificar, despachar	Cada estado habilita solo la acción siguiente	Conforme
CP-STA-02	Entregar sin firma un envío que la requiere	Rechazo: "requiere firma digital"	Conforme
CP-STA-03	ESTÁNDAR en RECOGIDO → despachar	Rechazo: el flujo estándar no pasa de RECOGIDO a EN_RUTA	Conforme
CP-STA-04	EXPRESS en RECOGIDO	Acciones permitidas: [DESPACHAR]	Conforme
CP-STA-05	Tercera incidencia y reprogramar	Rechazo: debe devolverse; solo DEVOLVER habilitado	Conforme
CP-STA-06	Acciones en ENTREGADO / DEVUELTO	Lista vacía (estados finales)	Conforme

Nota. Elaboración propia del grupo de trabajo.

6.3 Patrón Command

Escenario de aplicación

Tabla 27
Ficha del patrón Command (formato de la sesión de clase)

Elemento	Descripción
Definición	Encapsula una petición como un objeto, lo que permite parametrizar clientes con distintas peticiones, encolarlas, registrarlas y deshacerlas.
Intención	Desacoplar al objeto que invoca una operación (botón, menú) del que sabe ejecutarla.
Ventajas	Deshacer/rehacer; colas y ejecución diferida (modo sin conexión); macro-comandos; bitácora de acciones; nuevos comandos sin tocar el invocador.
Desventajas	Una clase por acción; el deshacer exige guardar información adicional (por eso se combina con Memento).

Nota. Adaptado de Gamma et al. (1994) y del material de clase del curso.

¿Qué problema tengo? El personal operativo realiza cientos de acciones por turno (recoger, clasificar, despachar, registrar incidencias, entregar). Tres necesidades del negocio no podían atenderse llamando directamente al servicio desde el botón del formulario: (1) **deshacer** una acción registrada por error —por ejemplo, clasificar el paquete equivocado al escanear— sin borrar el historial; (2) **trabajar sin conexión**: el repartidor pierde señal en algunas zonas y sus acciones deben guardarse y ejecutarse en orden cuando recupere la conexión; y (3) **despachar un manifiesto completo** como una sola operación de "todo o nada" y dejar **bitácora** de quién hizo cada acción (RF20).

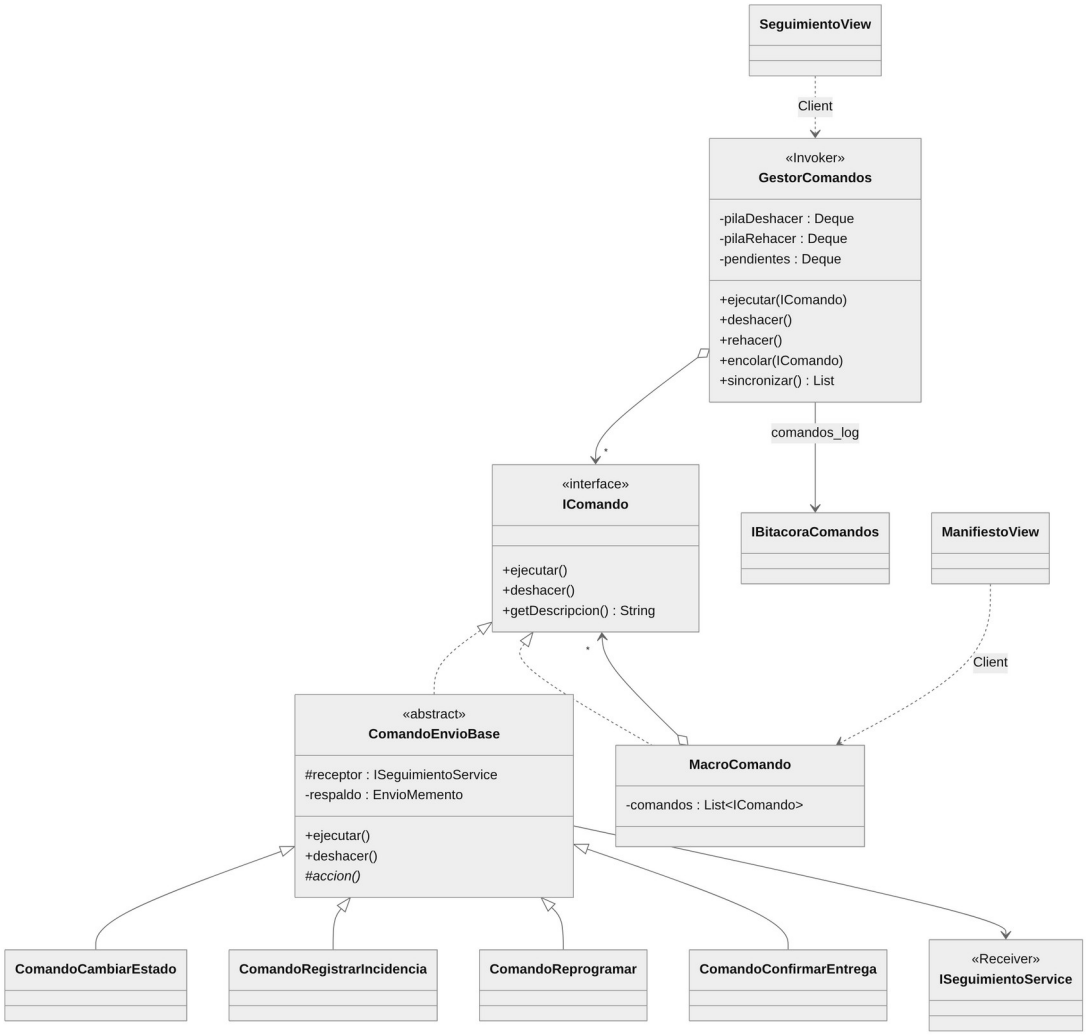
¿Por qué el patrón Command lo resuelve? Command encapsula una petición como un objeto, lo que permite parametrizar clientes con distintas peticiones, encolarlas, registrarlas y soportar operaciones que se pueden deshacer. Cada botón crea un comando (`ComandoCambiarEstado`, `ComandoRegistrarIncidencia`, `ComandoReprogramar`, `ComandoConfirmarEntrega`) y se lo entrega al **Invoker** `GestorComandos`, que mantiene la pila de deshacer, la pila de rehacer, la cola de pendientes y la bitácora. El **Receiver** es `ISeguimientoService`, protegido por el Proxy, de modo que un comando nunca evita los controles de seguridad. Para el manifiesto se usa un **MacroComando**, que agrupa un comando por paquete del árbol Composite y, si uno falla, deshace los ya ejecutados.

- **Command:** `IComando` (`ejecutar`, `deshacer`, `getDescripcion`).
- **ConcreteCommands:** heredan de `ComandoEnvioBase`, que implementa el *deshacer* con un Memento (sección 6.4).
- **Invoker:** `GestorComandos` (paquete `uni.controller`).
- **Receiver:** `ISeguimientoService` → `SeguimientoServiceProxy` → `SeguimientoServiceImpl`.
- **Client:** `SeguimientoView` y `ManifiestoView`.

Como el historial de seguimiento es inmutable (trigger `tg_seguimientos_no_update` del Avance 1), deshacer **no borra** registros: agrega un registro de *REVERSIÓN* y marca el comando como deshecho en la tabla `comandos_log`.

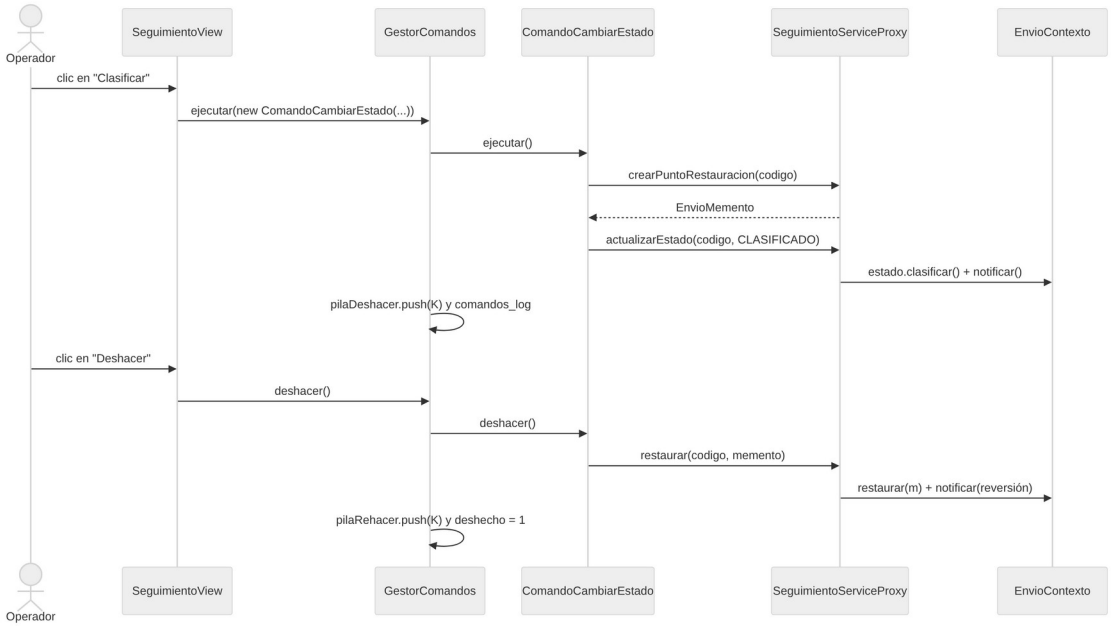
Diagrama UML

Figura 31
Diagrama de clases del patrón Command



Nota. Elaboración propia del grupo de trabajo.

Figura 32
Diagrama de secuencia: ejecutar y deshacer un comando



Nota. Elaboración propia del grupo de trabajo.

Implementación

Código 60
uni.service.IComando – Command

```
/**
 * PATRÓN COMMAND - Command.
 * Encapsula una acción del personal operativo (recoger, despachar,
 * registrar incidencia, entregar...) como un objeto que se puede
 * ejecutar, deshacer, encolar (modo sin conexión) y registrar en bitácora.
 */
public interface IComando {

    void ejecutar() throws Exception;

    void deshacer() throws Exception;

    String getNombre();

    String getDescripcion();

    String getCodigoEnvio();

    /** Estado antes y después de ejecutar (para la bitácora). */
    String getEstadoPrevio();

    String getEstadoNuevo();
}
```

Código 61
uni.service.ComandoEnvioBase – deshacer apoyado en Memento

```
/**
 * PATRÓN COMMAND - comando abstracto con deshacer basado en MEMENTO.
 * Antes de ejecutar pide al receptor (ISeguimientoService) un punto de
 * restauración; deshacer() le entrega ese memento para volver atrás.
 * El comando actúa como Caretaker: guarda el memento sin leer su contenido.
 */
public abstract class ComandoEnvioBase implements IComando {

    protected final ISeguimientoService receptor;
    protected final String codigoEnvio;
```

```
    private EnvioMemento respaldo;
    private String estadoPrevio;
    private String estadoNuevo;

    protected ComandoEnvioBase(ISeguimientoService receptor, String codigoEnvio) {
        this.receptor = receptor;
        this.codigoEnvio = codigoEnvio;
    }

    /** Operación concreta sobre el receptor. */
    protected abstract void accion() throws Exception;

    @Override
    public final void ejecutar() throws Exception {
        respaldo = receptor.crearPuntoRestauracion(codigoEnvio, getNombre());
        estadoPrevio = estadoActual();
        accion();
        estadoNuevo = estadoActual();
    }

    @Override
    public void deshacer() throws Exception {
        if (respaldo == null) {
            throw new IllegalStateException("El comando aún no se ha ejecutado.");
        }
        receptor.restaurar(codigoEnvio, respaldo, "deshacer \"" + getDescripcion() + "\"");
    }
    // ... getters
}
```

Código 62
uni.service.ComandoCambiarEstado – comando concreto

```
public class ComandoCambiarEstado extends ComandoEnvioBase {

    private final AccionEnvio accion;
    private final String ubicacion;
    private final String observacion;

    public ComandoCambiarEstado(ISeguimientoService receptor, String codigoEnvio,
                                AccionEnvio accion, String ubicacion, String observacion) {
        super(receptor, codigoEnvio);
        this.accion = accion;
        this.ubicacion = ubicacion;
        this.observacion = observacion;
    }

    @Override
    protected void accion() throws Exception {
        receptor.actualizarEstado(codigoEnvio, accion.getEstadoDestino(), ubicacion,
observacion);
    }

    @Override
    public String getNombre() {
        return accion.name();
    }

    @Override
    public String getDescripcion() {
        return accion.getEtiqueta() + " " + codigoEnvio + (ubicacion == null ? "" : " en " +
ubicacion);
    }
}
```

Código 63
uni.service.MacroComando – despacho de manifiesto "todo o nada" (fragmento)

```
public class MacroComando implements IComando {

    private final String nombre;
    private final String codigoReferencia;
```

```
    private final List<IComando> comandos = new ArrayList<>();
    private final List<IComando> ejecutados = new ArrayList<>();

    public MacroComando(String nombre, String codigoReferencia) {
        this.nombre = nombre;
        this.codigoReferencia = codigoReferencia;
    }

    public MacroComando agregar(IComando c) {
        comandos.add(c);
        return this;
    }

    @Override
    public void ejecutar() throws Exception {
        ejecutados.clear();
        for (IComando c : comandos) {
            try {
                c.ejecutar();
                ejecutados.add(c);
            } catch (Exception ex) {
                deshacer();                          // compensación de lo ya hecho
                throw new Exception("Macro \"" + nombre + "\" cancelada en "
                        + c.getCodigoEnvio() + ": " + ex.getMessage(), ex);
            }
        }
    }

    @Override
    public void deshacer() throws Exception {
        for (int i = ejecutados.size() - 1; i >= 0; i--) {
            ejecutados.get(i).deshacer();
        }
        ejecutados.clear();
    }
    // ...
}
```

Código 64
uni.controller.GestorComandos – Invoker (fragmento)

```
public class GestorComandos {

    private final Deque<IComando> pilaDeshacer = new ArrayDeque<>();
    private final Deque<IComando> pilaRehacer = new ArrayDeque<>();
    private final Deque<IComando> pendientes = new ArrayDeque<>();
    private final Map<IComando, Long> idsBitacora = new IdentityHashMap<>();
    private final IBitacoraComandos bitacora;     // null = sin persistencia (pruebas)

    public GestorComandos(IBitacoraComandos bitacora) {
        this.bitacora = bitacora;
    }

    public void ejecutar(IComando c) throws Exception {
        c.ejecutar();
        pilaDeshacer.push(c);
        pilaRehacer.clear();
        registrar(c);
    }

    public IComando deshacer() throws Exception {
        if (pilaDeshacer.isEmpty()) {
            throw new IllegalStateException("No hay acciones para deshacer.");
        }
        IComando c = pilaDeshacer.peek();
        c.deshacer();                    // si falla (p. ej. Proxy lo niega) queda en la
pila
        pilaDeshacer.pop();
        pilaRehacer.push(c);
        marcar(c, true);
        return c;
    }
```

```
    public IComando rehacer() throws Exception {
        if (pilaRehacer.isEmpty()) {
            throw new IllegalStateException("No hay acciones para rehacer.");
        }
        IComando c = pilaRehacer.peek();
        c.ejecutar();
        pilaRehacer.pop();
        pilaDeshacer.push(c);
        registrar(c);
        return c;
    }

    /** Modo sin conexión: el comando se guarda y se ejecuta al sincronizar. */
    public void encolar(IComando c) {
        pendientes.addLast(c);
    }

    /** Ejecuta en orden los comandos pendientes; devuelve el resultado de cada uno. */
    public List<String> sincronizar() {
        List<String> resultado = new ArrayList<>();
        while (!pendientes.isEmpty()) {
            IComando c = pendientes.pollFirst();
            try {
                ejecutar(c);
                resultado.add("OK    " + c.getDescripcion());
            } catch (Exception ex) {
                resultado.add("ERROR " + c.getDescripcion() + " -> " + ex.getMessage());
            }
        }
        return resultado;
    }
    // ... bitácora en comandos_log
}
```

Código 65

uni.formularios.SeguimientoView – el Client crea el comando y lo entrega al Invoker (fragmento)

```
            default:
                cmd = new ComandoCambiarEstado(servicio, cod, a, ubic, obs);
        }
        if (chkSinConexion.isSelected()) {
            gestor.encolar(cmd);
            txtEventos.append("[COLA] Sin conexión: se encoló \"" + cmd.getDescripcion() +
"\"\n");
            actualizarComandos();
            return;
        }
        try {
            gestor.ejecutar(cmd);
        } catch (Exception ex) {
            error(ex);
        }
        txtObservacion.setText("");
        cargar();
    }
```

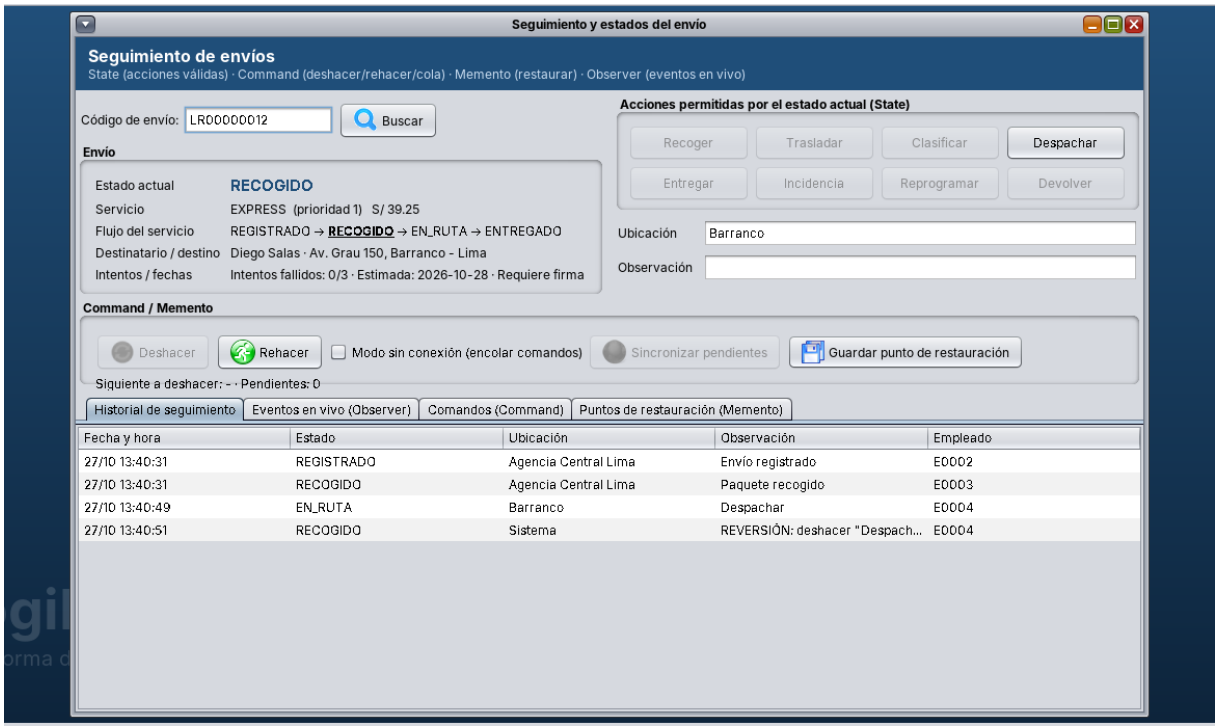
Pruebas

Prueba por consola: deshacer/rehacer, cola sin conexión y macro-comando con un envío inválido (se omiten mensajes repetidos de WhatsApp).

```
   [WA-API] +51945678123 template=estado_envio body="LogiRuta LR00000001 - Recogimos su
paquete."
Ejecutados 3 comandos -> CLASIFICADO (paso 3, intentos 0)
Pila de deshacer: [Clasificar LR00000001 en Almacén Callao, Trasladar LR00000001 en Almacén
Callao, Recoger LR00000001 en Almacén Callao]
   [WA-API] +51945678123 template=estado_envio body="LogiRuta LR00000001 - Corregimos el
estado de su envío: EN_TRANSITO"
Deshacer: Clasificar LR00000001 en Almacén Callao -> EN_TRANSITO (paso 2, intentos 0)
Deshacer: Trasladar LR00000001 en Almacén Callao -> RECOGIDO (paso 1, intentos 0)
Rehacer:  Trasladar LR00000001 en Almacén Callao -> EN_TRANSITO (paso 2, intentos 0)
Historial (inmutable, incluye reversiones):
   RECOGIDO     | Almacén Callao         | Recojo
   EN_TRANSITO  | Almacén Callao         | Traslado
   CLASIFICADO  | Almacén Callao         | Zona Lima Norte
   EN_TRANSITO  | Sistema                | REVERSIÓN: deshacer "Clasificar LR00000001 en
Almacén Callao"
   RECOGIDO     | Sistema                | REVERSIÓN: deshacer "Trasladar LR00000001 en
Almacén Callao"
   EN_TRANSITO  | Almacén Callao         | Traslado
Modo sin conexión (repartidor sin señal): se encolan 2 comandos
   Pendientes: 2 | estado aún EN_TRANSITO (paso 2, intentos 0)
   OK    Clasificar LR00000001 en Almacén
   OK    Despachar LR00000001 en Comas
   Tras sincronizar -> EN_RUTA (paso 4, intentos 0)
Macro-comando: despachar manifiesto de 3 envíos (uno inválido):
   ejecutar macro: RECHAZADO -> Macro "Despachar MF-2026-0460" cancelada en LR00000004:
Acción "Despachar" no permitida: el envío está REGISTRADO.
   LR00000002 -> CLASIFICADO (paso 3, intentos 0) | LR00000003 -> CLASIFICADO (paso 3,
intentos 0) | LR00000004 -> REGISTRADO (paso 0, intentos 0)
   Macro válida -> EN_RUTA (paso 4, intentos 0) | EN_RUTA (paso 4, intentos 0)
   Deshacer macro -> CLASIFICADO (paso 3, intentos 0) | CLASIFICADO (paso 3, intentos 0)
```

Figura 33

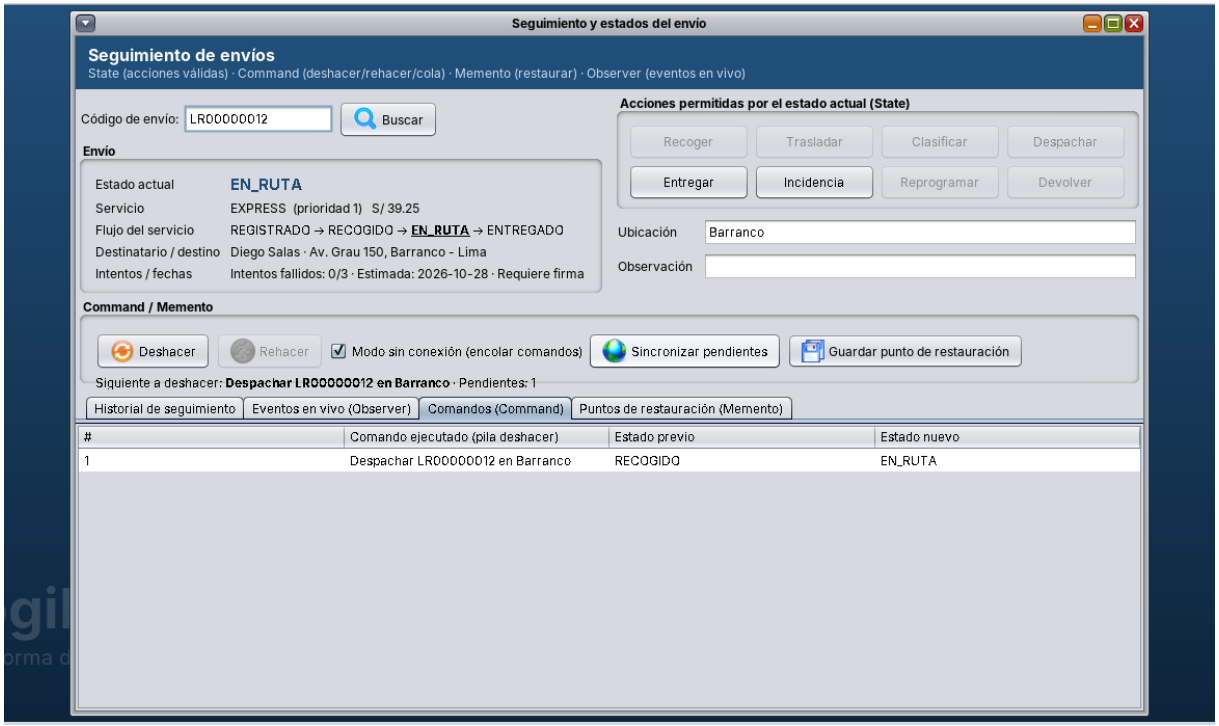
Historial tras "Deshacer": se agrega un registro de REVERSIÓN y el envío vuelve a RECOGIDO



Nota. Captura de AppLogistica. El historial nunca se borra: la reversión es un nuevo registro.

Figura 34

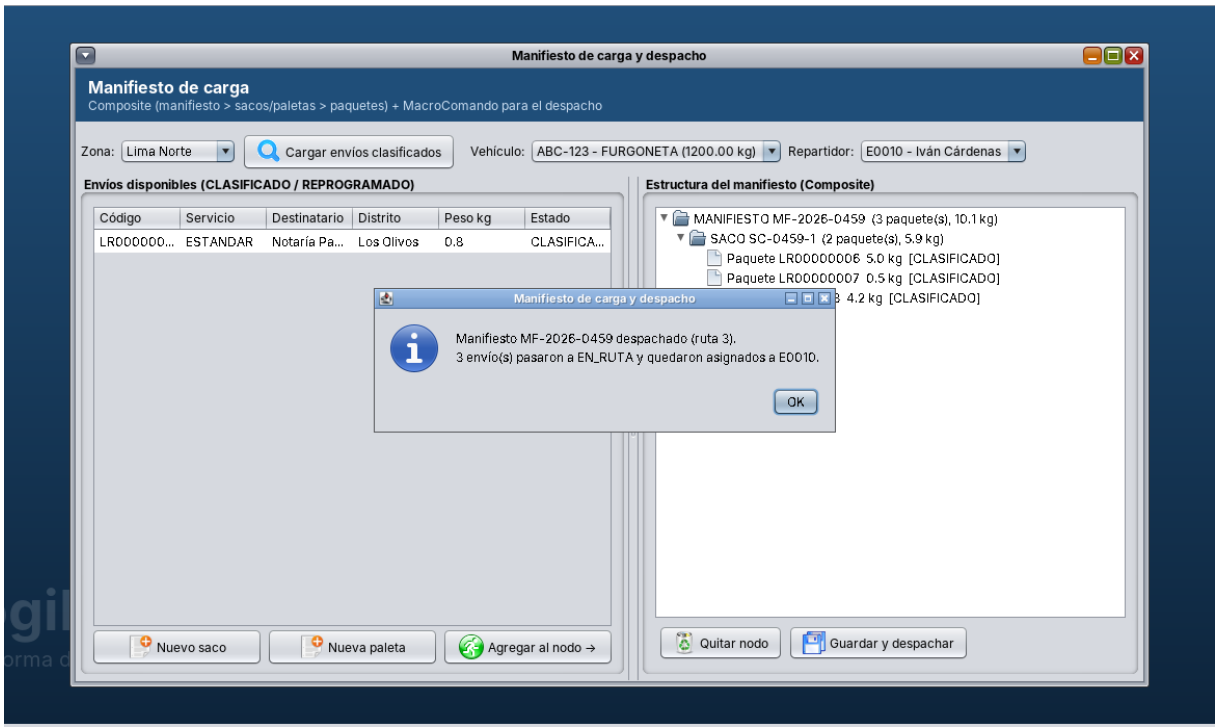
Modo sin conexión: la entrega queda encolada (Pendientes: 1) hasta pulsar "Sincronizar pendientes"



Nota. Captura de AppLogistica. La pestaña Comandos muestra la pila de deshacer del GestorComandos.

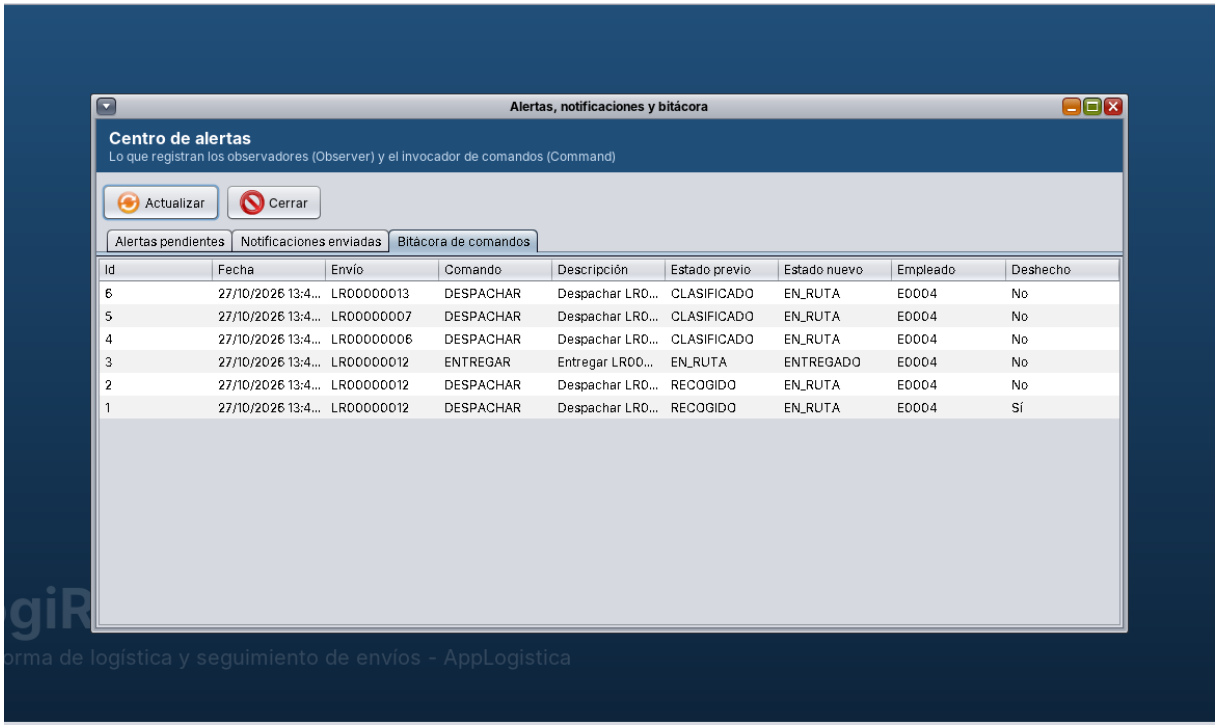
Figura 35

ManifiestoView: el árbol Composite se despacha con un MacroComando (3 envíos → EN_RUTA)



Nota. Captura de AppLogistica. Peso y cantidad de paquetes los calcula recursivamente el Composite.

Figura 36
Bitácora de comandos (tabla comandos_log): estado previo, estado nuevo, empleado y marca de deshecho



Nota. Captura de AppLogistica.

Tabla 28
Casos de prueba del patrón Command

Caso	Entrada	Resultado esperado	Resultado obtenido
CP-CMD-01	3 comandos y 2 "deshacer"	CLASIFICADO → EN_TRANSITO → RECOGIDO con registros de reversión	Conforme
CP-CMD-02	"Rehacer" tras deshacer	Vuelve a EN_TRANSITO	Conforme
CP-CMD-03	Encolar 2 comandos sin conexión y sincronizar	No cambia el estado hasta sincronizar; luego OK, OK	Conforme
CP-CMD-04	Macro con un envío REGISTRADO	Se cancela todo y los otros dos vuelven a CLASIFICADO	Conforme
CP-CMD-05	Deshacer macro válida	Ambos envíos regresan a CLASIFICADO	Conforme
CP-CMD-06	Ejecutar y deshacer en la app	comandos_log registra la acción y deshecho = 1	Conforme

Nota. Elaboración propia del grupo de trabajo.

6.4 Patrón Memento

Escenario de aplicación

Tabla 29
Ficha del patrón Memento (formato de la sesión de clase)

Elemento	Descripción
Definición	Captura y externaliza el estado interno de un objeto, sin violar su encapsulamiento, para poder restaurarlo más tarde.
Intención	Permitir volver a un estado anterior sin exponer los detalles internos del objeto.
Ventajas	Conserva el encapsulamiento; simplifica el Originator; combina de forma natural con Command para deshacer.
Desventajas	Consumo de memoria si se guardan muchas instantáneas (se limita a 10 por envío); el Caretaker no sabe cuánto ocupa cada memento.

Nota. Adaptado de Gamma et al. (1994) y del material de clase del curso.

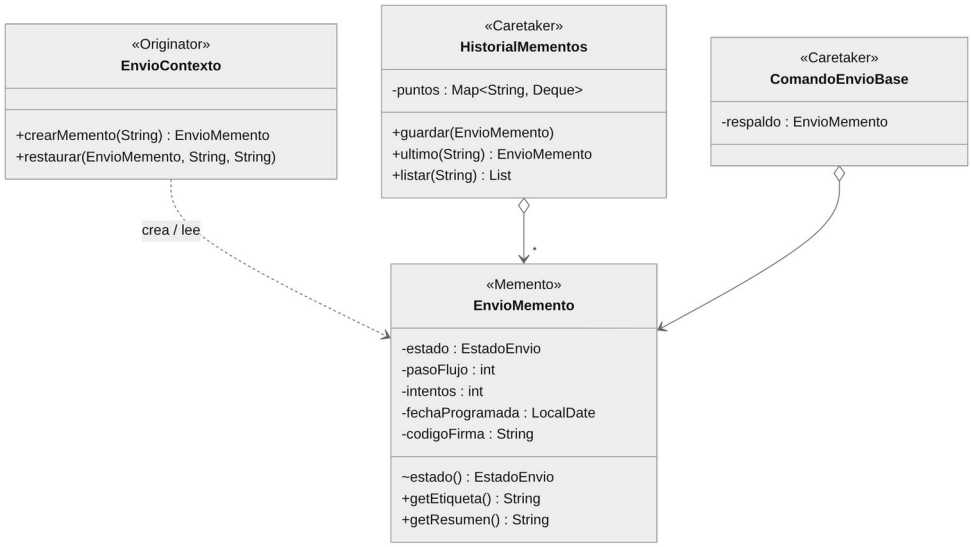
¿Qué problema tengo? Hay situaciones en las que se debe **volver exactamente** a un estado anterior del envío: el cliente solicita reprogramar la entrega y luego la cancela; un supervisor revierte una entrega confirmada por error; o un comando debe deshacerse. El estado relevante no es solo la columna `estado`: incluye el paso dentro del flujo del servicio, el número de intentos, la fecha programada y el código de firma. Si cada formulario o comando copiara y luego reasignara estos campos con *setters*, se rompería el encapsulamiento del envío y cualquier campo nuevo obligaría a modificar todos esos lugares.

¿Por qué el patrón Memento lo resuelve? Memento captura y externaliza el estado interno de un objeto **sin violar su encapsulamiento**, para poder restaurarlo después. El **Originator** (`EnvioContexto`) es el único que crea y lee el `EnvioMemento`; los **Caretakers** (`HistorialMementos` y los comandos) solo lo guardan. En Java esto se logra con dos interfaces: una *estrecha* pública (`getEtiqueta()`, `getResumen()`) y una *amplia* con visibilidad de paquete (`estado()`, `intentos()`...), accesible únicamente desde `uni.service`. Además, el Proxy del Capítulo V agrega una regla: solo un supervisor puede restaurar un envío que ya figura como *ENTREGADO*.

- **Originator:** `EnvioContexto` → `crearMemento(etiqueta)` y `restaurar(memento, ...)`.
- **Memento:** `EnvioMemento`, clase `final` e inmutable.
- **Caretaker:** `HistorialMementos` (puntos de restauración manuales, hasta 10 por envío) y `ComandoEnvioBase` (respaldo para deshacer).

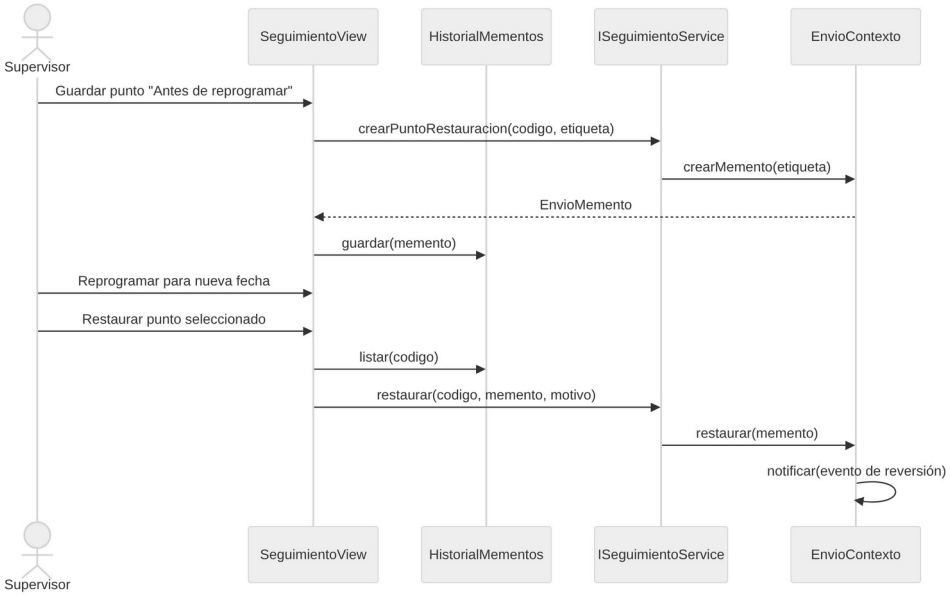
Diagrama UML

Figura 37
Diagrama de clases del patrón Memento



Nota. Elaboración propia del grupo de trabajo.

Figura 38
Diagrama de secuencia: cancelar una reprogramación restaurando un Memento



Nota. Elaboración propia del grupo de trabajo.

Implementación

Código 66
uni.service.EnvioMemento – Memento con interfaz amplia y estrecha

```
/**
 * PATRÓN MEMENTO - Memento.
 * Instantánea inmutable del estado interno de un envío. Los datos solo
 * pueden leerse desde el paquete uni.service (el Originator EnvioContexto);
 * el Caretaker (comandos, HistorialMementos) solo ve la etiqueta y la hora.
 */
public final class EnvioMemento {

    private final String etiqueta;
    private final String codigoEnvio;
```

```
    private final EstadoEnvio estado;
    private final int pasoFlujo;
    private final int intentos;
    private final LocalDate fechaProgramada;
    private final String codigoFirma;
    private final LocalDateTime creado = LocalDateTime.now();

    EnvioMemento(String etiqueta, String codigoEnvio, EstadoEnvio estado, int pasoFlujo,
                 int intentos, LocalDate fechaProgramada, String codigoFirma) {
        this.etiqueta = etiqueta;
        this.codigoEnvio = codigoEnvio;
        this.estado = estado;
        this.pasoFlujo = pasoFlujo;
        this.intentos = intentos;
        this.fechaProgramada = fechaProgramada;
        this.codigoFirma = codigoFirma;
    }

    // --- Interfaz "amplia": solo para el Originator (visibilidad de paquete)
    EstadoEnvio estado() { return estado; }
    int pasoFlujo() { return pasoFlujo; }
    int intentos() { return intentos; }
    LocalDate fechaProgramada() { return fechaProgramada; }
    String codigoFirma() { return codigoFirma; }

    // --- Interfaz "estrecha": pública, para el Caretaker
    public String getEtiqueta() { return etiqueta; }
    public String getCodigoEnvio() { return codigoEnvio; }
    public LocalDateTime getCreado() { return creado; }

    /** Texto para listas de la vista (no revela más que el estado guardado). */
    public String getResumen() {
        return creado.format(DateTimeFormatter.ofPattern("HH:mm:ss")) + " | " + etiqueta
                + " | volver a " + estado;
    }

    @Override
    public String toString() {
        return getResumen();
    }
}
```

Código 67

uni.service.EnvioContexto – Originator (fragmento)

```
    // ============================================================ MEMENTO
    /** Guarda una instantánea del estado interno (sin exponerlo). */
    public EnvioMemento crearMemento(String etiqueta) {
        return new EnvioMemento(etiqueta, envio.getCodigo(), envio.getEstado(),
envio.getPasoFlujo(),
                envio.getIntentos(), envio.getFechaProgramada(), envio.getCodigoFirma());
    }

    /** Vuelve exactamente a la instantánea y deja constancia en el historial. */
    public void restaurar(EnvioMemento m, String idempleado, String motivo) {
        if (!m.getCodigoEnvio().equals(envio.getCodigo())) {
            throw new IllegalArgumentException("El memento pertenece a otro envío.");
        }
        EstadoEnvio anterior = envio.getEstado();
        envio.setEstado(m.estado());
        envio.setPasoFlujo(m.pasoFlujo());
        envio.setIntentos(m.intentos());
        envio.setFechaProgramada(m.fechaProgramada());
        envio.setCodigoFirma(m.codigoFirma());
        estado = CatalogoEstados.de(m.estado());
        notificar(new EventoEnvio(envio, anterior, m.estado(), "Sistema",
                "REVERSIÓN: " + motivo, idempleado, true));
    }
```

Código 68
uni.util.HistorialMementos – Caretaker (fragmento)

```
/**
 * PATRÓN MEMENTO - Caretaker.
 * Guarda puntos de restauración por envío (por ejemplo, antes de una
 * reprogramación solicitada por el cliente). No puede leer ni modificar
 * el contenido de los mementos: solo los almacena y los devuelve.
 */
public class HistorialMementos {

    private static final int MAXIMO_POR_ENVIO = 10;
    private final Map<String, Deque<EnvioMemento>> puntos = new HashMap<>();

    public void guardar(EnvioMemento m) {
        Deque<EnvioMemento> pila = puntos.computeIfAbsent(m.getCodigoEnvio(), k -> new
ArrayDeque<>());
        pila.push(m);
        if (pila.size() > MAXIMO_POR_ENVIO) {
            pila.removeLast();                      // descarta el más antiguo
        }
    }

    public List<EnvioMemento> listar(String codigoEnvio) {
        return new ArrayList<>(puntos.getOrDefault(codigoEnvio, new ArrayDeque<>()));
    }

    public EnvioMemento ultimo(String codigoEnvio) {
        Deque<EnvioMemento> pila = puntos.get(codigoEnvio);
        return pila == null ? null : pila.peek();
    }
}
```

Código 69
uni.service.SeguimientoServiceProxy.restaurar() – el Proxy protege la restauración

```
    @Override
    public void restaurar(String codigoEnvio, EnvioMemento memento, String motivo) throws
Exception {
        EmpleadoTo u = SesionUsuario.getInstancia().getUsuario();
        EnvioTo actual = real.buscarEnvio(codigoEnvio);
        if (actual != null && actual.getEstado() == EstadoEnvio.ENTREGADO
                && !ROLES_SUPERVISION.contains(u.getRol())) {
            auditar("RESTAURAR(ENTREGADO)", codigoEnvio, false);
            throw new SecurityException("Solo un supervisor puede revertir una entrega
confirmada.");
        }
        verificarOperacion("RESTAURAR", codigoEnvio);
        real.restaurar(codigoEnvio, memento, motivo);
        cache.remove(codigoEnvio);
    }
```

Como `HistorialMementos` está en el paquete `uni.util`, cualquier intento de leer `memento.estado()` desde él produce un **error de compilación** ("estado() is not public in EnvioMemento; cannot be accessed from outside package"), lo que demuestra que el Caretaker no puede inspeccionar ni alterar la instantánea.

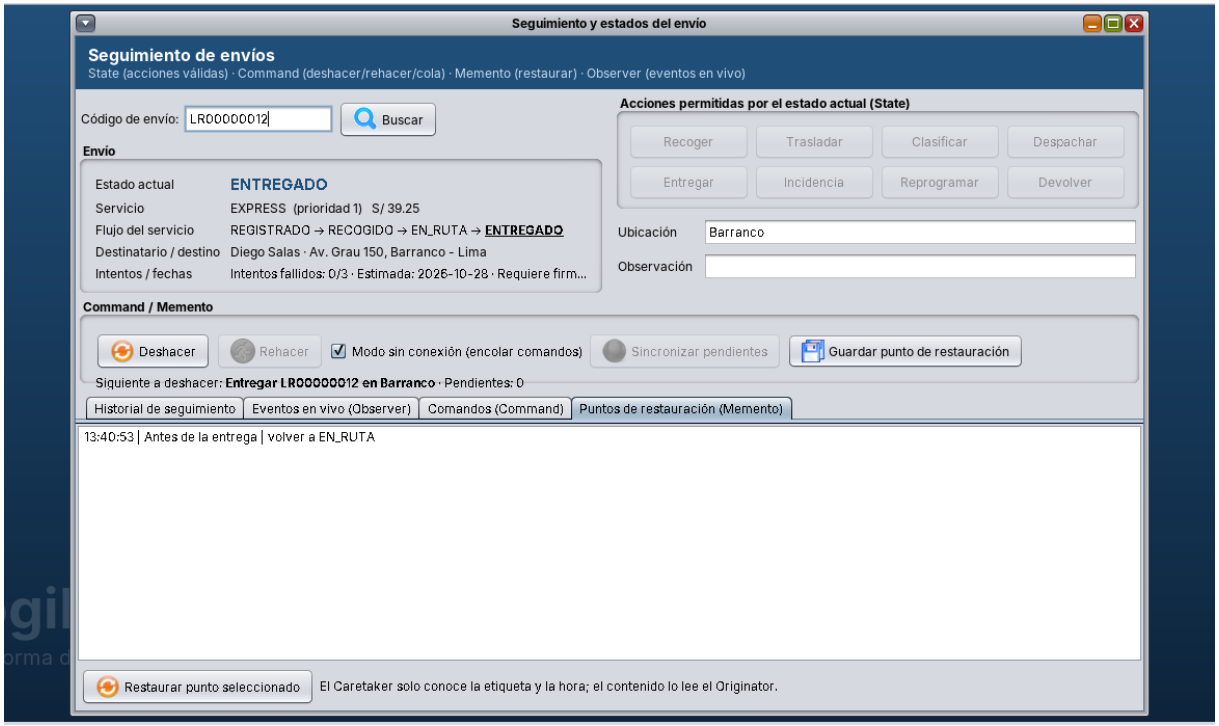
Pruebas

Prueba por consola: cancelar una reprogramación y proteger la reversión de una entrega.

```
   [WA-API] +51999888777 template=estado_envio body="LogiRuta LR00000001 - Recogimos su
paquete."
   [WA-API] +51999888777 template=estado_envio body="LogiRuta LR00000001 - Su paquete está
en ruta y llegará hoy."
Estado actual: OBSERVADO (paso 4, intentos 1)
Caretaker guarda: 19:10:22 | Antes de reprogramar | volver a OBSERVADO
Tras reprogramar: REPROGRAMADO (paso 4, intentos 1) fechaProgramada=2026-10-29
El cliente cancela la reprogramación -> restaurar último memento
Restaurado: OBSERVADO (paso 4, intentos 1) fechaProgramada=null
Deshacer una entrega con firma a través del Proxy:
[AUDITORÍA] E0010 (REPARTIDOR) -> ENTREGAR sobre LR00000002 : PERMITIDO
   Repartidor entrega -> ENTREGADO (paso 5, intentos 0)
[AUDITORÍA] E0010 (REPARTIDOR) -> RESTAURAR(ENTREGADO) sobre LR00000002 : DENEGADO
   Repartidor intenta deshacer: RECHAZADO -> Solo un supervisor puede revertir una entrega
confirmada.
[AUDITORÍA] E0004 (SUPERVISOR_RUTA) -> RESTAURAR sobre LR00000002 : PERMITIDO
   Supervisor deshace -> EN_RUTA (paso 4, intentos 0)
   Último registro del historial: EN_RUTA      | Sistema               | REVERSIÓN:
deshacer "Entregar LR00000002 en Barranco"
```

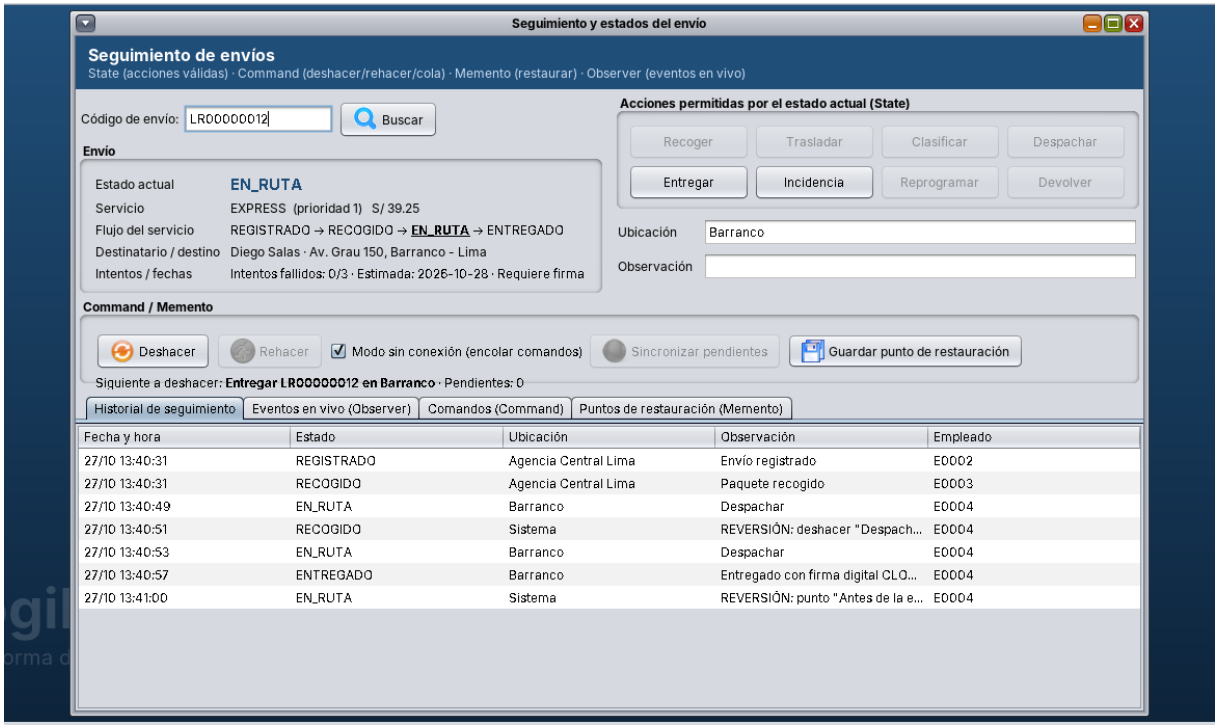
Figura 39

Pestaña "Puntos de restauración (Memento)": el Caretaker solo muestra etiqueta, hora y estado de retorno



Nota. Captura de AppLogistica.

Figura 40
Restauración del punto "Antes de la entrega": el envío vuelve a EN_RUTA y queda el registro de REVERSIÓN



Nota. Captura de AppLogistica.

Tabla 30
Casos de prueba del patrón Memento

Caso	Entrada	Resultado esperado	Resultado obtenido
CP-MEM-01	Guardar punto en OBSERVADO, reprogramar y restaurar	OBSERVADO, mismos intentos y fecha programada nula	Conforme
CP-MEM-02	Repartidor intenta deshacer una entrega	Proxy rechaza: "Solo un supervisor puede revertir..."	Conforme
CP-MEM-03	Supervisor deshace la entrega	EN_RUTA, firma eliminada y registro de reversión	Conforme
CP-MEM-04	Restaurar memento de otro envío	IllegalArgumentException "pertenece a otro envío"	Conforme
CP-MEM-05	Más de 10 puntos para un envío	El Caretaker descarta el más antiguo	Conforme

Nota. Elaboración propia del grupo de trabajo.

6.5 Aplicación complementaria: State y Observer en el empleado (ejercicio de la Sesión 12)

En la Sesión 12 se aplicaron State y Observer a la clase **Empleado** (estados laborales que cambian el comportamiento y áreas que son notificadas ante un cambio). Se trasladó ese ejercicio al caso de LogiRuta, donde tiene un impacto operativo directo.

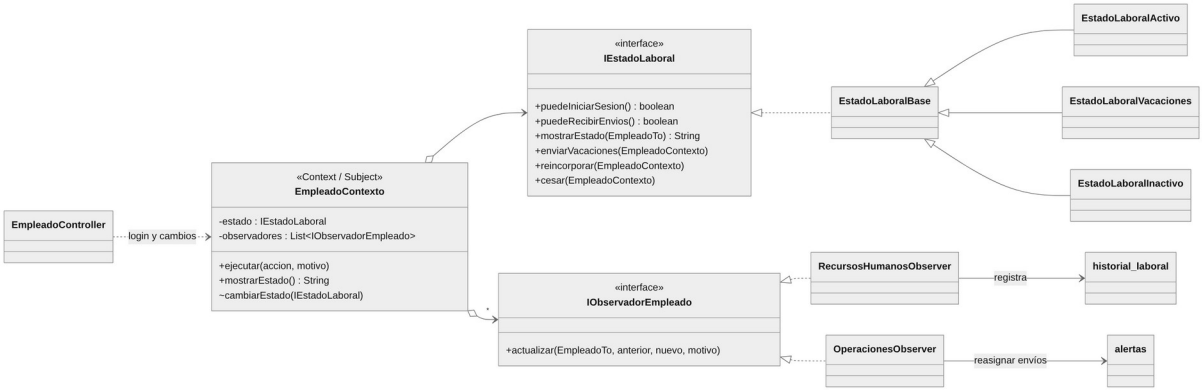
Escenario de aplicación

¿Qué problema tengo? Un repartidor o un operador puede estar **activo**, **de vacaciones** o **inactivo** (cesado). Hasta el Avance 1 solo existía la columna `activo` (sí/no): un repartidor de vacaciones podía iniciar sesión, aparecía en la lista de repartidores al armar un manifiesto y, si tenía envíos asignados en ruta, nadie se enteraba de que debían reasignarse. Además, el área de Recursos Humanos necesitaba un historial de esos cambios.

¿Por qué State y Observer lo resuelven? Con **State**, cada estado laboral es una clase que decide si el empleado puede iniciar sesión (`puedeIniciarSesion()`), si puede recibir envíos (`puedeRecibirEnvios()`), cómo se describe (`mostrarEstado()`, el mismo método del ejercicio de clase) y a qué estados puede pasar (por ejemplo, no se puede cesar a alguien que está de vacaciones sin reincorporarlo antes). Con **Observer**, el `EmpleadoContexto` (sujeto) notifica el cambio a `RecursosHumanosObserver`, que guarda el estado y el historial laboral, y a `OperacionesObserver`, que genera una alerta por cada envío pendiente del repartidor para que el supervisor lo reasigne.

Diagrama UML

Figura 41
State y Observer aplicados al estado laboral del empleado



Nota. Elaboración propia del grupo de trabajo.

Implementación

Código 70
uni.service.IEstadoLaboral – State del empleado

```
/**
 * PATRÓN STATE aplicado al empleado (ejercicio de la Sesión 12, adaptado a
 * LogiRuta): el estado laboral decide si el trabajador puede ingresar al
 * sistema, si puede recibir envíos asignados y a qué estados puede pasar.
 */
public interface IEstadoLaboral {

    String getCodigo();

    boolean puedeIniciarSesion();

    boolean puedeRecibirEnvios();

    /** Comportamiento que cambia según el estado (mostrarEstado() de la clase). */
    String mostrarEstado(EmpleadoTo e);

    void enviarVacaciones(EmpleadoContexto c);

    void reincorporar(EmpleadoContexto c);

    void cesar(EmpleadoContexto c);

    /** Acciones válidas: VACACIONES, REINCORPORAR, CESAR. */
    List<String> accionesPermitidas();
}
```

Código 71
uni.service.EstadoLaboralActivo y EstadoLaboralVacaciones – estados concretos (fragmento)

```
public class EstadoLaboralActivo extends EstadoLaboralBase {

    @Override
    public String getCodigo() { return "ACTIVO"; }

    @Override
    public boolean puedeIniciarSesion() { return true; }

    @Override
    public boolean puedeRecibirEnvios() { return true; }

    @Override
    public String mostrarEstado(EmpleadoTo e) {
        return e.getNombreCompleto() + " está ACTIVO: puede operar y recibir envíos
asignados.";
    }

    @Override
    public void enviarVacaciones(EmpleadoContexto c) { c.cambiarEstado(new
EstadoLaboralVacaciones()); }

    @Override
    public void cesar(EmpleadoContexto c) { c.cambiarEstado(new EstadoLaboralInactivo()); }

    @Override
    public List<String> accionesPermitidas() { return Arrays.asList("VACACIONES",
"CESAR"); }
}

public class EstadoLaboralVacaciones extends EstadoLaboralBase {

    @Override
    public String getCodigo() { return "VACACIONES"; }

    @Override
    public boolean puedeIniciarSesion() { return false; }

    @Override
    public boolean puedeRecibirEnvios() { return false; }
```

```
    @Override
    public String mostrarEstado(EmpleadoTo e) {
        return e.getNombreCompleto() + " está DE VACACIONES: sin acceso al sistema ni
asignación de envíos.";
    }

    @Override
    public void reincorporar(EmpleadoContexto c) { c.cambiarEstado(new
EstadoLaboralActivo()); }

    @Override
    public List<String> accionesPermitidas() { return Arrays.asList("REINCORPORAR"); }
}
```

Código 72

uni.service.EmpleadoContexto – Context y Subject (fragmento)

```
public class EmpleadoContexto {

    private final EmpleadoTo empleado;
    private IEstadoLaboral estado;
    private final List<IObservadorEmpleado> observadores = new ArrayList<>();
    private String motivo;

    public EmpleadoContexto(EmpleadoTo empleado) {
        this.empleado = empleado;
        this.estado = EstadoLaboralBase.de(empleado.getEstadoLaboral());
    }

    // ------------------------------------------------------------ State
    public void ejecutar(String accion, String motivo) {
        this.motivo = motivo;
        switch (accion) {
            case "VACACIONES":   estado.enviarVacaciones(this); break;
            case "REINCORPORAR": estado.reincorporar(this); break;
            case "CESAR":        estado.cesar(this); break;
            default: throw new IllegalArgumentException("Acción laboral desconocida: " +
accion);
        }
    }

    void cambiarEstado(IEstadoLaboral nuevo) {
        String anterior = estado.getCodigo();
        estado = nuevo;
        empleado.setEstadoLaboral(nuevo.getCodigo());
        empleado.setActivo(!"INACTIVO".equals(nuevo.getCodigo()));
        for (IObservadorEmpleado o : new ArrayList<>(observadores)) {
            o.actualizar(empleado, anterior, nuevo.getCodigo(), motivo);
        }
    }
    // ...
}
```

Código 73
uni.service.OperacionesObserver – reasignación de envíos (fragmento)

```
    @Override
    public void actualizar(EmpleadoTo e, String anterior, String nuevo, String motivo) {
        if (e.getRol() != RolUsuario.REPARTIDOR || "ACTIVO".equals(nuevo)) {
            return;
        }
        try {
            List<String> pendientes = asignaciones.enviosPendientesDe(e.getIdempleado());
            for (String codigo : pendientes) {
                alertas.registrar(new AlertaTo(codigo, "ALTA", "Reasignar: el repartidor "
                        + e.getNombreCompleto() + " pasó a " + nuevo + "."));
            }
            System.out.println("   [Operaciones] " + pendientes.size() + " envío(s) por
reasignar de "
                    + e.getIdempleado() + ": " + pendientes);
        } catch (Exception ex) {
            System.out.println("   [Operaciones] " + ex.getMessage());
        }
    }
```

Código 74
uni.controller.EmpleadoController – el login consulta al estado (fragmento)

```
    public boolean EmpleadoValidar(String usuario, String clave) throws Exception {
        EmpleadoTo e = dao.validar(usuario, clave);
        if (e == null) {
            return false;
        }
        // PATRÓN STATE: el estado laboral decide si puede ingresar
        uni.service.IEstadoLaboral estado =
uni.service.EstadoLaboralBase.de(e.getEstadoLaboral());
        if (!estado.puedeIniciarSesion()) {
            throw new SecurityException(estado.mostrarEstado(e));
        }
        SesionUsuario.getInstancia().iniciar(e);
        return true;
    }
```

En la base de datos se agregó la columna `empleados.estado_laboral` y la tabla `historial_laboral`; la lista de repartidores del manifiesto solo muestra a los que están en estado ACTIVO.

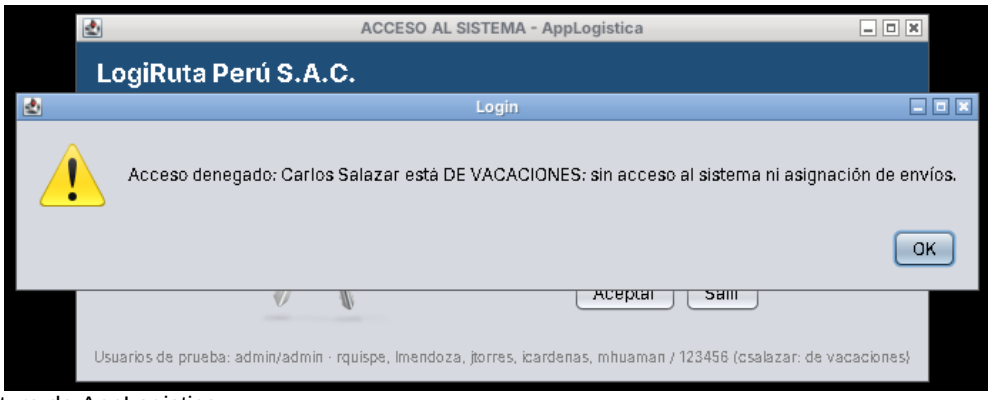
Pruebas

Prueba por consola (`uni.util.PruebaComportamiento`):

```
mostrarEstado(): Iván Cárdenas está ACTIVO: puede operar y recibir envíos asignados.
Acciones permitidas: [VACACIONES, CESAR]
Enviar de vacaciones:
   [RR.HH.] E0010: ACTIVO -> VACACIONES (Vacaciones programadas)
   [Operaciones] 2 envío(s) por reasignar de E0010: [LR00000004, LR00000005]
   Iván Cárdenas está DE VACACIONES: sin acceso al sistema ni asignación de envíos.
   ¿Puede iniciar sesión? false | ¿Puede recibir envíos? false
   Alertas de Operaciones: [[ALTA] LR00000004 - Reasignar: el repartidor Iván Cárdenas pasó
a VACACIONES., [ALTA] LR00000005 - Reasignar: el repartidor Iván Cárdenas pasó a
VACACIONES.]
Cesar estando de vacaciones: RECHAZADO -> No se puede cesar: el empleado está VACACIONES.
Reincorporar:
   [RR.HH.] E0010: VACACIONES -> ACTIVO (Fin de vacaciones)
   Iván Cárdenas está ACTIVO: puede operar y recibir envíos asignados.
```

Figura 42

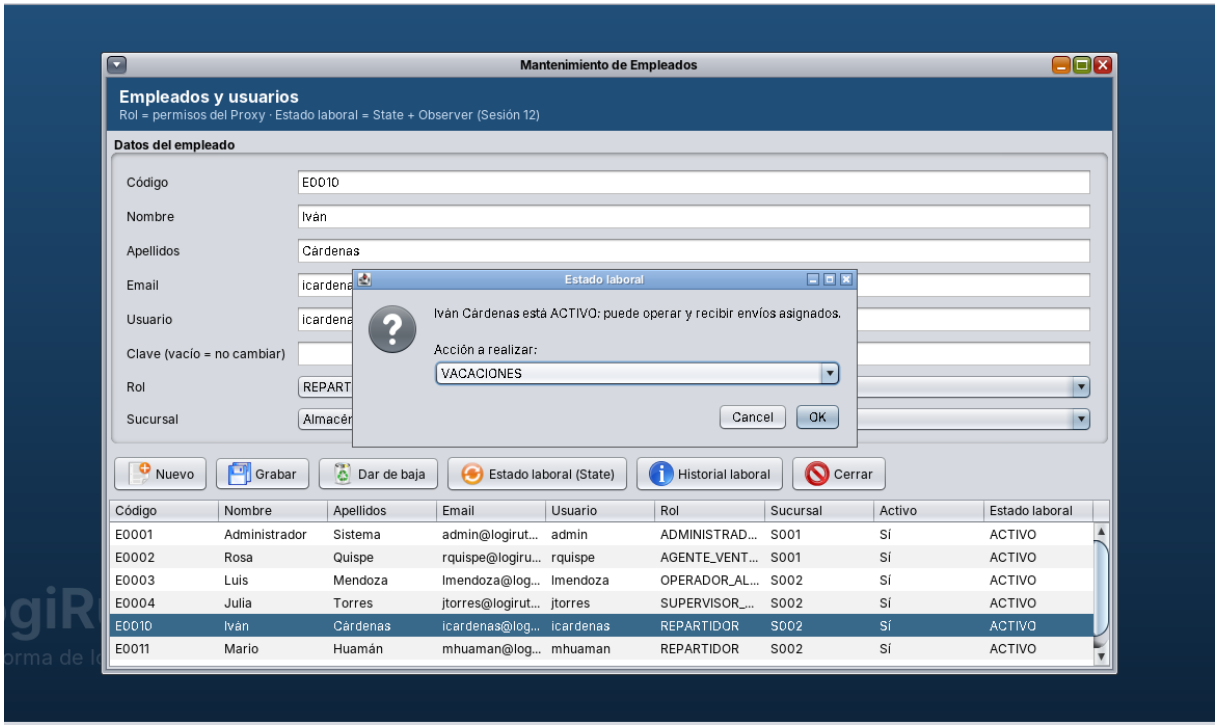
El estado VACACIONES impide el ingreso del usuario csalazar



Nota. Captura de AppLogistica.

Figura 43

EmpleadoView: el estado ACTIVO solo ofrece las acciones VACACIONES y CESAR



Nota. Captura de AppLogistica. Al confirmar se notifica a Recursos Humanos y a Operaciones.

Tabla 31

Casos de prueba de State y Observer en el empleado

Caso	Entrada	Resultado esperado	Resultado obtenido
CP-EMP-01	Login de csalazar (VACACIONES)	Acceso denegado con el texto de mostrarEstado()	Conforme
CP-EMP-02	E0010 ACTIVO → VACACIONES	Historial laboral + 2 alertas de reasignación (LR00000004, LR00000005)	Conforme
CP-EMP-03	Armar manifiesto con E0010 de vacaciones	E0010 no aparece en la lista de repartidores	Conforme
CP-EMP-04	Cesar a un empleado en VACACIONES	Rechazo: "No se puede cesar: el empleado está VACACIONES"	Conforme

Caso	Entrada	Resultado esperado	Resultado obtenido
CP-EMP-05	Reincorporar a E0010	ACTIVO, puede ingresar y recibir envíos	Conforme

Nota. Elaboración propia del grupo de trabajo.

Comparación de patrones de comportamiento

La Tabla 32 resume la intención de cada patrón, el problema concreto de LogiRuta que resolvió y el principio SOLID que refuerza.

Tabla 32
Comparación de los patrones de comportamiento aplicados en AppLogistica

Patrón	Intención	Problema resuelto	Clases clave	SOLID
Observer	Notificar a muchos objetos cuando uno cambia	Reacciones múltiples ante cada cambio de estado	IObservadorEnvio, EnvioContexto	OCP, SRP, DIP
State	Cambiar el comportamiento según el estado interno	Transiciones válidas por estado y servicio	IEstadoEnvio, 9 estados	OCP, SRP, LSP
Command	Encapsular una petición como objeto	Deshacer, cola sin conexión, macro y bitácora	IComando, GestorComandos	SRP, OCP
Memento	Guardar y restaurar el estado sin romper el encapsulamiento	Volver atrás una reprogramación o entrega	EnvioMemento, HistorialMementos	SRP, encapsulamiento

Nota. Elaboración propia del grupo de trabajo.

Relación con los capítulos anteriores. Los patrones de comportamiento se apoyan en los ya implementados: el observador de notificación reutiliza los **Adapter** creados por la **Abstract Factory**; el flujo de estados lo aporta la **Factory** de servicios; los comandos pasan siempre por el **Proxy** de seguridad; el **MacroComando** recorre el árbol **Composite** del manifiesto, y `ConfiguracionSistema` (**Singleton**) arma todas las dependencias inyectándolas por interfaces (**DIP**).

Diferencias entre patrones parecidos. State y Strategy comparten estructura, pero en State es el propio estado el que provoca la transición al siguiente, mientras que en Strategy el cliente elige el algoritmo. Command y Memento cooperan: Command define *qué* se deshace y Memento *cómo* se recupera el estado anterior. Observer y Facade también difieren: la fachada coordina subsistemas de forma explícita y sincrónica dentro de un método, mientras que en Observer el sujeto desconoce quién reacciona.

Tabla 33
Trazabilidad entre casos de uso y patrones de comportamiento

Caso de uso	Observer	State	Command	Memento
CU04 Clasificar y consolidar carga	Historial, notificación	CLASIFICADO → EN_RUTA	MacroComando del manifiesto	Respaldo por paquete
CU05 Actualizar estado	Persistencia, historial, alertas	Todas las transiciones	Comandos con deshacer y cola	Deshacer
CU06 Consultar seguimiento	Pantalla en vivo	Acciones habilitadas	---	---
CU07 Gestionar incidencias	Alertas CRITICA/ALTA	OBSERVADO, REPROGRAMADO, DEVUELTO	Incidencia, Reprogramar	Cancelar reprogramación
CU08 Consultar reportes	Indicadores en tiempo real	---	Bitácora comandos_log	---

Nota. Elaboración propia del grupo de trabajo.

Avance hacia la siguiente entrega. Con los patrones de comportamiento implementados, el Capítulo VII (semana 14) analizará los antipatrones que estos patrones eliminaron del diseño original —por ejemplo, el *God Object* que habría sido `SeguimientoServiceImpl` y el *Spaghetti Code* de los `switch` de transición— y el Capítulo VIII aplicará los patrones GRASP (Unidad 5).

CAPÍTULO VII. ANTIPATRONES IDENTIFICADOS

> **En desarrollo para futuras entregas.**

7.1 Concepto

En desarrollo para futuras entregas.

7.2 Antipatrones encontrados

En desarrollo para futuras entregas.

7.3 Solución propuesta

En desarrollo para futuras entregas.

CAPÍTULO VIII. PATRONES GRASP

> **En desarrollo para futuras entregas.** Este capítulo corresponde a la Unidad 5 del curso (Experto en Información, Creador, Controlador, Bajo Acoplamiento, Alta Cohesión, Fabricación Pura y Polimorfismo), con evidencias UML y justificación técnica.

8.1 Experto en Información

En desarrollo para futuras entregas.

8.2 Creador

En desarrollo para futuras entregas.

8.3 Controlador

En desarrollo para futuras entregas.

8.4 Bajo Acoplamiento

En desarrollo para futuras entregas.

8.5 Alta Cohesión

En desarrollo para futuras entregas.

8.6 Fabricación Pura

En desarrollo para futuras entregas.

8.7 Polimorfismo

En desarrollo para futuras entregas.

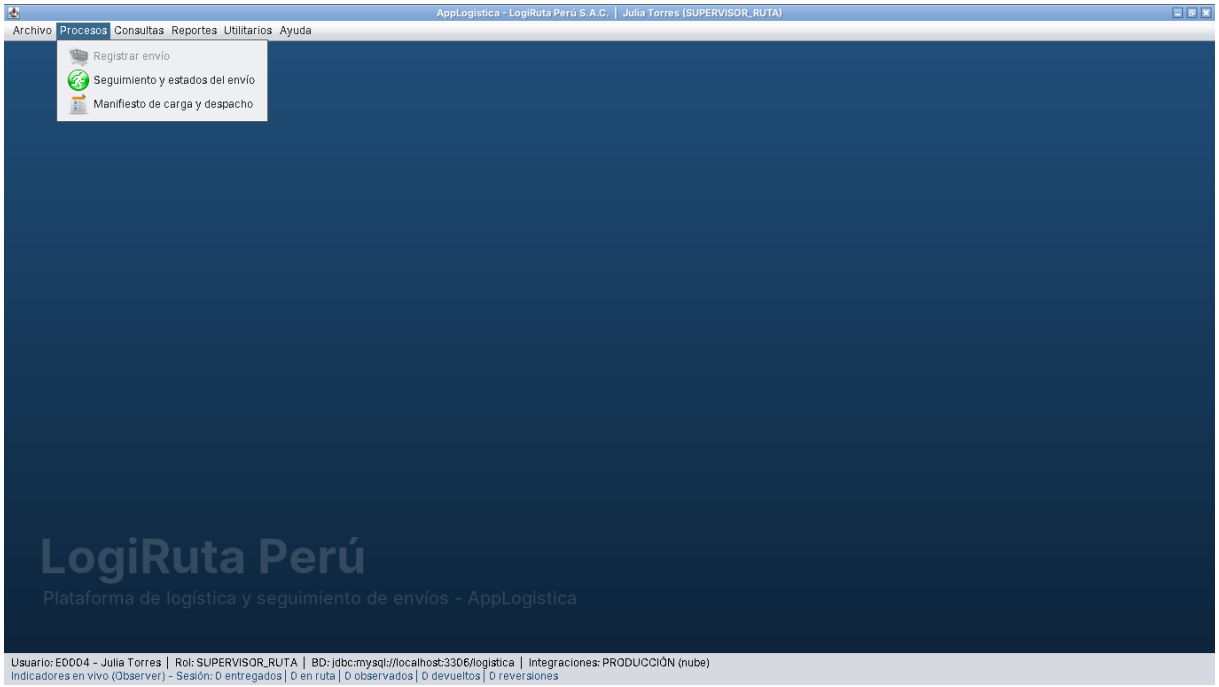
CAPÍTULO IX. IMPLEMENTACIÓN DEL SISTEMA

> **En desarrollo para futuras entregas.** Se adelanta en esta entrega un **avance** de la arquitectura, de la aplicación funcional y de la base de datos, porque sustentan el código presentado en los capítulos III al VI. Los diagramas de clases y de secuencia completos y el diccionario de datos definitivo se presentarán en la entrega final.

9.1 Arquitectura propuesta

Avance: la arquitectura en tres capas (Presentación, Negocio y Datos) y su correspondencia con los paquetes de AppEmpresa se presentó en la Figura 4 del Capítulo III. En este avance la capa de presentación ya está implementada en `uni.formularios`: `LoginView` (clase principal del proyecto) valida al usuario con `EmpleadoController` y abre `PrincipalView`, que organiza los formularios internos en los menús Archivo, Procesos, Consultas, Reportes, Utilitarios y Ayuda, igual que AppEmpresa. Cada opción se habilita según el rol del usuario en sesión.

Figura 44
Ventana principal de AppLogistica (menú Procesos) con la barra de estado del usuario y los indicadores en vivo



Nota. Captura de AppLogistica ejecutándose contra MySQL 8.

9.2 Diagrama de clases

En desarrollo para futuras entregas.

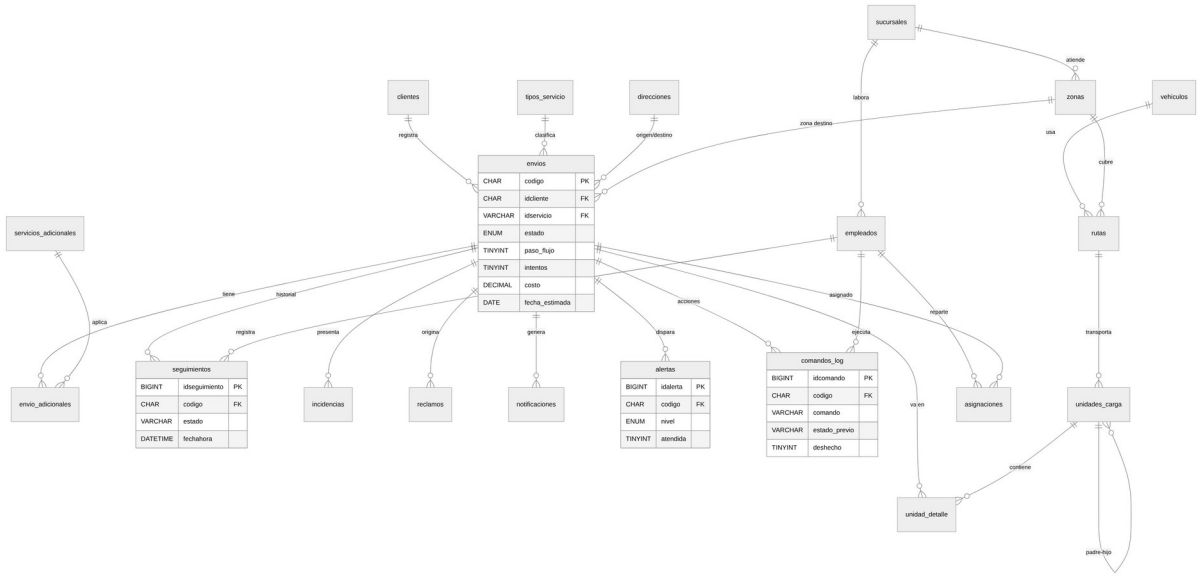
9.3 Diagrama de secuencia

En desarrollo para futuras entregas. Como avance, la Figura 16 del Capítulo V muestra la secuencia de la operación de registro de envíos.

9.4 Base de datos

Avance. La base de datos logistica se diseñó en MySQL 8 siguiendo el mismo molde del script de AppEmpresa: recreación de la estructura, tabla control de correlativos (*Envios*, *Clientes*, *Empleados*, *Manifiestos*, *Reclamos*), datos de prueba, procedimientos almacenados con prefijo sp_ (varios con parámetro de salida, como USP_PROVEEDOR_INSERTA), triggers que garantizan la inmutabilidad del historial y vistas para los reportes estratégicos. En el Avance 2 se agregaron las columnas `paso_flujo` e `intentos` en `envios` (patrón State), la tabla `alertas` (Observer), la tabla `comandos_log` (Command/Memento), la columna `estado_laboral` y la tabla `historial_laboral` (State y Observer del empleado), la vista `vw_productividad_reparto` y el usuario de conexión `logistica`. Las claves de los empleados se guardan con hash SHA-256 (RNF01). El script completo se entrega en el archivo base_Datos_logistica.sql.

Figura 45
Modelo lógico de la base de datos logistica (actualizado al Avance 2)



Nota. Elaboración propia del grupo de trabajo.

Tabla 34
Tablas principales de la base de datos (avance del diccionario de datos)

Tabla	Descripción	Patrón o principio relacionado
`control`	Correlativos de códigos (molde AppEmpresa)	—
`clientes`, `empleados`, `sucursales`, `zonas`	Maestros del negocio; `empleados.rol` define permisos	Proxy (protección por rol)

Tabla	Descripción	Patrón o principio relacionado
`tipos_servicio`	Parámetros de cada servicio: cargo fijo, costo por kg, días, prioridad, peso máximo	Factory, OCP
`servicios_adicionales`, `envio_adicionales`	Adicionales y su monto por envío	Decorator
`direcciones`	Direcciones con latitud y longitud	Adapter (geolocalización)
`envios`	Envío con su estado actual, costo y fechas estimada y real	Builder, Prototype
`unidades_carga`, `unidad_detalle`	Sacos, paletas y manifiestos con auto-relación `idpadre`	Composite
`seguimientos`	Historial inmutable de estados (triggers)	ISP
`asignaciones`, `auditoria`	Envíos asignados a repartidores y bitácora de accesos	Proxy
`proveedores_integracion`, `notificaciones`	Familias de proveedores y registro de mensajes	Abstract Factory, Adapter
`incidencias`, `reclamos`	Incidencias de entrega y reclamos de clientes	—
`vw_incidencias_zona`, `vw_cumplimiento_tiempos`	Vistas para reportes estratégicos	Bridge
`alertas`	Alertas para supervisores (express observado, 3.er intento, devolución)	Observer
`comandos_log`	Bitácora de comandos: estado previo, estado nuevo, empleado y marca de deshecho	Command, Memento
`envios.paso_flujo`, `envios.intentos`	Parte persistente del estado del envío	State
`vw_productividad_reparto`	Entregas y observados por repartidor	Bridge, Observer
`empleados.estado_laboral`, `historial_laboral`	Estado laboral del empleado y su historial	State, Observer (Sesión 12)

CAPÍTULO X. PRUEBAS DEL SISTEMA

> **En desarrollo para futuras entregas.** Las evidencias de cada patrón se incluyen en los capítulos III al VI. Como **avance**, se presentan los casos de prueba funcionales ejecutados sobre la aplicación de escritorio conectada a MySQL con los usuarios de prueba del script.

10.1 Casos de prueba

Tabla 35
Casos de prueba funcionales ejecutados sobre AppLogistica (avance)

Caso	Entrada	Resultado esperado	Resultado obtenido
CP01	Login admin / admin	Acceso a PrincipalView con todas las opciones	Conforme
CP02	Login admin / clave incorrecta	Mensaje "Acceso denegado..."	Conforme
CP03	Registrar cliente natural con DNI de 7 dígitos	Validación: requiere DNI de 8 dígitos	Conforme
CP04	Eliminar cliente con envíos (C0005)	Rechazo del procedimiento: "El cliente tiene envíos registrados"	Conforme
CP05	Cotizar EXPRESS 2.5 kg + seguro S/ 400 + firma	S/ 39.25 con detalle de adicionales (Decorator)	Conforme
CP06	Registrar el envío anterior	Código LR correlativo, geolocalización y notificación (Facade)	Conforme
CP07	Repartidor icardenas abre LR00000008 (no asignado)	Solo consulta; aviso del Proxy	Conforme
CP08	Supervisor despacha manifiesto de 3 envíos	Ruta creada, asignaciones y 3 envíos EN_RUTA	Conforme
CP09	Manifiesto que supera la capacidad del vehículo	Mensaje de capacidad excedida	Conforme
CP10	Reporte de cumplimiento en formato HTML	Tabla HTML generada por el Bridge	Conforme
CP11	Exportar reporte a CSV	Archivo .csv separado por ";"	Conforme
CP12	Rol AGENTE_VENTANILLA	Menús de Reportes, Manifiesto y Empleados deshabilitados	Conforme

Nota. Elaboración propia del grupo de trabajo.

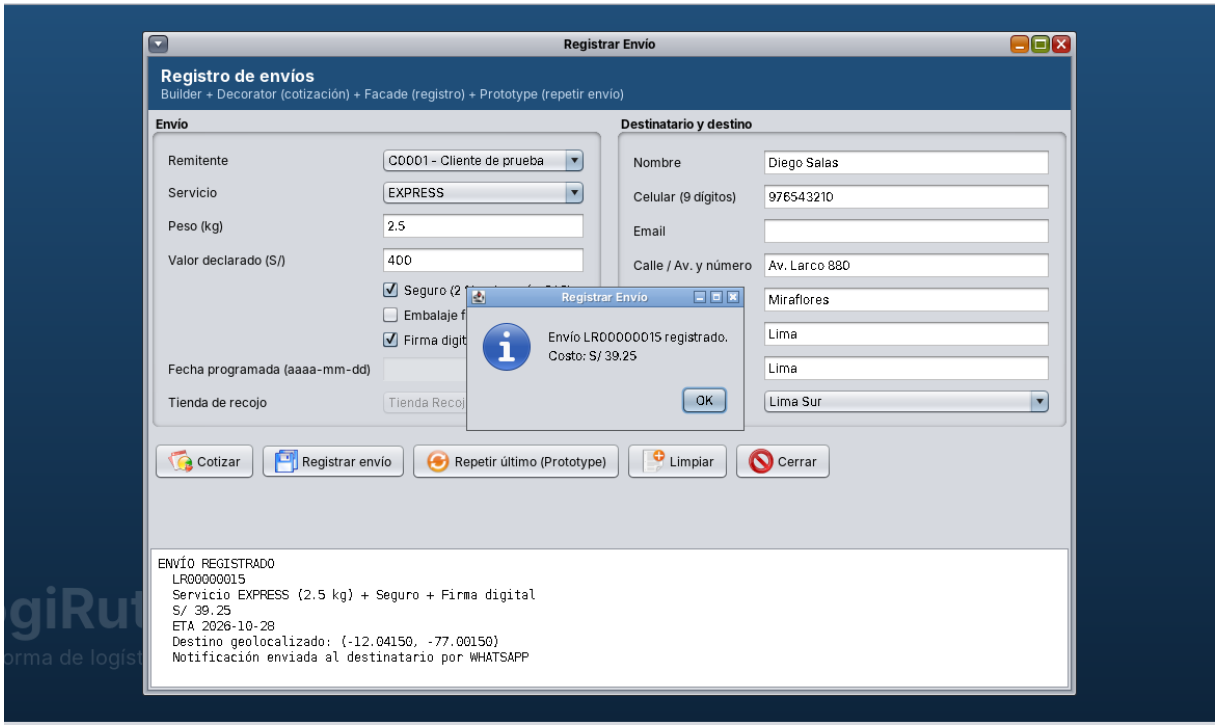
10.2 Evidencias

Figura 46
Ventana de acceso LoginView (molde AppEmpresa)



Nota. Captura de AppLogistica.

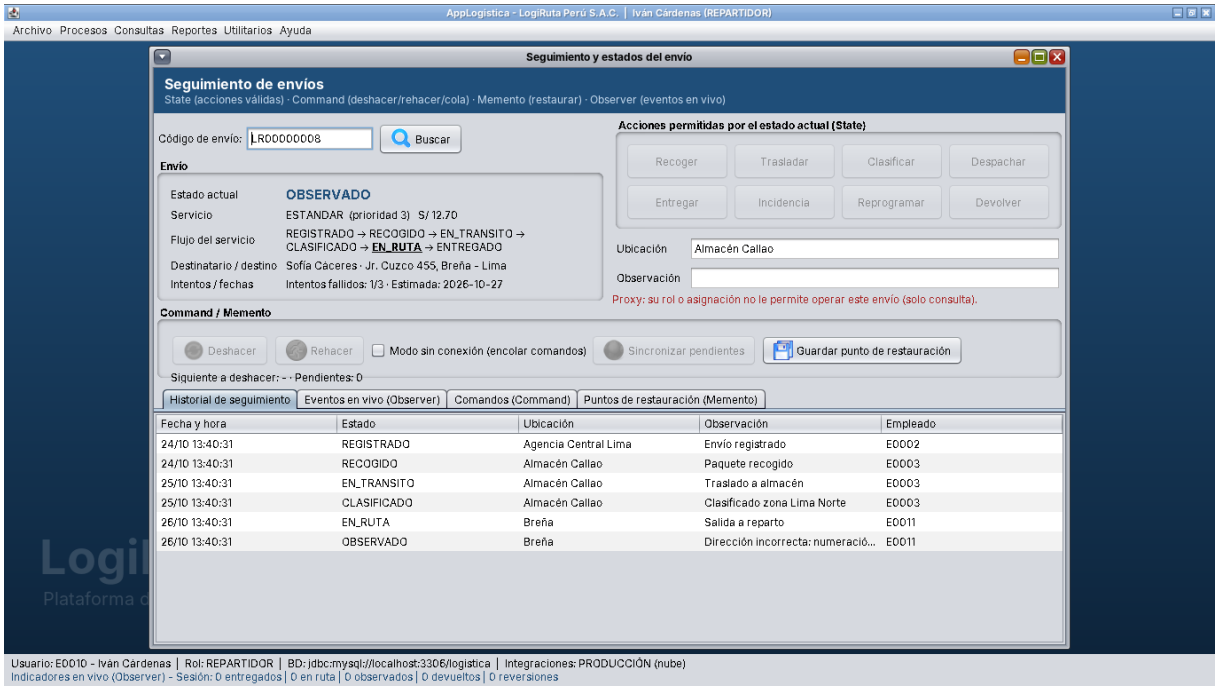
Figura 47
Registro de envío (Builder + Decorator + Facade): el envío queda registrado, geolocalizado y notificado



Nota. Captura de AppLogistica.

Figura 48

Proxy de protección: el repartidor E0010 solo puede consultar un envío que no tiene asignado



Nota. Captura de AppLogistica.

Figura 49

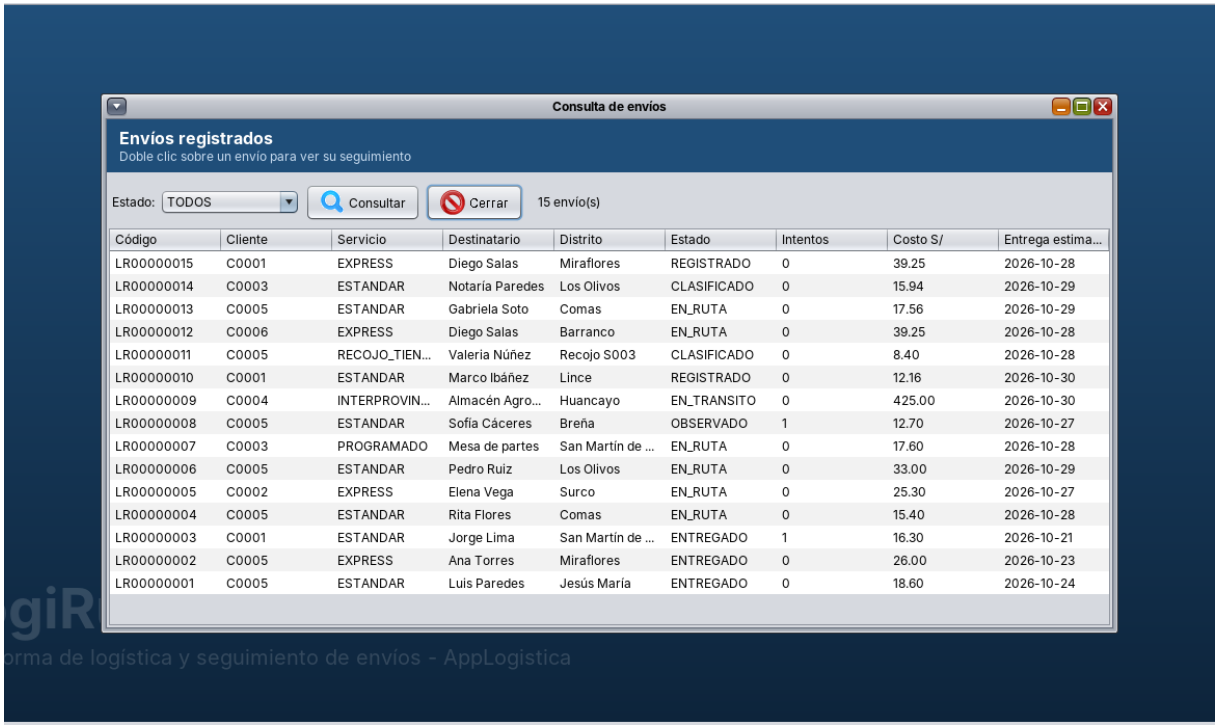
Reporte de cumplimiento de tiempos en formato HTML (Bridge)



Nota. Captura de AppLogistica.

Figura 50

Consulta de envíos por estado (doble clic abre el seguimiento)



Nota. Captura de AppLogistica.

CAPÍTULO XI. CONCLUSIONES

En desarrollo para futuras entregas.

CAPÍTULO XII. RECOMENDACIONES

En desarrollo para futuras entregas.

REFERENCIAS BIBLIOGRÁFICAS

Freeman, E., & Robson, E. (2020). *Head first design patterns: Building extensible and maintainable object-oriented software* (2.ª ed.). O'Reilly Media.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Larman, C. (2004). *Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development* (3.ª ed.). Prentice Hall.

Martin, R. C. (2003). *Agile software development: Principles, patterns, and practices*. Prentice Hall.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Oracle. (2024). *JDBC basics: Using stored procedures*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/jdbc/basics/storedprocedures.html

Oracle. (2024). *Creating a GUI with Swing*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/

Shvets, A. (2021). *Sumérgete en los patrones de diseño*. Refactoring.Guru. https://refactoring.guru/es/design-patterns

Universidad Tecnológica del Perú. (2026a). *Diseño de Patrones: Sesión 12 – Patrones de comportamiento: State y Observer* [Material de clase]. Facultad de Ingeniería.

Universidad Tecnológica del Perú. (2026b). *Sílabo del curso Diseño de Patrones (100000SI47), ciclo 2026-II*. Facultad de Ingeniería.

Universidad Tecnológica del Perú. (2026c). *Diseño de Patrones: Sesiones 06 a 11 – Patrones creacionales y estructurales* [Material de clase]. Facultad de Ingeniería.

ANEXOS

> **En desarrollo para futuras entregas** (manual de usuario completo y capturas de todas las pantallas). Como avance, junto con este informe se entregan: el proyecto NetBeans **AppLogistica** (ejecutable), el script **base_Datos_logistica.sql**, el archivo **diagramas_uml.md** con el código Mermaid de todos los diagramas y el archivo **salida_pruebas.txt** con la ejecución de las pruebas.

Anexo A. Guía rápida de instalación y ejecución

- Requisitos: JDK 17 o superior, Apache NetBeans 17+ y MySQL Server 8 (con MySQL Workbench).
- Abrir MySQL Workbench, cargar **base_Datos_logistica.sql** y ejecutarlo completo (rayo). El script crea la base **logistica**, el usuario de conexión `logistica` / `logistica`, las tablas, procedimientos, triggers, vistas y datos de prueba. Puede ejecutarse de nuevo para reiniciar los datos.
- En NetBeans: File > Open Project y seleccionar la carpeta **AppLogistica**. El driver JDBC ya está en la carpeta `lib` y referenciado en el proyecto.
- Si se usa otro usuario o puerto de MySQL, editar el archivo `db.properties` de la carpeta del proyecto. La clave `integraciones=CONTINGENCIA` cambia la familia de proveedores (Abstract Factory).
- Probar la conexión con clic derecho sobre `uni.database.prueba` > Run File y luego ejecutar el proyecto (F6): se abre `LoginView`.
- Las pruebas por consola de los patrones se ejecutan con Run File sobre `uni.util.PruebaPatrones` (capítulos III a VI) o `uni.util.PruebaComportamiento` (Capítulo VI); no requieren MySQL.

Tabla 36
Usuarios de prueba creados por el script de base de datos

Usuario	Clave	Rol	Para probar
admin	admin	ADMINISTRADOR	Todas las opciones
rquispe	123456	AGENTE_VENTANILLA	Clientes y registro de envíos
lmendoza	123456	OPERADOR_ALMACEN	Recoger, clasificar, manifiestos
jtorres	123456	SUPERVISOR_RUTA	Manifiestos, reportes, alertas, revertir entregas
icardenas	123456	REPARTIDOR	Envíos asignados LR00000004 y LR00000005
mhuaman	123456	REPARTIDOR	Envío asignado LR00000008
csalazar	123456	REPARTIDOR	Estado VACACIONES: acceso denegado (State)

Nota. Elaboración propia del grupo de trabajo.